Univerza v Ljubljani
Fakulteta za računalništvo in informatiko
Fakulteta za matematiko in fiziko

Peter Šavron

Zasnova cloud native arhitekture za trajno hrambo vektoriziranih informacij agentnih sistemov

diplomsko delo

interdisciplinarni univerzitetni
študijski program prve stopnje
računalništvo in matematika

Mentor: prof. dr. Branko Matjaž Jurič

Ljubljana, 2026

Copyright. Rezultati diplomske naloge so intelektualna lastnina avtorja in matične fakultete Univerze v Ljubljani. Za objavo in koriščenje rezultatov diplomske naloge je potrebno pisno privoljenje avtorja, fakultete ter mentorja.

Besedilo je oblikovano z urejevalnikom besedil LaTeX.

Kandidat: Peter Šavron

Naslov: Zasnova cloud native arhitekture za trajno hrambo vektoriziranih informacij agentnih sistemov

Vrsta naloge: Diplomska naloga na interdisciplinarnem univerzitetnem programu prve stopnje Računalništvo in matematika

Mentor: prof. dr. Branko Matjaž Jurič

Opis:

Kandidat naj preuči koncepte vektorskih podatkovnih baz s poudarkom na vektorskih kazalih, postopkih kvantizacije in filtriranih poizvedbah ter primerja funkcionalnosti dveh razširjenih rešitev, Milvus in Qdrant. Nato naj pregleda delovanje agentov, ki temeljijo na velikih jezikovnih modelih, in analizira, kako lahko trajna hramba vloženih podatkov zmanjša porabo kontekstnega okna. Na tej podlagi naj zasnuje oblačno arhitekturo mikrostoritve, ki agentom in drugim storitvam prek enotnega vmesnika omogoča vnos in semantično iskanje podatkov, pri čemer naj posebno pozornost nameni večnajemniški podatkovni shemi, nadzoru dostopa in skladnosti z GDPR. Zasnovo naj potrdi z implementacijo vmesnika. Zmogljivost vektorske podatkovne baze naj empirično ovrednoti pri različnih nastavitvah kazal, drobcev in replik ter postopkov kvantizacije, in sicer glede prepustnosti, latence in priklica, tudi ob omejenih računskih virih. Na podlagi meritev naj oblikuje priporočila za postavitev in nastavitev vektorske podatkovne baze ter oceni stroške njenega vzdrževanja v oblaku.

Title: Design of a cloud native architecture for the persistent storage of vectorized information of agentic systems

Description:

The candidate should study the concepts underlying vector databases, focusing on vector indexes, quantization methods and filtered queries, and compare the capabilities of two widely used systems, Milvus and Qdrant. The work should then review how agents based on large language models operate and analyse how persistent storage of embeddings can reduce their context window usage. On this basis, the candidate should design a cloud-native microservice architecture that allows agents and other services to ingest and semantically search data through a single interface, paying particular attention to a multi-tenant data schema, access control and GDPR compliance. The design should be validated by implementing the interface. The performance of the vector database should be evaluated empirically under different index, shard and replica settings and quantization methods, measuring throughput, latency and recall, including under constrained compute resources. Finally, the candidate should derive recommendations for deploying and configuring a vector database and estimate the cost of operating it in the cloud.

Vsem, ki imajo kaj dobrega v srcu.

Kazalo

Povzetek

Abstract

1 Uvod **1**

2 Osnovni pojmi **5**

2.1 Vektorji 5

2.2 Prekletstvo dimenzionalnosti 6

2.3 Gosti in redki vektorji 7

2.4 Vložitev besedila 8

2.5 Kvantizacija vektorjev 9

2.6 Iskanje najbližjega soseda 12

2.7 Vektorska kazala 13

3 Vektorske podatkovne baze **17**

3.1 Nastavitve kazal nad vektorji 17

3.2 Dodajanje in brisanje zapisov 18

3.3 Filtrirana poizvedba 18

3.4 Hibridno iskanje 19

3.5 Ponovna uvrstitev rezultatov 19

3.6 Varčevanje z infrastrukturnimi viri 20

4 Primerjava funkcionalnosti Milvus in Qdrant VDB **23**

4.1 Arhitektura sistema 24

4.2 Implementacija kazal in kvantizacije 26
4.3 Namembnost 27

5 Agenti **29**
5.1 Protokol MCP 30
5.2 Kontekstno okno 30

6 Arhitektura storitev **33**
6.1 Izpostavljene funkcionalnosti 35
6.2 VDB 36
6.3 Vmesnik VDB 39
6.4 Varnost 40
6.5 Posrednik 42

7 Testiranje nastavitev Qdrant **45**
7.1 Okolje 46
7.2 Spremenljivke 48
7.3 Nastavitve kazala nad metapodatki 49
7.4 Nastavitve gruče Qdrant 55
7.5 Vrste kvantizacije 57
7.6 Vertikalno skaliranje vozlišč 62

8 Analiza rezultatov in priporočila **67**
8.1 Postavitev gruče 67
8.2 Nastavitve zbirke 70
8.3 Raba vmesnika za dostop do VDB 72
8.4 Omejitve in grožnje veljavnosti 73

9 Zaključek **75**
9.1 Nadaljnje delo 76

Literatura **79**

A Konfiguracije in odseki kode **91**
A.1 Končne točke vmesnika VDB 91
A.2 Qdrant 96
A.3 Envoy 97

B Tabele meritev **99**
B.1 Testi nastavitev kazal nad tovorom 99
B.2 Testi nastavitev Qdrant gruče 104
B.3 Testi nastavitev kvantizacije 108
B.4 Testi izkoristka CPU 113
B.5 Testi porabe pomnilnika 117

Slike

2.1 Primer preskočnega seznama (generirano prek UI) 14
2.2 Graf HNSW s tremi sloji (generirano prek UI) 14

6.1 Shema oblačne arhitekture z dvema ločenima gručama Qdrant (lastna shema) 35

7.1 Primerjava prepustnosti pri različnih nastavitvah kazala nad tovorom. 52
7.2 Primerjava P95 latence pri različnih nastavitvah kazala nad tovorom. 53
7.3 Primerjava priklica pri različnih nastavitvah kazala nad tovorom. 54
7.4 Prepustnost različnih nastavitev Qdrant gruče. 56
7.5 Primerjava prepustnosti pri različnih vrstah kvantizacije, samostojno vozlišče 2 CPU in 4 GB RAM. Faktor vzorčenja $v = 2,0$. Vse meritve so na voljo na `https://github.com/peter-savron/vector-db-benchmark`. 59
7.6 Priklic pri različnih vrstah kvantizacije, samostojno vozlišče 2 CPU in 4 GB RAM. Faktor vzorčenja $v = 2,0$. Vse meritve so na voljo na `https://github.com/peter-savron/vector-db-benchmark`. 60
7.7 Primerjava časa gradnje kazala pri različnih vrstah kvantizacije 61
7.8 Primerjava razmerja med prepustnostjo in priklicem več načinov kvantizacije in faktorjev vzorčenja pri 1,2 GB dodeljenega pomnilnika. 64

7.9 Prepustnost v odvisnosti od števila CPU. Faktor vzorčenja $v = 2, 0$. 65
7.10 Primerjava prepustnosti ob različni porabi pomnilnika in različnih faktorjih vzorčenja. 66

Tabele

3.1 Primerjava cen procesorja, pomnilnika, diska in objektne hrambe na OCI [12]. 21

6.1 Pregled polj, na katerih definiramo kazala nad tovorom 39

7.1 Okolje, v katerem smo izvajali teste. 47

B.1 Gruča brez kazala nad tovorom 100
B.2 Gruča s kazalom nad tovorom, brez podgrafa HNSW 101
B.3 Gruča s kazalom nad tovorom, s podgrafom HNSW 102
B.4 Gruča z najemniškim kazalom nad tovorom, s podgrafom HNSW 103
B.5 Gruča, 3 drobci in 1 replika 104
B.6 Gruča, 1 drobec in 3 replike 105
B.7 Samostojno vozlišče, 6 CPU, 12 GB RAM 106
B.8 Gruča, 3 drobci in 3 replike. Vozlišča po 2 CPU in 1,5 GB RAM 107
B.9 Skalarna kvantizacija 109
B.10 Kvantizacija produkta na 2 bita 110
B.11 Rotacijska kvantizaija na 4 bite 111
B.12 Rotacijska kvantizacija na 1 bit 112
B.13 Rotacijska kvantizacija na 4 bite, 4 CPU 114
B.14 Rotacijska kvantizacija na 4 bite, 1 CPU 115
B.15 Rotacijska kvantizacija na 4 bite, 0,5 CPU 116
B.16 Rotacijska kvantizacija na 4 bite, 2,2 GB RAM 118
B.17 Rotacijska kvantizacija na 2 bita, 2,2 GB RAM 119
B.18 Skalarna kvantizacija, 1,2 GB RAM 120

B.19 Rotacijska kvantizacija na 4 bite, 1,2 GB RAM 121
B.20 Rotacijska kvantizacija na 4 bite, 1,2 GB RAM, brez rabe kazala122
B.21 Rotacijska kvantizacija na 2 bita, 1,2 GB RAM 123

Seznam uporabljenih kratic

kratica	angleško	slovensko
DB	database	podatkovna baza
VDB	vector database	vektorska podatkovna baza
BM25	best matching 25	algoritem najboljšega ujemanja 25
BQ	binary quantization	binarna kvantizacija
PQ	product quantization	kvantizacija produkta
RQ	rotational quantization	rotacijska kvantizacija
K-ANNS	k-approximate nearest neighbor search	iskanje k aproksimativnih najbližjih sosedov
ANN	approximate nearest neighbor	aproksimativni najbližji sosed
HNSW	hierarchical navigable small world	hierarhični prehodni mali svetovi
IVF	inverted file index	inverzno datotečno kazalo
RRF	reciprocal rank fusion	združevanje prek medsebojnega ranga
WAL	write-ahead log	dnevnik za predčasno pisanje
CAP	consistency, availability, partition tolerance	konsistenca, razpoložljivost, toleranca na particije
OCI	Oracle cloud infrastructure	oblačna infrastruktura Oracle
REST	representational state transfer	prenos reprezentativnega stanja
API	application programming interface	aplikacijski programski vmesnik
JWT	JSON web token	spletni žeton JSON
RBAC	role based access control	kontrola dostopa na podlagi vlog
FGA	fine grained authorization	natančno določanje pravic do dostopa
GDPR	general data protection regula-	splošna uredba o varstvu po-

Povzetek

Naslov: Zasnova cloud native arhitekture za trajno hrambo vektoriziranih informacij agentnih sistemov

Avtor: Peter Šavron

Povzetek:

V delu obravnavamo problem učinkovite trajne hrambe in iskanja vloženih podatkov za potrebe agentov, ki temeljijo na velikih jezikovnih modelih. Ker neprestano večanje kontekstnega okna poslabša učinkovitost in poveča stroške, predlagamo oblačno arhitekturo, ki temelji na vektorski podatkovni bazi. Prek poznavanja osnovnih konceptov vektorskih podatkovnih baz ter analizo baz Milvus in Qdrant pridobimo znanje, ki nam omogoča zasnovati arhitekturo sistema, ki bo te podatke trajno hranil. Rešitev temelji na večnajemniški arhitekturi s Qdrantom, posrednikom Envoy, vmesnikom Quarkus in sistemom OpenFGA za zagotavljanje varnosti. Na podlagi obsežnega empiričnega testiranja različnih konfiguracij kazal, gruč in metod kvantizacije (kot je rotacijska kvantizacija) pokažemo, da lahko z optimizirano uporabo podgrafov najemnikov in ustreznih vrst kvantizacije bistveno zmanjšamo porabo virov ter izboljšamo prepustnost podatkovne baze, ne da bi pri tem žrtvovali priklic poizvedb.

Ključne besede: vektorska podatkovna baza, vložitev, računalništvo v oblaku, kvantizacija, kazala, vmesniki, MCP, RAG, večnajemništvo, iskanje približnih najbližjih sosedov, merilo.

Abstract

Title: Design of a cloud native architecture for the persistent storage of vectorized information of agentic systems

Author: Peter Šavron

Abstract:

In this work, we address the challenge of efficient persistent storage and retrieval of embedded data for large language model–based agents. Since continuously expanding the context window degrades performance and increases costs, we propose a cloud architecture built on a vector database. By examining the fundamental concepts of vector databases and analyzing Milvus and Qdrant, we establish the foundational knowledge needed to design the system architecture.

Our solution implements a multi-tenant architecture that uses Qdrant, an Envoy proxy, a Quarkus interface, and OpenFGA for security. Through extensive empirical testing of various index configurations, cluster setups, and quantization methods (such as rotational quantization), our results show that optimizing the use of tenant subgraphs and selecting appropriate quantization techniques can significantly reduce resource consumption and improve database throughput without sacrificing query accuracy.

Keywords: vector database, embedding, cloud computing, quantization, index, interface, MCP, RAG, multi-tenancy, approximate nearest neighbor search, benchmark.

Poglavje 1

Uvod

V zadnjih letih je prišlo do velikih prebojev na področju umetne inteligence, posebej jezikovnih modelov. Najpomembnejši preboj je bil razvoj modela GPT-3, ki je bil treniran na splošnih podatkih, pridobljenih na spletu, in je bil na podlagi teh sposoben reševati nove naloge, ki niso bile del njegovih učnih podatkov, s pomočjo le nekaj primerov [28]. Skupaj z LLM-ji se je razširilo in napredovalo širše področje strojnega učenja in umetne inteligence. Razvili so se novi postopki razumevanja dokumentov na podlagi multimodalnih LLM-jev [32], izboljšani in novi postopki vložitve besedila in drugih medijev [29] ter generiranja multimedijskih vsebin [72].

Nekaj velikih podjetij je prevzelo razvoj velikih jezikovnih modelov, uporabniki pa so zdaj odgovorni le za to, da posredujejo svoje domensko specifične podatke LLM-ju, da ta opravi nalogo. Pojavila se je potreba po razvoju sistemov za s poizvedovanjem obogateno generiranje, ki omogočajo, da se prek enostavnih poizvedb v bazo zasebnih podatkov obogati izvajanje jezikovnega modela [65].

Z uvedbo agentnih sistemov se je razširila namembnost jezikovnih modelov, saj lahko agenti poleg generiranja besedil samostojno interagirajo z okoljem ter opravljajo krajše in daljše naloge [68]. Pojavila se je potreba po arhitekturi, ki je zmožna spojiti agentno umetno inteligenco s poizvedbo po zasebnih podatkih, je prenosljiva med domenami in zagotavlja visoke standarde

varnosti podatkov. Čeprav se kontekstna okna povečujejo, raziskave kažejo, da vnašanje večjih količin podatkov v LLM ne zagotavlja večje uspešnosti pri opravljanju nalog [48, 61]. Ogrodja za razvoj agentov omogočajo, da agenti izvajajo dolgoročne naloge [68], vendar se ob veliki rabi kontekstnega okna poraba žetonov poveča, natančnost agenta pa zmanjša. Med predlaganimi rešitvami za zmanjšanje kontekstnega okna je tudi raba podatkovnih baz kot dolgoročnega spomina [42].

Na podlagi vložitve podatkov se je razvil nov postopek poizvedovanja po podatkih, ki ne temelji na iskanju po vrednostih posameznih atributov, temveč na iskanju vložitev, ki so podobne vložitvi poizvedbe. Razvilo se je več novih struktur za iskanje k najbližjih sosedov visoko razsežnih vektorjev [44, 45, 50]. Na osnovi teh struktur so nastale vektorske podatkovne baze, ki omogočajo enostavno vzpostavitev sistema za hranjenje velikih količin vloženih podatkov.

Cilji tega dela so zasnova arhitekture sistema, ki se lahko zlahka integrira v obstoječe oblačne aplikacije, optimizacija VDB za hrambo podatkov, ki pripadajo več uporabnikom, ter izpostavitev funkcionalnosti tega sistema agentom.

Prvi del te naloge bo posvečen pregledu konceptov, povezanih z vektorskimi podatkovnimi bazami, značilnostim uporabljenih podatkovnih struktur, metod kvantizacije in vrst poizvedb po vektorskih podatkovnih bazah. Primerjali bomo funkcionalnosti dveh razširjenih VDB, Milvusa in Qdranta, ter opravili kratek pregled konceptov, povezanih z agenti, podrobneje z njihovo porabo kontekstnega okna. V drugem delu predstavimo zasnovo arhitekture oblačne storitve, ki omogoča vektorizacijo velikih količin uporabniških podatkov za potrebe izboljšanja delovanja agentnih sistemov. Glavni izziv je zasnovati arhitekturo storitve tako, da je prilagojena okolju z velikim številom najemnikov in omogoča hitro integracijo z že obstoječim okoljem. V tretjem delu testiramo različne nastavitve VDB, saj ta predstavlja ozko grlo sistema. Testiramo njeno zmogljivost ob različnih nastavitvah z namenom doseči čim bolj optimalno nastavitev z vidika porabe računskih

virov. Nazadnje pregledamo rezultate testiranja in izpostavimo dobre prakse pri nastavitvi VDB.

Poglavje 2

Osnovni pojmi

Preden se posvetimo komponentam VDB, obnovimo najprej poznavanje nekaterih matematičnih pojmov in podatkovnih struktur, ki nam omogočajo hitrejše razumevanje koncepta VDB. Nato se osredotočimo na osnovne komponente VDB in jih primerjamo z ekvivalentnimi komponentami relacijskih DB, ki so večini razvijalcev storitev bolje znane.

2.1 Vektorji

Osnovni pojem, na katerega se sklicujemo v tem delu, je vektor. Vektor je urejena n-terica števil, ki predstavlja koordinate točke v n-razsežnem prostoru. i-ta komponenta vektorja predstavlja položaj v i-ti razsežnosti prostora.

$$\vec{x} = [x_1, x_2, \ldots, x_n]$$

Nad vektorji so definirani seštevanje, množenje s skalarjem in skalarni produkt. Skalarni produkt med vektorjema $\vec{x}$ in $\vec{y}$ je definiran na naslednji način:

$$\vec{x} \cdot \vec{y} = x_1 y_1 + x_2 y_2 + \ldots + x_n y_n \tag{2.1}$$

Na množici vektorjev definiramo tudi razdaljo L_p kot

$$D_p(\vec{x}, \vec{y}) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p} \tag{2.2}$$

L_1 poznamo tudi kot manhattansko razdaljo, L_2 pa kot evklidsko. L_2-razdaljo lahko zapišemo tudi kot koren skalarnega produkta razlike med vektorji: $\sqrt{(\vec{x} - \vec{y}) \cdot (\vec{x} - \vec{y})}$. Dolžina oziroma norma vektorja je definirana kot razdalja vektorja od izhodišča ($[0, 0, \ldots, 0]$). Zanimala nas bo evklidska norma, v nadaljevanju bomo pisali

$$\|x\| = D_2(\vec{0}, \vec{x}) = \sqrt{\vec{x} \cdot \vec{x}} \tag{2.3}$$

Pravimo, da je vektor normiran, če je njegova norma 1, to pomeni $\|x\| = 1$.

V nadaljevanju nas bodo zanimale naslednje štiri metrike za primerjanje vektorjev po parih: skalarni produkt, manhattanska in evklidska razdalja, ki smo jih že definirali, ter kosinus kota med vektorji. Prek kosinusnega izreka lahko dokažemo, da za kot med vektorjema, ki ga označimo z γ, velja:

$$\cos(\gamma) = \frac{\vec{x} \cdot \vec{y}}{\|\vec{x}\| \|\vec{y}\|}$$

Če sta vektorja normirana, je skalarni produkt enak kosinusu kota med njima. Če sta si pravokotna, bo njun produkt enak 0 [54].

2.2 Prekletstvo dimenzionalnosti

V nadaljevanju se bomo ukvarjali z vektorji, definiranimi na prostorih velikih razsežnosti, torej z več sto ali več tisoč dimenzijami. Pri velikih dimenzijah določeni pristopi, ki učinkovito delujejo na prostorih majhnih razsežnosti, ne delujejo več optimalno [37].

Recimo, da bi vektorje v 3-dimenzionalnem prostoru uporabili za razdelitev prostora na odseke, prek katerih bi klasificirali točke v prostoru, da bi pohitrili iskanje najbližjih sosedov. Če bi vsako razsežnost razdelili na 10

odsekov, bi celoten prostor razdelili na 1000 podmnožic. Če razdelimo milijon točk v prostoru med te množice (ob predpostavki, da so odseki dobro določeni), bo v vsaki podmnožici približno 1000 vektorjev; točke, ki so najbližje poizvedbi, bomo iskali znotraj podmnožice, kamor spada poizvedba, in morda še v nekaj sosednjih podmnožicah.

Na 1000-dimenzionalnem prostoru takšna razdelitev ni smiselna, saj bi z razdelitvijo vsake razsežnosti na dvoje dobili $2^{1000} \approx 10^{300}$ podmnožic. Če bi milijardo (10^9) vektorjev razdelili po prostoru, bi bila večina podmnožic prazna, ostale pa bi vsebovale po en vektor. Da bi našli točko, najbližjo poizvedbi v tako velikem prostoru, bi bilo lažje izračunati razdaljo do vseh obstoječih točk, kot poizvedovati, ali je kak vektor v sosednji podmnožici (množic je toliko, da so povečini prazne, možnost, da je množica neprazna, je približno $\frac{10^9}{2^{1000}}$).

Indeksi nad vektorskimi polji so zato implementirani na podlagi struktur, ki niso občutljive na razsežnost vektorskega prostora.

2.3 Gosti in redki vektorji

Vektorje z določenim številom razsežnosti, katerih komponente so večinoma neničelne, imenujemo gosti vektorji. Informacija je v teh vektorjih porazdeljena po vseh komponentah.

V določenih kontekstih, kjer je struktura dokumentov strogo predpisana, na primer na področjih programiranja, medicine in prava, lahko iskanje po ključnih besedah nudi boljše rezultate kot iskanje prek semantične podobnosti, saj so imena za vsak pojem strogo predpisana. Pri programiranju imajo podatkovni tipi enolično definirano ime (na primer v Javi bomo nize vedno referencirali prek tipa String, na vrhu dokumenta pa bomo uvozili tip java.lang.String), posledično ne obstaja nevarnost, da bi pri iskanju z ujemanjem besed spregledali rezultat poizvedbe [46].

Iskanje z ujemanjem besed temelji na redkih vektorjih: redek vektor ima veliko razsežnosti (tudi v milijonih), a je le nekaj koordinat vektorja

neničelnih, torej je informacija porazdeljena le po nekaj razsežnostih. Indeks koordinate predstavlja mesto besede v slovarju, vrednost na tem mestu pa število pojavitev besede znotraj vloženega besedila.

$$\vec{x} = [0, \ldots, 0, x_i, 0, \ldots, 0, x_j, 0, \ldots, 0]$$

Za primerjanje redkih vektorjev je razširjena uporaba različic algoritma BM25. Osnovna različica algoritma pri izračunu relevantnosti besedila daje prednost besedam, ki se pojavljajo bolj poredkoma, medtem ko zmanjša utež besed, ki se pogosto uporabljajo, in hkrati zmanjšuje relevantnost zelo dolgim besedilom, saj vsebujejo več besed kot krajša besedila, zato je v njih lažje najti ujemanje ključnih besed [62].

2.4 Vložitev besedila

Vložitev besedila je postopek, prek katerega besedilo iz tekstovnega zapisa pretvorimo v n-razsežen vektor, pri tej transformaciji pa se njegov semantični pomen ohrani. To pomeni, da bo razdalja med vložitvama dveh besedil s podobnim pomenom majhna. Glavne naloge, s katerimi se ocenjuje kakovost modelov za vložitev besedil, so grupiranje, iskanje prevodov besedila, klasifikacija in poizvedovanje [33, 51].

Prednost vložitve besedila pred iskanjem po ujemanju ključnih besed pri generiranju z obogatenim poizvedovanjem (RAG) je, da bo iskanje uspešno vrnilo rezultat tudi, ko iščemo po sorodnih pojmih, saj se pri iskanju ne uporablja ujemanje besedila, temveč se zapise primerja po semantični podobnosti. Vložitve besedila so goste, kar pomeni, da je večina komponent vektorja vložitve neničelna.

Novejši modeli za ustvarjanje vložitev, kot na primer Qwen3-Embedding, uporabljajo LLM-transformerje za generacijo vložitev; vložitev je ustvarjena na podlagi stanja zadnjega sloja transformerja ob koncu prebiranja vhodnih podatkov [71]. Taka arhitektura omogoča, da imajo modeli za vložitev besedila zelo velika kontekstna okna; modeli na vrhu MTEB-lestvice lahko

večinoma vložijo besedila, ki vsebujejo več kot 32.000 žetonov. Poleg tega uporabljajo učenje matrjoškega prikaza (angl. *Matryoshka representation learning*) [47], kar omogoča, da lahko vektor skrajšamo na prvih n komponent, pri tem pa se natančnost vektorskega prikaza ne izrazito poslabša. Ta lastnost vložitev besedila omogoča zmanjšanje uporabe pomnilnika in diska [29].

Če bi na primer poizvedovali o besedilih, ki govorijo o rabi »elektronskih virov«, ne bi našli vseh tistih besedil, ki govorijo o »digitalnih gradivih«, če bi iskali zgolj po ujemanju besed. Iskanje prek podobnosti vložitev besedila pa bi vrnilo tako ena kot druga besedila.

2.5 Kvantizacija vektorjev

Običajen vektor z n dimenzijami vsebuje n 32-bitnih števil s plavajočo vejico. Pri 1024-razsežnem vektorju (vložitve besedil imajo običajno od 768 do 4096 razsežnosti [51]) moramo za vsak vektor v pomnilniku hraniti 4 KB podatkov, pri 1 milijonu vektorjev pa količina podatkov znaša približno 4 GB. Za ohranjanje nizke zakasnitve pri iskanju najbližjih sosedov vektorja je pomembno, da so vrednosti vektorjev hranjene v pomnilniku, saj se tako izognemo branju z diska, ki je bistveno počasnejša operacija. Opazimo pa, da količina podatkov že pri 10 milijonih vektorjev preseže zmogljivosti običajnega osebnega računalnika. V modernih oblačnih arhitekturah lahko sicer sami izberemo količino pomnilnika, ki ga bomo uporabljali, vendar je pomnilnik drag računski vir, zato si želimo čim bolj zmanjšati njegovo uporabo. V 16 GB pomnilnika lahko hranimo le 4 milijone 1024-razsežnih vektorjev v njihovem originalnem zapisu.

Za zmanjšanje rabe pomnilnika je bilo razvitih več postopkov kvantizacije, to je kompresije podatkov vektorja v prikaze, ki ohranjajo lastnosti vektorja, uporabljene pri iskanju najbližjih sosedov, porabijo pa manj podatkov za prikaz in shranjevanje vektorja. Pri rešitvah, ki uporabljajo kvantizacijo, se običajno kompresiran prikaz vektorja hrani v pomnilniku, originalni vektor pa na disku.

Skalarna kvantizacija preslika koordinate vektorja iz 32-bitnega števila s plavajočo vejico v 8-bitno celo število in pri tem doseže štirikratno kompresijo podatkov [18]. Na podlagi učne množice se ustvari slovar, ki preslika 8-bitno število v število s plavajočo vejico, na podlagi katerega se računa razdalja. V 16 GB pomnilnika lahko zdaj hranimo 16 milijonov 1024-razsežnih vektorjev.

Bitna kvantizacija ohrani predznak komponent vektorja, občutno pa zmanjša natančnost metrik, kot sta razdalja med vektorji ali skalarni produkt dveh vektorjev [18]. Prednost bitne kvantizacije je, da ni potrebe po fazi analize podatkov, saj temelji na predznaku komponente vektorja, ter da doseže 32-kratno kompresijo, kar omogoča, da v 16 GB pomnilnika hranimo 128 milijonov 1024-razsežnih vektorjev. To je velika prednost, kadar imamo opravka z velikimi nabori podatkov, vendar, kot bomo videli, bo takšna stopnja kvantizacije negativno vplivala na priklic poizvedb.

Da bi se pri enaki kompresiji bolje ohranile začetne lastnosti vektorjev, so bili razviti naprednejši postopki kompresije, ki jih bolj učinkovito ohranjajo.

2.5.1 Kvantizacija produkta

Osnovna ideja kvantizacije produkta je gručenje po metodi voditeljev [49]. Gručenje celotnih vektorjev se ne splača, saj bi ohranili le minimalno količino informacije iz vektorja. Pri k gručah bi vektor skrčili na $\log_2(k)$ bitov informacije. Osnovna ideja kvantizacije produkta je, da vektor razdelimo na m podvektorjev enakih razsežnosti in vsak podvektor kvantiziramo ločeno [45].

Kvantizacijo produkta izvedemo v fazi vnosa vektorjev n razsežnosti nad učno množico, ki obsega del ali celoto vnesenih vektorjev. Vektor razdelimo na m skupin po $n^* = \frac{n}{m}$ komponent.

$$\mathbf{x} = \big(\underbrace{x_1, \ldots, x_{n^*}}_{\mathbf{u}_1}, \underbrace{x_{n^*+1}, \ldots, x_{2n^*}}_{\mathbf{u}_2}, \ldots, \underbrace{x_{(m-1)n^*+1}, \ldots, x_n}_{\mathbf{u}_m}\big) \in \mathbb{R}^n \tag{2.4}$$

Za vsak vektorski podprostor dobimo po postopku gručenja k^* centroidov, ki jih zapišemo v slovar. Ob koncu postopka imamo torej m slovarjev, vsak s

po k^* centroidi. V fazi kvantizacije vektorjev za vsak izmed m podvektorjev poiščemo najbližji pripadajoči centroid [45].

Pri kvantizaciji produkta lahko na podlagi izbire faktorjev m in k^* dosežemo razmerje kompresije $\frac{32n}{m \log_2 k^*}$, ob predpostavki, da originalni zapis komponente vektorja porabi 32 bitov. Ob izbiri dovolj majhnih m in k^* lahko vsako komponento vektorja skrčimo na manj kot en bit.

2.5.2 Rotacijska kvantizacija

Osnovna ideja metod kvantizacije, ki temeljijo na psevdo-naključni rotaciji vektorja, je, da lahko pride do velikih nihanj med vrednostmi komponent vektorja in posledično do velikih napak, ko zmanjšamo natančnost prikaza vektorja. Metode, kot sta RaBitQ in TurboQuant, v osnovi temeljijo na tem, da vektor pred kvantizacijo najprej rotiramo s pomočjo ortogonalne matrike z determinanto 1 ali -1 in tako enakomerno porazdelimo informacije znotraj vektorja po vseh komponentah. S tem dosežemo, da so vrednosti komponent vektorja približno normalno porazdeljene. Različne implementacije vpeljejo popravke v algoritem, da dosežejo porazdelitev $\mathcal{N}(0, 1)$. Ker rotacija ohrani vrednost skalarnega produkta in razdalje, bosta razdalja in produkt dveh vektorjev ali njunih rotacij enaka [38, 39, 70].

Če je R ortogonalna matrika, potem velja $R^\top R = I$.

$$\vec{x} \cdot \vec{y} = \vec{x}^\top \vec{y} = \vec{x}^\top I \vec{y} = \vec{x}^\top R^\top R \vec{y} = \langle R\vec{x}, R\vec{y} \rangle \tag{2.5}$$

Pri dovolj velikih dimenzijah prostora se porazdelitev vrednosti posamezne komponente točke na unitarni krogli približa normalni porazdelitvi. Ker ob naključni rotaciji normiranega vektorja pričakujemo, da bo rezultat naključna točka na unitarni krogli, bo porazdelitev komponent naključno rotiranega vektorja sledila porazdelitvi komponent naključne točke na krogli, torej normalni porazdelitvi.

Vredno je omeniti, da se v primeru implementacije v vektorski podatkovni bazi Weaviate za rotacijo vektorja uporablja več iteracij hitre Walsh-Hadamardove transformacije, kar bistveno pohitri računanje rotacije (za ta

poseben primer matričnega množenja dosežemo računsko zahtevnost $\mathcal{O}(n \log n)$) in poenostavi grajenje rotacijske matrike.

$$\mathbf{H}_1 = \begin{bmatrix} 1 \end{bmatrix}, \quad \mathbf{H}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{H}_4 = \begin{bmatrix} \mathbf{H}_2 & \mathbf{H}_2 \\ \mathbf{H}_2 & -\mathbf{H}_2 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix} \tag{2.6}$$

Prednost algoritmov kvantizacije, ki temeljijo na rotacijah vektorjev, je, da postopek zagotavlja zgornjo mejo napake pri kvantizaciji, obenem pa ne zahteva ločene računsko zahtevne faze računanja gruč in njihovih centroidov. Poskusi kažejo, da vektorji, kvantizirani s temi postopki, dosegajo enake rezultate kot drugi postopki kvantizacije pri dvakrat večji kompresiji [30].

Če vektorje normaliziramo in normo vektorja hranimo ločeno, lahko RQ uporabimo tudi na nenormiranih vektorjih.

2.6 Iskanje najbližjega soseda

Pri množicah, na katerih je definirana linearna ureditev, je iskanje elementov po bližini ali enakosti enostavno, saj lahko elemente, kot že ime relacije pove, uredimo v vrsto. Nasprotno pa je natančno iskanje točke v množici vektorjev, ki je najbližji sosed danemu vektorju, računsko zahtevna operacija, za katero v visokih dimenzijah nobena znana struktura ne doseže poizvedbe, ki bi bila hkrati podlinearna in točna, ne da bi bil prostor eksponenten v d.

Ker je natančno iskanje najbližjih sosedov zahtevno, se vektorske podatkovne baze posvetijo problemu iskanja k aproksimativnih najbližjih sosedov (K-ANNS) poizvedbe (v našem primeru je poizvedba vložitev besedila poizvedbe). Problem iskanja najbližjih sosedov poenostavimo na iskanje k vektorjev, ki so po dani metriki z veliko verjetnostjo najbližji poizvedbi.

Raziskave si niso enotne glede tega, katera mera podobnosti je primernejša za posamezen tip poizvedbe, običajno se svetuje kosinus kota med vektorjema

ali njun skalarni produkt [57, 66].

2.7 Vektorska kazala

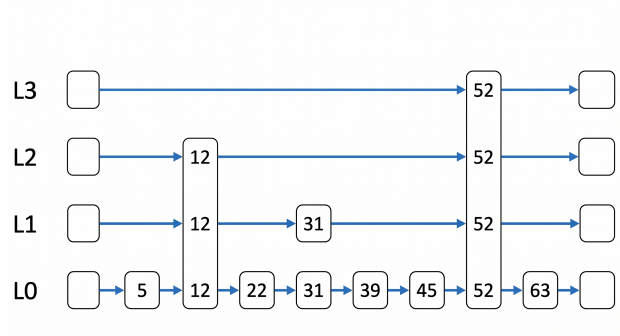
V namen pospeševanja iskanja se znotraj podatkovne baze ustvarijo kazala, ki pohitrijo iskanje podatkov. V odsotnosti kazala bi morali pri poizvedbi pogoj aplicirati na vsak posamezen zapis v podatkovni bazi. Najbolj razširjen tip kazala za skalarna polja je drevo B+. Ta podatkovna struktura omogoča hitre poizvedbe z zahtevnostjo $O(\log n)$ po indeksiranih poljih. Uporabna je na skalarnih (enorazsežnih) poljih, ki jih lahko linearno uredimo po velikosti (v primeru števil), leksikografski urejenosti (v primeru nizov) ali drugi linearni ureditvi; poleg tega omogoča iskanje elementov znotraj intervala.

Pri iskanju po vektorskem polju iščemo najbližjega soseda vektorja poizvedbe, natančneje k najbližjih sosedov. Ker ne obstaja linearna ureditev, ki bi pomagala pri reševanju problema K-ANN, so bile razvite druge vrste kazal, ki pohitrijo poizvedbo.

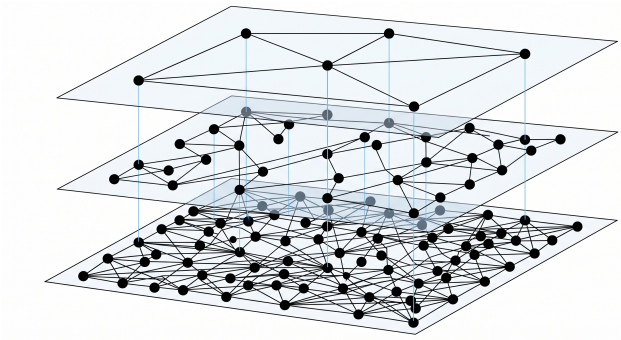
2.7.1 Graf HNSW

Najbolj razširjena implementacija kazala nad vektorskim poljem, ki omogoča iskanje rešitve problema K-ANNS, so hierarhični prehodni mali svetovi (HNSW). Osnovna podatkovna struktura, prek katere si lahko predstavljamo graf HNSW, je preskočni seznam. Vozlišča v preskočnem seznamu imajo svojo vrednost in višino. Verjetnost, da ima element višino vsaj H, se eksponentno manjša: $P(H \geq h) = p^{h-1}$. Na vsaki višini je element seznama povezan s tistim elementom, ki je vsaj tako visok, kot je višina povezave, in je njegov najbližji naslednik na tej višini. Pri iskanju elementa v seznamu najprej preskakujemo po višjih povezavah, saj so redkejše in omogočajo hitrejše iskanje; ko ne moremo več nadaljevati na trenutni višini, se spustimo po hierarhiji za en sloj nižje, dokler ne pridemo do iskanega elementa ali do dna in nimamo več naslednikov na razpolago, ki bi jih lahko iskali.

Graf HNSW deluje po podobnem principu, vendar vektorjev ne moremo



Slika 2.1: Primer preskočnega seznama (generirano prek UI)



Slika 2.2: Graf HNSW s tremi sloji (generirano prek UI)

linearno urediti za namen reševanja problema K-ANNS, zato bodo vozlišča grafa HNSW, podobno kot pri preskočnem seznamu, vsebovala vektor, imela višino, ki se izračuna enako kot pri preskočnem seznamu (pri preskočnih seznamih bo p velik, pri grafih HNSW pa občutno manjši, ker ima vsako vozlišče veliko več povezav; pri originalni implementaciji je priporočena vrednost $\frac{1}{\ln(m)}$, kjer je m največje število povezav, ki jih lahko ima vozlišče na vsakem sloju), na vsaki višini pa bo vozlišče povezano z vozlišči istega nivoja. Lahko so to najbližji sosedi ali pa vozlišča, izbrana tako, da omogočajo skoke v drugo kopico vektorjev [50].

Glavna težava kazala HNSW pri velikih količinah vektorjev je, da se morata graf in vektorji v celoti nahajati v pomnilniku, da se lahko iskanje hitro izvede, kar v veliki meri prispeva k stroškom vzdrževanja vektorske podat-

kovne baze.

2.7.2 IVF kazalo

Obrnjeno datotečno kazalo (IVF) je kazalo, katerega cilj je zmanjšati porabo pomnilnika vektorske podatkovne baze. V fazi generiranja kazala se na podlagi učne množice iz nabora vektorjev v podatkovni bazi najprej razdeli množica hranjenih vektorjev v k' gruč prek metode voditeljev (angl. *k-means clustering*) in se za vsako gručo izračuna ter shrani centroid. Pri majhnem naboru vektorjev se za sestavo gruč uporabi celoten nabor vektorjev, pri velikem številu pa le del vektorjev, da se kazalo hitreje generira. Vsak vektor je nato razvrščen v gručo tistega centroida, kateremu je najbližji. V primeru IVF_PQ kazala vektor, shranjen v gruči, kvantiziramo prek kvantizacije produkta, da zmanjšamo količino podatkov vektorja. Dobimo strukturo, ki vsebuje slovar k' parov centroidov gruč in pripadajočih vektorjev. Pri iskanju najbližjih sosedov se najprej poišče `nprobe` najbližjih centroidov gruč, nato pa se znotraj teh gruč poišče k najbližjih sosedov [45].

2.7.3 DiskANN

Ker se kazali HNSW in IVF v osnovni zasnovi v celoti hranita v pomnilniku, nastane potreba, da se del kazala hrani na disku, s čimer se zmanjšajo stroški vzdrževanja VDB. DiskANN je nastal z namenom zmanjšati uporabo pomnilnika, ne da bi se pri tem povečala zakasnitev poizvedb ali zmanjšala prepustnost. DiskANN kazalo v pomnilniku hrani seznam kvantiziranih vektorjev, medtem ko sta originalni vektor in seznam njegovih sosedov hranjena na disku. Vamana graf sestavimo na podlagi naključnega k-regularnega grafa. V prvi fazi vzamemo medoid s za izhodišče požrešnega K-ANNS in za vsako vozlišče p v grafu izvedemo K-ANNS. Na podlagi obiskanih vozlišč popravimo povezave tako, da bodo čim krajše. Z namenom zmanjšanja diametra grafa se v drugi fazi konstrukcije Vamana grafa, na katerem temelji DiskANN, obreže vozlišča v grafu tako, da ohranimo najbližjega soseda, vsak nadaljnji

sosed pa mora biti za faktor $\alpha > 1$ bolj oddaljen od izhodiščnega vozlišča v primerjavi s prejšnjim (pri preizkušanju zmogljivosti kazala so uporabili $\alpha = 1{,}2$). Pri rabi kazala se pri vsakem branju iz diska prebere podatke za več vektorjev in tako zmanjša latenca. Dodatno se v pomnilniku ohranijo točke, ki so bile obiskane med prvimi iteracijami iskanja [44].

Poglavje 3

Vektorske podatkovne baze

Da bi lahko podatke iskali prek podobnosti njihovega vektorskega prikaza je nastala potreba po razvoju aplikacij, ki zmorejo iskati ujemanja znotraj velikih količin vektorjev. Glavna pristopa k razvoju podatkovnih baz, ki podpirajo iskanje po zbirkah vektorjev, sta dva: razširitve obstoječih podatkovnih baz, ki podpirajo iskanje po vektorjih (npr. pgvector za PostgreSQL), ter razvoj novih vektorskih podatkovnih baz (VDB). V nadaljevanju navedemo osnovne operacije, ki jih vektorske podatkovne baze podpirajo.

3.1 Nastavitve kazal nad vektorji

Vsaka implementacija vektorske podatkovne baze podpira različne vrste kazal nad vektorji in različna merila za iskanje najbližjih sosedov. Običajno so podprti razdalji L_1 in L_2, skalarni produkt ter kosinus med vektorjema. Najbolj razširjene implementacije kazal so različice grafov HNSW in IVF. Naprednejše podatkovne baze omogočajo določitev metode kvantizacije, ki bo uporabljena na kazalu, ter nastavitev parametrov, ki vplivajo na strukturo in učinkovitost kazala. Brez kazala nad vektorji so poizvedbe učinkovite, kadar je vektorjev le nekaj tisoč.

3.2 Dodajanje in brisanje zapisov

Medtem ko je dodajanje referenc v B+ drevesa enostaven postopek, je dodajanje vozlišč v vektorska kazala zapleteno, ker vpliva na strukturo grafa. VDB novo dodana vozlišča običajno ne zapišejo v kazalo, temveč v seznam novo dodanih točk, ki bodo vključene v kazalo, ko bo dosežena zadostna količina novih vnosov [7].

Podobno pri brisanju vozlišč teh ne moremo odstraniti iz grafa, ker se lahko zgodi, da graf ne bo več povezan. Namesto da bi vozlišča dejansko odstranili iz grafa, jih označimo kot izbrisana, dejansko pa bodo odstranjena naslednjič, ko bo kazalo ponovno zgrajeno. Dodajanje in brisanje zapisov spreminjata strukturo kazala in zmanjšujeta njegovo učinkovitost.

3.3 Filtrirana poizvedba

V številnih primerih se pojavi zahteva, da se omeji dostop uporabnikov do zapisov v podatkovni bazi ali da pridobljeni rezultati zadoščajo določenim pogojem (recimo, da poizvedba o zakonskih predpisih vrne le tiste zakone, ki zadevajo Slovenijo). V ta namen se K-ANNS doda filter. Glavni pristopi so trije:

- **filtriranje pred iskanjem:** najprej se izračuna predikat za vse zapise in generira bitna maska, ki označi dopustne kandidate za iskanje po grafu oziroma znotraj IVF particije. To je najpogosteje uporabljena strategija filtriranja, saj omogoča, da imamo informacije o naboru dopustnih zapisov, preden se lotimo računsko potratnega iskanja. Če je nabor kandidatov, ki zadošča filtru, dovolj majhen, se določene podatkovne baze odločijo za izčrpno iskanje po naboru dopustnih rešitev brez uporabe kazala;
- **filtriranje med izvajanjem:** izračun predikata se izvede le, ko dostopamo do vozlišča med potovanjem po grafu oziroma ko dostopamo do particije znotraj IVF;

- **filtriranje po poizvedbi:** izvede se šele, ko se k-ANNS zaključi; med dobljenimi k najbližjimi sosedi izbere le tiste, ki zadoščajo filtru. Pri selektivnih filtrih se lahko zgodi, da v naboru rezultatov ni nobenega, ki zadošča pogojem, in poizvedba ne vrne nobenega rezultata, čeprav v bazi obstajajo zapisi, ki zadoščajo pogojem [25].

Poleg tega so se razvile specializacije kazal, ki pri gradnji upoštevajo povezanost vozlišč, ki zadoščajo pogojem filtriranja. Pomanjkljivost tega pristopa je, da mora biti pogoj filtra znan ob tvorbi kazala, kar pomeni, da bi morali ob vsakem novem pogoju za iskanje uporabiti enega izmed zgornjih postopkov ali na novo sestaviti kazalo. Ker se učinkovitost algoritmov za K-ANNS običajno meri prek poizvedb, ki se ne poslužujejo filtriranja, so primerjalna merila za učinkovitost posameznih implementiranih rešitev za filtrirano iskanje redkejša in manj standardizirana.

V določenih primerih želimo imeti več vektorskih polj v isti shemi (na primer gosto polje za vložitve in redko polje za iskanje ključnih besed). Ko želimo rezultate iskati na podlagi ne enega, temveč več vektorskih polj, pravimo, da izvajamo hibridno iskanje.

3.4 Hibridno iskanje

Hibridno iskanje običajno poteka v dveh korakih: v prvem koraku se izvede iskanje po posameznem vektorskem polju in za vsako polje pridobi k najboljših ujemanj (število k rezultatov se lahko med polji razlikuje). Dobljene rezultate nato združimo prek primernega algoritma, kot je na primer združevanje prek medsebojnega ranga (angl. *reciprocal rank fusion*) [24, 60].

3.5 Ponovna uvrstitev rezultatov

Ponovna uvrstitev rezultatov je postopek, pri katerem dobljene k rezultate poizvedbe v bazi dodatno analiziramo in jih ponovno uvrstimo po relevantnosti na podlagi drugega merila, ki ga je težje izračunati, ali na podlagi

vrednosti drugega vektorskega polja, na primer redkega vektorja. Cilj ponovne uvrstitve rezultatov je izboljšati relevantnost poizvedbe [22].

3.6 Varčevanje z infrastrukturnimi viri

VDB pri izvajanju uporabljajo predvsem dva vira: pomnilnik za hrambo podatkov in kazal ter računsko moč za grajenje kazal in izvajanje poizvedb.

Poraba pomnilnika ni vezana na posamezne naloge, ki jih VDB izvaja, temveč na trenutno stanje VDB, število hranjenih vektorjev in kazal. Porabo pomnilnika lahko omejimo na več načinov.

Najpreprostejši način je kvantizacija vektorjev. Prek različnih načinov kvantizacije porabo pomnilnika zmanjšamo tudi do 64-krat (ob uporabi PQ), pri tem pa se pri kvantiziranih vektorjih zmanjša uspešnost poizvedbe. Da se izboljša točnost poizvedbe, lahko poiščemo αk najbližjih sosedov, kjer je $\alpha \geq 1$ faktor vzorčenja poizvedbe, iz nabora rešitev pa nato na podlagi ponovne uvrstitve rezultatov prek originalnih vektorjev najdemo k najbližjih sosedov.

Druga rešitev, ki jo implementirajo DiskANN in različice drugih kazal, je hranjenje dela podatkov na disku. Običajno se v pomnilniku hranijo kvantizirani prikazi vektorjev, kazala nad tovorom in najpogosteje dostopani podatki, medtem ko se na disku nahajajo originalni zapisi vektorjev, vrednosti ostalih polj VDB zapisa in povezave med vozlišči kazala. Določene VDB omogočajo, da se neuporabljene dele zbirk odstrani iz pomnilnika in hrani izključno v objektni shrambi. Dostop do podatkov, ki se ne nahajajo v pomnilniku, je počasnejši, zato se pri tem pristopu poveča latenca poizvedb v bazo.

Vir OCI	**Mesečni stroški OCI**
A1 procesor	6,70 € na vCPU
Pomnilnik	1,00 € na GB
SSD disk	0,024 € na GB*
Objektna shramba	0,024 € na GB

Tabela 3.1: Primerjava cen procesorja, pomnilnika, diska in objektne hrambe na OCI [12].

* Opomba: ceni je treba prišteti še stroške, vezane na zmogljivost diska.

Poglavje 4

Primerjava funkcionalnosti Milvus in Qdrant VDB

Milvus in Qdrant sta dve porazdeljeni podatkovni bazi, kar pomeni, da so podatki porazdeljeni po več vozliščih znotraj omrežja. Ko imamo opravka s porazdeljenimi sistemi, je vredno imeti v mislih CAP izrek, ki trdi, da ob omrežni particiji sistem izbira med konsistenco in razpoložljivostjo; brez particije lahko zagotavlja oboje:

- **konsistenco podatkov:** operacije so na vseh vozliščih izvedene v enakem vrstnem redu; ne glede na to, katero particijo gledamo, vidimo enako stanje. Sistem je strogo konsistenten, če se porazdeljen sistem obnaša kot ena sama enota;

- **visoko razpoložljivost:** poizvedba na poljubno vozlišče bo vedno vrnila podatek, v formulaciji izreka pa ni definirana časovna omejitev za vračanje odgovora;

- **toleranco do omrežnih particij:** sistem bo deloval tudi, če pride do napak pri komunikaciji med vozlišči in se sporočila, posredovana med participami, izgubijo. Sistem deluje ne glede na to, koliko sporočil v omrežju je bilo izgubljenih [27, 40].

Ko so podatki porazdeljeni po omrežju, lahko pri arhitekturi sistema odločamo le o tem, ali bo sistem omogočal visoko razpoložljivost ali strožjo konsistenco podatkov. Če se odločimo ohraniti konsistenco podatkov, bodo spremembe stanja v sistemu opazne z zamikom, ker bodo particije podatkov usklajevale svoje stanje. Če se odločimo za večjo razpoložljivost podatkov, bodo spremembe stanja podatkov lahko opazne takoj, ni pa nujno, da dve enaki poizvedbi po podatkovni bazi v istem trenutku vrneta isti rezultat. Velja pa, da lahko dosežemo ravnotežje med konsistentnostjo in razpoložljivostjo podatkov. Lahko se na primer odločimo, da nam zadošča, da je vsaj polovica particij posodobljena, preden postane podatek razpoložljiv.

4.1 Arhitektura sistema

Milvus in Qdrant uporabljata dva različna koncepta arhitekture podatkovne baze. Qdrant, podobno kot Weaviate [21], sloni na gruči enakih vozlišč, ki se med seboj usklajujejo prek Raft protokola. Podatki posameznih vozlišč so trajno shranjeni na disku, kar pomeni, da so vozlišča Qdrant nameščena prek krmilnika StatefulSet [2], ki strokom (angl. *pod*) zagotavlja stabilno identiteto, omrežno ime in trajen nosilec (PVC). Da se podatki ne izgubijo ob odpovedi diska, se običajno odločimo za replikacijo podatkov na več vozlišč, praviloma treh, da je ob izpadu enega vozlišča večina še vedno razpoložljiva. Qdrant vzpostavlja tudi API za izvoz varnostne kopije podatkov iz podatkovne baze; to kopijo lahko nato hranimo v objektni shrambi in jo ponovno uvozimo v podatkovno bazo. Qdrant prek Raft konsenza usklajuje le podatke, ki zadevajo topologijo sistema, ne pa stanja posameznih podatkov znotraj podatkovne baze. To pomeni, da mora operacijo potrditi vsaj del vozlišč znotraj Raft gruče, če spremenimo shemo zbirke ali ustvarimo novo zbirko (delež vozlišč se nastavi prek konfiguracije, običajno je to vsaj polovica vozlišč); prek Raft konsenza se zagotavlja, da bodo na vseh vozliščih izvedene vse operacije v enakem zaporedju. Operacije nad zapisi znotraj zbirke privzeto niso konsistentne, kar pomeni, da je pisalna operacija veljavna takoj,

ko se zapiše na eno samo vozlišče; omogočeno je, da se za posamezno zbirko določi, na koliko vozliščih se mora izvesti pisalna operacija, da bo veljavna. Qdrant pri izvajanju daje prednost razpoložljivosti, konsistenca podatkov pa ni zagotovljena [52]. Qdrant izvaja iskanje prek CPU, vendar nudi možnost, da se kazala gradijo s pomočjo GPU [14].

Arhitektura Milvusa deluje po drugem principu. Vozlišča znotraj Milvus podatkovne baze so specializirana. Koordinator vozlišč je en sam in upravlja tri različne kategorije vozlišč: vozlišča za upravljanje toka podatkov, vozlišča za poizvedbe in podatkovna vozlišča; ta zadnja imajo nalogo graditi in optimizirati obstoječa kazala. Koordinator prevzema naloge, ki jih v Qdrant arhitekturi upravljajo vozlišča prek Raft protokola – to so operacije nad strukturo zbirk –, poleg tega upravlja avtoritativni vir časovnih žigov (timestamp oracle) in ostalim vozliščem posreduje utrip notranje ure. Koordinator, kot že ime pove, upravlja naloge in delovanje ostalih vozlišč ter razporeja opravila med njimi. Vozlišča za upravljanje toka podatkov so odgovorna za pisanje v podatkovno bazo. Nove podatke hranijo znotraj dnevnikov za predčasno pisanje (WAL – write ahead log) prek razširjenih rešitev, kot so Kafka, Pulsar ali Woodpecker. Samo topologija baze in metapodatki zbirk so hranjeni v etcd-ju. Milvus zapisane podatke trajno hrani znotraj objektne shrambe. Prednost tega pristopa je, da je odpornost na odpovedi posameznih diskov zagotovljena, slabost pa, da je poizvedba v objektno shrambo časovno potratna. Arhitektura Milvusa je zasnovana tako, da lahko odjemalcu podatkov zagotovi konsistenco podatkov prek rabe časovnih žigov. Vsako vozlišče prejema od koordinatorja časovne žige in na podlagi teh lahko primerja svoj trenutni časovni žig s stanjem hranjenih podatkov; podatki, hranjeni na vozliščih, vključujejo tudi podatek o tem, do katerega trenutka je zagotovljeno, da so vse operacije zapisane na vozlišče (`guarantee_timestamp`). Na podlagi tega podatka se lahko poizvedbe uskladijo tako, da zagotavljajo želeno konsistentnost [4, 10, 11].

4.2 Implementacija kazal in kvantizacije

Qdrant podpira en sam tip kazala nad gostimi vektorskimi polji, to je graf HNSW. Implementacija HNSW kazala v Qdrantu se razlikuje od osnovne, saj omogoča kvantizacijo vektorjev ter premik izvirnih vektorjev in kazala iz pomnilnika na disk. Povezave znotraj grafa so dodatno optimizirane z namenom podpore filtriranim poizvedbam: ustvarijo se dodatne povezave med vozlišči, pri katerih je vrednost indeksiranega polja enaka. Pri izvajanju poizvedb Qdrant podpira uporabo naprednih algoritmov, ki omogočajo sprehod po zelo selektivno filtriranih grafih, kot je ACORN [56]. Qdrant sproti optimizira in združuje grafe, da izboljša vektorske poizvedbe, ter stremi k temu, da posamezne grafe, ki tvorijo zbirko, združi v enega samega. Pri tem ima Qdrant nastavljeno omejitev, kolikšna je lahko največja velikost odseka kazala, saj večji odseki potrebujejo več časa za gradnjo. Pri kazalih moramo biti pozorni tudi na to, kako so podatki shranjeni na disk. Qdrant omogoča, da nastavimo polje, ki bo uporabljeno za t. i. kazalo najemnikov: vektorski zapisi bodo na podlagi tega polja zbrani na istih odsekih diska, namesto da bi bili razpršeni po njem. Deljenje podatkov na particije deluje znotraj enega Qdrant vozlišča. Na ravni gruče Qdrant omogoča, da se vektorska zbirka razdeli na drobce (angl. *shard*) na podlagi vrednosti specifičnega polja znotraj zapisa. Taka nastavitev omogoča, da se pri velikih količinah podatkov poizvedbe še vedno izvajajo na enem samem vozlišču, kar bistveno pohitri poizvedbo [15, 17, 18].

Milvus podpira več implementacij kazal, vendar pri posamezni implementaciji ne podpira vseh metod kvantizacije vektorjev. Podpira več različic optimiziranih kazal: IVF, graf HNSW, DiskANN in druga kazala, ki temeljijo na rabi diska, ter kazala, optimizirana za poizvedbe prek grafičnih kartic. Milvus podpira filtriranje pred iskanjem na podlagi skalarnih polj, na podlagi uradne dokumentacije pa ne zasledimo optimizacij kazal, namenjenih izboljšanju uspešnosti filtriranih poizvedb. Filter znotraj zbirke je implementiran z uporabo bitne mape. Podobno kot Qdrant podpira tudi Milvus mapiranje polj na disk in hranjenje kazal na disku. Milvus, podobno kot

Qdrant, ne hrani podatkov in kazal v obliki globalnega kazala, temveč zapise v podatkovni bazi razdeli v odseke (angl. *segment*). Vsak odsek se sproti zapečati, ko doseže določeno velikost; privzeto je to 512 MB. Ko se odsek napolni, vozlišča za obdelavo podatkov sestavijo kazalo za posamezen odsek. Milvus omogoča, da se določi polje, po katerem se bodo združevali odseki podatkovne baze; to je lahko ključ particije (Milvus podpira deljenje podatkov na logični ravni v particije) ali drugo polje, uporabljeno kot ključ za gručenje. Ko se odseki utrjujejo, so podatki združeni po odsekih na podlagi vrednosti izbranega polja. Podobno kot pri Qdrantu taka rešitev omogoča, da je za izvedbo poizvedbe potrebnih manj vozlišč, kar poizvedbo pohitri [5, 6, 8, 9].

4.3 Namembnost

Kot vidimo iz te primerjave, je arhitektura Milvus VDB bolj zapletena, vendar omogoča ločeno povečevanje posameznih zmogljivosti VDB, kot so nalaganje podatkov, grajenje kazal in poizvedovanje. Veliko specifičnih lastnosti Milvusa je namenjenih izredno velikim korpusom podatkov. Arhitektura Qdranta je v nasprotju z Milvusom preprosta; minimalna postavitev Qdranta obsega približno enak obseg kot postavitev PostgreSQL DB. Qdrant hkrati omogoča boljše nastavitve za filtriranje poizvedb; glavna prednost je možnost ustvarjanja ločenih HNSW podgrafov na podlagi indeksiranih polj. Qdrant je primernejši za storitve, kjer morajo biti podatki ločeni po najemnikih, in za manjše aplikacije, kjer bi bili stroški vzdrževanja arhitekture Milvus previsoki.

Direktne empirične primerjave med Qdrantom in Milvusom ne izvedemo, saj ne moremo direktno primerjati rabo virov štiritipne arhitekturo Milvusa z gručo enakovrednih Qdrant vozlišč

Poglavje 5

Agenti

> "Avtonomni agent je sistem, ki je umeščen v okolje in je njegov del, lahko zaznava okolje in skozi čas deluje v njem, da doseže svoje cilje ter vpliva na to, kar bo zaznal v prihodnosti." [36]

Veliki jezikovni modeli v osnovi proizvajajo besedilo na podlagi vnešenega ukaza. Glavno sredstvo za ustvarjanje besedila sta nevronska mreža in informacije, ki so vanjo zapečene v obliki uteži. V zgodnjih LLM-jih se je zato pojavljala težava pri poizvedbah o tekočih dogodkih ali informacijah, ki niso javno dostopne. Da bi model izučili tudi o teh informacijah, bi jih morali nabrati in jih zapeči v uteži grafa. Če bi želeli, da je jezikovni model seznanjen z najnovejšimi novicami, bi ga morali stalno ponovno trenirati na novih podatkih. Hkrati imajo LLM-ji velike sposobnosti reševanja problemov, tudi takih, na katerih niso bili trenirani, in so zmožni generirati ukaze za programske vmesnike. Prvi modeli so bili sposobni bodisi generirati tekstovni odgovor bodisi posamezen ukaz. Smernice ReAct so podale teoretične osnove za pripravo ogrodij za agente, osnovane na LLM-jih (v nadaljevanju: LLM-agenti). Predlagali so agenta, ki v vsakem koraku pridobi informacije iz okolja ($o_t \in \mathcal{O}$), nato pa na podlagi določenih pravil $\pi(a_t, c_t)$ izvede dejanje iz množice razpoložljivih dejanj $a_t \in \mathcal{A}$, v sebi pa hrani kontekst izvajanja, torej zaporedje prejetih informacij in izvedenih dejanj: $c_t = (o_1, a_1, ..., o_{t-1}, a_t)$. LLM-agenti po teh principih najprej razmislijo o svoji nalogi, nato izvedejo

ukaz ter na podlagi rezultatov bodisi zaključijo izvajanje bodisi se vrnejo na korak razmišljanja. Sodobni LLM-agenti dosežejo to, kar specificira definicija avtonomnega agenta: prek RAG-a lahko pridobijo informacije o okolju in so v njega umeščeni, prek interakcij z orodji pa lahko vplivajo na okolje z namenom doseganja ciljev, ki so jim bili dodeljeni. Pri tem so bolj zmogljivi kot običajen program, saj lahko med izvajanjem samostojno povežejo klice orodij z namenom doseganja določenega rezultata, medtem ko so predhodniki temeljili na jasno določenem zaporedju ukazov, ki so bili zapečeni v kodo [68, 69].

5.1 Protokol MCP

Agenti pridobijo sposobnost interakcije z okoljem prek orodij. Posamezna ogrodja za razvoj agentov ponujajo različne vmesnike, prek katerih lahko definiramo orodja, ki jih ima agent na voljo. Najbolj razširjen standard je protokol za pridobivanje konteksta modela, ki ga je razvil Anthropic, in definira standardizirano obliko, v kateri definiramo funkcionalnosti orodja, modeli pa v formatu JSON-RPC 2.0 izvajajo ukaze [23]. Modelu na kratko opišemo funkcionalnosti orodja in parametre, ki jih ima na razpolago za interakcijo z orodjem. Pri definicijah orodij pa se pojavi težava: čeprav lahko funkcionalnosti LLM-ja razširimo prek orodij, lahko podajanje prevelikega števila definicij orodij v model porabi njegovo kontekstno okno, poveča stroške izvajanja in hkrati zmanjša njegovo učinkovitost. Pojavi se potreba po tem, da selektivno izberemo orodja, ki jih bo agent uporabil [26, 41].

5.2 Kontekstno okno

Kontekstno okno je ključni del LLM-agenta. LLM, na katerem agent deluje, generira odgovor na podlagi predelave informacij, ki jih je prejel. Agenti nadgradijo osnovno delovanje LLM-ja tako, da na dno obstoječega konteksta dodajo rezultat trenutnega koraka in prek tega posodabljajo trenutno stanje

agenta [69]; natančna implementacija tega koraka se razlikuje med agenti. Ob izvajanju dolgotrajnih nalog pride do tega, da se kontekstno okno začne polniti in LLM ni več sposoben izkoristiti informacij, vsebovanih v kontekstnem oknu. Vsebino kontekstnega okna imenujemo tudi delovni spomin, saj vsebuje informacije, ki so relevantne za delo oziroma nalogo modela. Poleg uporabnikovega vnosa se sem vnesejo še sistemski vnosi, med katere spadajo sistemska navodila, namenjena agentu, in deklaracije orodij, na podlagi katerih lahko agent interagira z okoljem. Med izvedbo se delovni spomin polni še s sledmi razmišljanja agenta in odgovori klicev orodij [42, 61]. Ker v mnogih primerih dodajanje konteksta v kontekstno okno poslabša učinkovitost modela, so se razvili različni pristopi, kako zmanjšati količino konteksta in izboljšati fokus agenta. Verma [67] predlaga, da se med izvajanjem agenta sproti ustvari povzetek vsebine kontekstnega okna, Packer in dr. [55] predlagajo, da se zgodovina izvajanja agenta trajno shrani in se po potrebi doda v delovni spomin samo del zgodovine, ki je relevanten za trenutno fazo izvajanja. Repantis in dr. [61] zagovarjajo stališče, da lahko povečanje selektivnosti (pridobivanje manjšega števila podatkov) na račun priklica (odstotek pridobljenih točk med k najbližjimi poizvedbi) zmanjša porabo konteksta, ne da bi poslabšalo natančnost (odstotek dobljenih točk, ki so med najbližjimi k rezultati) informacij, ki so posredovane agentu. Debenedetti in dr. [31] pa ugotavljajo, da ponavljanje začetnih navodil med izvajanjem agenta izboljša njegovo uspešnost pri izvajanju nalog, čeprav dodatno poveča porabo kontekstnega okna.

Poglavje 6

Arhitektura storitev

Predlagamo koncept oblačne storitve, ki se poslužuje VDB. Glavni namen VDB je nuditi dolgoročen spomin LLM-agentom in prispevati k zmanjšanju rabe kontekstnega okna. VDB hrani zgodovino klepetov in tako omogoča, da agent v fazi razmišljanja ohranja le relevantne informacije v delovnem spominu. Definicija orodij porabi dodaten del delovnega spomina [61]; ker definicija orodja prek MCP temelji na opisu orodja, je semantična podobnost smiseln pristop za primerjavo in izbiro orodij. Sistem izpostavlja sposobnost, da se opis orodja vloži in podatki o orodju shranijo v VDB.

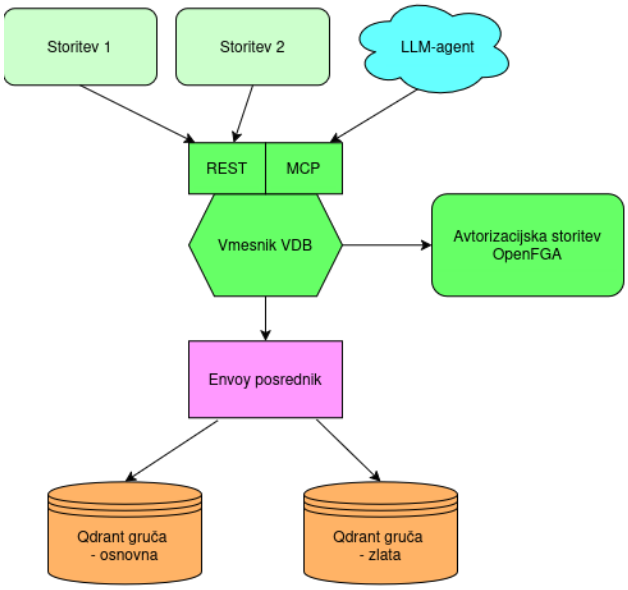
VDB dodatno omogoča, da lahko vložimo podatke vsakega uporabnika in jih hranimo na enem samem mestu ter tako uporabniku in njegovim agentom omogočimo, da prek ene same poizvedbe hkrati dostopajo do podatkov, ki bi sicer bili razkropljeni po več virih in po katerih bi lahko poizvedovali le prek specifičnih filtrov. Vpeljava VDB je smiselna, saj omogoča, da so podatki, ki bi bili sicer razkropljeni po več storitvah in bi jih morali pridobivati prek več različnih API-klicev, dostopni na enem mestu. Uporabniku in agentom tako ni treba poznati celotne strukture aplikacije, da pridobijo želeno informacijo. Še dodatna možnost uporabe je hranjenje zgodovine interakcij uporabnika z AI-agentom, da lahko uporabnik lažje poizveduje po preteklih pogovorih. Običajen pogovor z LLM-agentom namreč vsebuje pretežno nestrukturirane podatke, po katerih bi bila običajna poizvedba prek ujemanja metapodatkov

težavna. Poizvedba prek semantične podobnosti nudi uporabniku možnost, da ponovno najde odgovor agenta. Ker so LLM-ji nedeterministični in agenti podajajo odgovore tudi na podlagi predhodnih interakcij, ni zagotovil, da ponovna poizvedba vrne isti odgovor.

Glavne komponente sistema so:

- **VDB:** zagotovi trajno hrambo podatkov uporabnikov, dostop do nje ima le vmesnik VDB.
- **Vmesnik VDB:** upravlja dostop do vektorske podatkovne baze. Ima dve glavni nalogi: abstrahira operacije na VDB in omejuje dostop do te. Abstrakcija dostopa do vektorske podatkovne baze je smiselna, saj omogoča, da lahko ostale storitve berejo in pišejo v VDB, ne da bi pri tem poznale specifičen domenski jezik ponudnika baze. Ta izbira olajša razvoj in vzdrževanje vseh ostalih storitev znotraj aplikacije: v primeru menjave ponudnika VDB je vmesnik VDB edina storitev, ki jo je treba spremeniti. V razvojni ekipi tako ne nastane potreba, da posamezen razvijalec pozna VDB, ampak le vzdrževalec vmesnika. Vmesnik je implementiran kot mikrostoritev, saj lahko tako izpostavljen REST API uporabljajo odjemalci, razviti v različnih programskih jezikih. Če bi to funkcionalnost izpostavili kot knjižnico, bi morali verzije in konfiguracije usklajevati med storitvami, kar prinaša nepotrebna tveganja. Dodatno vzpostavimo na storitvi strežnik MCP, da lahko razvijalci agentov enostavno dostopajo do VDB.
- **Sistem za avtorizacijo:** ponudniki VDB privzeto ne dopuščajo veliko možnosti omejitve dostopa, zato je toliko pomembneje, da se omejitev implementira na vmesniku. Za upravljanje dostopa se lahko uporabi že obstoječa rešitev ali nov sistem, razvit po potrebah VDB. V našem primeru uporabimo OpenFGA, ki dopušča veliko fleksibilnosti pri določanju pravil za dostop do virov [53].
- **Ostale storitve:** v storitve dodamo odjemalca VDB. Kodo razširimo tako, da se ob spremembi podatkov v bazi sočasno sproži pisanje v

VDB. Če storitev že vsebuje podatke v bazi, pripravimo migracijo obstoječih podatkov v VDB.



Slika 6.1: Shema oblačne arhitekture z dvema ločenima gručama Qdrant (lastna shema)

6.1 Izpostavljene funkcionalnosti

Sistem ima naslednje naloge:

- **Vnesti podatke:** preveri pravice do vnosa podatkov, vloži vsebino vnosa ter vnos shrani v VDB;
- **Izvesti poizvedbo nad podatki:** preveri pravico do dostopa do podatkov in sestavi poizvedbo v domenskem jeziku ponudnika VDB;
- **Brisati uporabniške podatke:** omogoči brisanje uporabniških podatkov v skladu z GDPR [35].

6.2 VDB

Pri izbiri vektorske podatkovne baze se osredotočimo na zahteve storitve, ki jo razvijamo. Prednost pri izbiri podatkovne baze ima podpora filtriranemu iskanju, saj so poizvedbe omejene na podatke posameznega uporabnika. Qdrant podpira ustvarjanje podgrafov HNSW, ki povezujejo vozlišča, ki ustrezajo istemu filtru. Ko pričakujemo, da bo uporabnikov več sto ali več tisoč, je to smotrna izbira, saj bi bili alternativni rešitvi razdelitev zbirke na drobce na podlagi ključa ali ustvarjanje ločene zbirke za vsakega uporabnika, kar odsvetujeta dokumentaciji tako Milvus kot Qdrant VDB. Če bi izbrali podatkovno bazo Milvus, bi morali odseke optimizirati tako, da bi se podatki istega uporabnika znašli na istem odseku. Če predvidevamo filtriranje po več različnih poljih, ustvarimo kazalo za vsako polje, po katerem nameravamo filtrirati poizvedbe. Podgraf HNSW ustvarimo le za polje z identifikatorjem uporabnika. Enostavna arhitektura Qdranta prinaša še dodatno prednost pred Milvusom pri postavitvi v oblaku, saj porabijo majhne namestitve Qdranta manj virov kot osnovna namestitev Milvusa. Postavimo tri vozlišča, da zagotovimo odpornost na izpade, vsak drobec ima tri kopije.

6.2.1 Podatkovna shema

Na podlagi nasvetov iz dokumentacije vektorske podatkovne baze Qdrant definiramo eno samo zbirko; ob uporabi več vrst vložitev definiramo več vektorskih polj. Dodatno zbirko ustvarimo, če nameravamo hraniti drugo vrsto podatkov z drugačnimi lastnostmi, nad katerimi bi poizvedbe potekale po drugačnem vzorcu, na primer če bi poleg uporabniških podatkov hranili še zapise iz aplikacijskih dnevnikov, ki bi jih brali vzdrževalci. Takih zapisov bi bilo veliko, poizvedbe pa redke. Tako zbirko bi bilo smiselno postaviti na ločeno gručo, ki ima na razpolago manj procesorjev (ker so poizvedbe redke), a več pomnilnika (ker je podatkov veliko), da optimiziramo rabo virov in ne vplivamo na ostale poizvedbe. Z uporabo posrednika lahko prek nekaj povezav na posrednik komuniciramo z vsemi gručami Qdrant in tako

zmanjšamo število odprtih povezav.

Faktor replikacije zbirke povišamo na tri, da zagotovimo trajno hrambo podatkov; znotraj gruče Raft uporabimo princip konsenza večine. Pod takimi pogoji so pisalne operacije še vedno izvedljive tudi, ko eno vozlišče ni razpoložljivo.

Qdrant od različice 1.19.0 omogoča možnost, da originalne vektorje hranimo v obliki zapisa `turbo4`, poleg oblik `float32`, `float16` in `uint8`. Čeprav shranjevanje originalnih vektorjev v obliki `turbo4` bistveno zmanjša porabo diska, izgubimo originalni vektor. To pomeni, da ob migraciji podatkov iz baze lahko uporabimo ali nenatančen vektor, rekonstruiran iz zapisa `turbo4`, ali pa moramo vektorje ponovno vložiti; poleg tega nudi uporaba ponovne uvrstitve manj natančne rezultate. Vektorje hranimo v obliki `float16`, ki ohrani večji del začetnih informacij, hkrati pa prepolovi rabo diska [20]. Take podatke lahko še migriramo med bazami, ne da bi jih morali ponovno vložiti.

Znotraj sheme določimo nekaj polj koristnega tovora (angl. *payload*), prek katerih natančno urejamo dostop do podatkov, za vsako izmed teh polj ustvarimo kazalo brez ločenega podgrafa HNSW.

Podatke delimo na podlagi identifikatorja uporabnika, shranimo ga znotraj polja najemnika, da so podatki istega uporabnika hranjeni v sosednjih pomnilniških blokih. Izklopimo globalni graf HNSW in namesto njega uporabljamo ločene podgrafe za vsakega uporabnika. Dodamo kazalo nad poljem, ki označuje vir podatka: ali je podatek generiral LLM ali pa je bil ustvarjen prek klica na drugo storitev (na primer rezervacija termina, nakup izdelka ipd.).

Seznam polj, ki jih je smiselno indeksirati, ker so v splošnem uporabna, ne le pri specifičnih virih:

- `najemnik`: ta ključ je glavni filter poizvedbe, uporabljamo ga ob vsaki poizvedbi; ključ je sestavljen tako, da omogoča rabo več vrst najemnikov: prvi del je koda za vrsto najemnika, drugi del pa identifikator najemnika;
- `vrstaVira`: na podlagi te vrednosti lahko ugotovimo, iz katerega vira

podatkov prihaja podatek; tako lahko na primer ločimo, ali je podatek replika podatkov druge storitve ali je zgodovina interakcije z LLM-agentom ipd. Polje uporabimo za iskanje podatkov, ki pripadajo enemu samemu viru;

- `identifikatorVira`: na podlagi vrednosti tega polja lahko najdemo specifičen predmet, ki pripada viru. Identifikator je pomemben, ko želimo poizvedovati po večjih predmetih, ki so bili razdeljeni na več delov in nato vstavljeni v bazo. Zgodovina klepeta z LLM-jem je primer takšnega predmeta: klepet ima en sam identifikator, vsaka izmenjava vnosa in odgovora pa se shrani kot ločen zapis v zbirki;

- `ustvarjenDne`: polje je uporabno, če želimo poizvedovati po podatkih iz določenega časovnega obdobja.

Qdrant poleg tega razlikuje med kazali za polja, ki hranijo nize z majhnim naborom vrednosti, torej ključne besede (`keyword`) – kazala nad ključnimi besedami prilagodi iskanju po ujemanju – in polji, ki hranijo splošne nize (`text`); kazala nad nizi so prilagojena iskanju po vsebini niza [15].

Qdrant ne omejuje sheme metapodatkov, ki jih lahko zapišemo ob točki. Kazala je smiselno uporabiti le nad polji, ki jih lahko uporabljamo kot filter prek vseh virov; če je nek filter zelo specifičen, ni smiselno dodati ločenega kazala, saj lahko filtriramo po najemniku in vrsti vira, pričakujemo pa, da je tak filter dovolj restriktiven, da odsotnost kazala ne poveča bistveno latence poizvedbe.

Za zmanjšanje velikosti grafa uporabljamo kvantizacijo. Za pregled uporabniških podatkov uporabimo rotacijsko kvantizacijo TurboQuant na 4 bite, ki omogoča dober kompromis med priklicem in kompresijo [18], saj enakomerno porazdeli informacijo, hranjeno v vektorju, med vse komponente.

Tabela 6.1: Pregled polj, na katerih definiramo kazala nad tovorom

Polje	**Vrsta podatka**	**Vrsta kazala**	**Primer vrednosti**
`najemnik`	Ključna beseda	Najemniško kazalo z ločenimi podgrafi HNSW	`"uporabnik:34018"`
`vrstaVira`	Ključna beseda	Kazalo nad tovorom, brez podgrafa	`"spomin_agenta"`
`identifikatorVira`	Ključna beseda	Kazalo nad tovorom, brez podgrafa	`"3ae27cd8f92a1b"`
`ustvarjenDne`	Časovni žig	Kazalo nad tovorom, brez podgrafa	`"2026-08-15"`

6.3 Vmesnik VDB

Vmesnik za dostop do VDB implementiramo v Javi na podlagi ogrodja Quarkus, ki omogoča razvoj učinkovitih mikrostoritev z nizko porabo virov. Za dostop do Qdrant VDB uporabimo uradno knjižnico odjemalcev Qdrant za Javo. Storitev je izpostavljena prek REST API-ja. Na vmesniku implementiramo preverjanje avtorizacije za pisanje in branje v VDB. Poleg tega implementiramo logiko, vezano na predelavo vnesenih podatkov: vmesnik VDB pretvori vnesene dokumente v besedilo (to lahko implementiramo na več načinov, na primer prek hevristične predelave vsebine datoteke v besedilo ali prek OCR-ja) in vloži podatke. Na podlagi vnosa podatkov sestavi klic na VDB.

Pri definiciji končnih točk, ki jih izpostavlja vmesnik, je glavni cilj, da je definicija enostavna, možnosti uporabe pa številne in prilagodljive.

Vmesnik izpostavlja dve končni točki, namenjeni uporabnikom, in dve

končni točki, namenjeni storitvam za pisanje in branje v podatkovno bazo. Poleg teh štirih izpostavlja javno še končno točko za pridobivanje vrst virov, ki jih VDB hrani, končne točke za dodajanje, brisanje in posodabljanje vrst virov pa niso javno izpostavljene.

Izpostavimo tudi strežnik MCP, ki ponuja iste funkcionalnosti kot končne točke, izpostavljene prek javnega REST API-ja, torej branje in pisanje v bazo. Primer implementacije vmesnika je dostopen na `https://github.com/peter-savron/vmesnik-vdb`.

6.4 Varnost

Pri shranjevanju velike količine podatkov več uporabnikov na eno samo mesto se pojavi več tveganj. Prva vrsta tveganj so tveganja, vezana na varnost zasebnih podatkov v skladu z GDPR, druga vrsta tveganj pa so tveganja, vezana na integriteto podatkovne sheme, hranjene v bazi, saj so podatki, ki pripadajo več virom, shranjeni v isti zbirki. Glavno orodje, ki ga uporabimo za zavarovanje delovanja VDB, je OpenFGA. OpenFGA je avtorizacijska storitev, ki omogoča definicijo natančnih shem pravil omejitve dostopa [53]. V našem primeru izkoristimo OpenFGA za zapis pravil dostopa uporabnikov do podatkov in pravil glede vnosa in branja podatkov, vezanih na vrsto vira.

Varstvo osebnih podatkov se začne s tem, da ne dovolimo nepooblaščenim uporabnikom dostopa do zasebnih podatkov. Ker ima vsak uporabnik aplikacije svojo particijo na podlagi polja `najemnik`, lahko definiramo osnovni relaciji med uporabnikom in particijo podatkov `bralec` in `pisec`. V osnovni implementaciji ustvarimo povezavo le med uporabnikom in njegovo particijo podatkov. Drugi izzivi na področju varstva podatkov so skladnost upravljanja podatkov z GDPR. Brisanje uporabniških podatkov pri VDB ni enostavno, saj zaradi zahtev vektorskih kazal ne smemo odstranjevati vozlišč iz grafa, ampak jih le označimo kot odstranjene. Ob brisanju točke Qdrant ohrani vektor v grafu HNSW, točka pa postane nedosegljiva prek API-jev.

Ker GDPR zahteva, da “upravljavec brez nepotrebnega odlašanja izbriše

osebne podatke" [35], moramo zagotoviti, da so mehko izbrisani uporabniški podatki (to so vektorji v vektorskem kazalu) odstranjeni v čim krajšem času, saj se zapisi vektorjev ohranijo v grafu HNSW in se označijo kot izbrisani; šele ob ponovni gradnji kazala se bodo izbrisane točke dokončno odstranile. Prva strategija je znižanje vrednosti `deleted_threshold`, s čimer se prej sproži ponovna gradnja grafa in torej dokončno brisanje podatkov. Drugi pristop k temu problemu je dodelitev ločenega drobca vsakemu uporabniku, vendar je ob veliki množici uporabnikov z malo podatki tak pristop po uradni dokumentaciji odsvetovan [17], ker porabi veliko virov. Podatke zavarujemo pred nepooblaščeno uporabo tako, da določimo, katera storitev lahko bere vnose pripadajočega tipa vira znotraj zbirke. Dodaten izziv je omogočanje uporabniku izvoza kopije njegovih podatkov, ki so hranjeni v storitvi. Qdrant izpostavlja končno točko, ki omogoča paginirano iskanje po točkah v zbirki [19]; s pomočjo tega API-ja lahko zberemo vse podatke, shranjene v bazi, in jih posredujemo uporabniku, kot to zahtevajo predpisi. Ker zapisi v VDB vsebujejo uporabnikove zasebne podatke, je lahko raba podatkov v bazi namenjena le uporabniku samemu, ne pa sistemom za analitiko in predelavo podatkov [34, 35, 63].

Druga težava je zagotavljanje verodostojnosti podatkov, vnesenih v VDB. Ker je vmesnik za dostop do VDB en sam, mora biti uporabnik javnih končnih točk avtenticiran. Uporabniki končnih točk, namenjenih storitvam, pa morajo specificirati, za katero vrsto odjemalca gre. Vrsta odjemalca je lahko dovolj splošna, da obsega celo kategorijo, kot sta uporabnik ali LLM-agent, ali pa bolj specifična, ki zajema le posamezno storitev. Na primer, samo odjemalci, ki so vrste storitve za planiranje, lahko spreminjajo podatke v VDB, ki zadevajo vir koledar. Na tak način lahko zagotovimo, da je shema podatkov vnosov vira koledar v zbirki konsistentna s shemo, ki jo uporablja storitev za planiranje. Ostale storitve in agenti lahko še vedno planirajo dejavnosti, vendar prek rabe končnih točk, ki jih izpostavlja storitev za planiranje. Tako onemogočimo, da bi posamezen odjemalec spreminjal podatke, ki se ga ne tičejo.

Odsek kode 6.1: Osnovna konfiguracija OpenFGA sheme

```
model
  schema 1.1

type uporabnik

type sistem
  relations
      define administrator: [uporabnik]


type vdb_particija_podatkov
  relations
      define pooblascen_uporabnik: [uporabnik]

      define lahko_bere: pooblascen_uporabnik
      define lahko_pise: pooblascen_uporabnik

type vrsta_odjemalca

type vrsta_vira
  relations
      define pisec: [vrsta_odjemalca]
      define bralec: [vrsta_odjemalca]
      define vdb_pisec: [vrsta_odjemalca]
      define vdb_bralec: [vrsta_odjemalca]

      define lahko_pise: pisec
      define lahko_bere: bralec
      define lahko_pise_vdb: vdb_pisec
      define lahko_bere_vdb: vdb_bralec
```

6.5 Posrednik

Med vmesnikom VDB in gručo Qdrant postavimo dodatnega posrednika, ki upravlja omrežje. Namen posrednika (v našem primeru je to Envoy) je

zmanjšati število odprtih povezav na vmesniku VDB in na posameznem vozlišču Qdrant ter enakomerno porazdeliti klice po vozliščih. Qdrant stroki so namreč izpostavljeni prek brezglave storitve, zato nimajo skupnega navideznega naslova IP. Dodatna prednost je, da lahko prepustnost upravljamo na posredniku in tako razbremenimo gručo. Uporabimo Envoy, hiter posrednik z majhno porabo virov [1].

Odsek kode 6.2: Del konfiguracije Envoy posrednika, ki preslika vrednost glave zahteve v metapodatek za usmerjanje zahtevka

```
          http_filters:
          - name: envoy.filters.http.header_to_metadata
            typed_config:
              "@type": type.googleapis.com/envoy.extensions.
                 filters.http.header_to_metadata.v3.Config
              request_rules:
                - header: "x-cluster-name"
                  on_header_present:
                    metadata_namespace: "envoy.lb"
                    key: "name"
                    type: STRING
                  on_header_missing:
                    metadata_namespace: "envoy.lb"
                    key: "name"
                    value: "qdrant-osnovni"
                    type: STRING
```

Poglavje 7

Testiranje nastavitev Qdrant

Glavni predmet testiranja je konfiguracija VDB. Zanima nas, katera konfiguracijska vrednost v največji meri vpliva na zmogljivost VDB. Rezultate testiranja bomo uporabili, da prilagodimo nastavitev VDB potrebam naše aplikacije: te so večnajemništvo, visoka prepustnost in varčevanje z viri. Kot glavna merila zmogljivosti konfiguracije smo izbrali število poizvedb na sekundo, hitrost poizvedbe in priklic poizvedbe.

Prepustnost oziroma število poizvedb na sekundo je najpomembnejša metrika za aplikacije, ki morajo hkrati streči več odjemalcem, npr. spletne aplikacije, algoritmi za priporočanje vsebin ipd. Hitrost poizvedbe oziroma latenca je pomembnejša za sisteme, ki morajo hitro sprejemati odločitve, npr. sistem za pregledovanje bančnih transakcij, sistem za prepoznavanje obrazov na posnetkih varnostnih kamer v živo itd. Priklic poizvedbe je najpomembnejši za naloge, ki zahtevajo čim bolj natančne podatke; take naloge so npr. analiza podatkov, pomoč pri pripravi diagnoze itd. Pričakujemo, da sta latenca in prepustnost korelirani in da se priklic manjša ob povečanju prepustnosti.

Uporabimo `vector-db-benchmark`[59] za merjenje zmogljivosti pri rahlo spremenjenem naboru vrednosti `random-768-100-tenants`, pri čemer ohranimo isti nabor podatkov v podatkovni bazi, število poizvedb pa desetkrat povečamo tako, da dodamo kopije obstoječih testnih poizvedb v datoteko

`tests.jsonl`, da evalviramo podatke o zmogljivosti na podlagi večjega nabora poizvedb, saj bi pri velikem številu vzporednih poizvedb majhna količina testnih primerov lahko popačila vrednost repa latence. Tak pristop izboljša verodostojnost podatkov, ne izboljša pa natančnosti meritve priklica, ker se bodo poizvedbe ponavljale. Pri veliki količini sočasnih klicev se namreč zgodi, da ima en počasen klic velik vpliv na skupni čas izvedbe in torej na izmerjene metrike. Ker Qdrant ne hrani rezultatov preteklih poizvedb, ponavljanje poizvedb ne bo popačilo meritev latence in prepustnosti. Nabor vrednosti hrani milijon ločenih vnosov; v prvotni verziji je naboru pripadalo 200 poizvedb. Točke v bazi so enakomerno porazdeljene med 100 najemnikov, vsakemu pripada okoli 10.000 točk. Vektorji imajo 768 razsežnosti in so bili naključno ustvarjeni, vsaka komponenta vektorja je pozitivno število na intervalu $[0, 1]$. Priklic merimo kot delež pravih 25 najbližjih sosedov, ki jih vrne iskanje 25 najbližjih sosedov. Definicijo razlike med priklicem in natančnostjo najdete v poglavju 5.2. Taka sestava nabora podatkov nam omogoča, da testiramo poizvedbe filtrirane po najemnikih.

Prilagoditve izvirnega repozitorija in rezultati so na voljo na `https://github.com/peter-savron/vector-db-benchmark`.

Glavni namen meritev je raziskati zmogljivosti različnih nastavitev VDB v robnih primerih, ki jih običajne meritve [25, 64] ne pokrivajo, saj se običajno meri zmogljivost na sistemih, ki imajo na voljo veliko količino virov za izvajanje poizvedb. V produkcijskih aplikacijah, kjer so viri omejeni, da se omeji stroške vzdrževanja, bo torej manj virov na voljo kot jih je bilo na voljo za poganjanje meritev.

7.1 Okolje

Teste smo izvedli na osebnem računalniku z Intel®Core™Ultra 5 125H procesorjem, dva modula po 32 GB DDR5 pomnilnika z maksimalno hitrostjo 5600 MT/s in Crucial CT2000P310SSD8 NVMe diskom po 2 TB, ki pri branju dosega maksimalno hitrost 7.100 MB/s.

Testirali smo tako gručo Qdrant vozlišč kot samostojno Qdrant vozlišče. Osnovnemu vozlišču so dodeljeni 2 procesni enoti in 4 GB pomnilnika. Vire, ki jih imajo vozlišča na voljo, smo v različnih fazah testiranja spreminjali.

Dostop do gruče upravlja vmesnik Envoy, ki porazdeli promet med vozlišči. V realnih situacijah nam tak vmesnik omogoča, da zmanjšamo število odprtih povezav na Qdrant vozlišča ter omejimo število zahtev, ki so hkrati poslane gruči.

Gručo postavimo na osebni računalnik, na katerem uporabljamo kind [3] Kubernetes gručo. Qdrant gruče bomo namestili prek uradne `qdrant/qdrant` helm predloge.

Odjemalec in Qdrant gruča sta se izvajala na istem računalniku, da bi zmanjšali vpliv omrežja na rezultate meritev.

Komponenta	**Model/Verzija**
CPU	Intel®Core™Ultra 5 125H
Pomnilnik	Kingston 9905789-161.A00G
NVMe disk	Crucial CT2000P310SSD8
Operacijski sistem	Ubuntu 24.04.4 LTS
Kubernetes	kind v1.35.0
Qdrant	1.19.0

Tabela 7.1: Okolje, v katerem smo izvajali teste.

Odsek kode 7.1: Primer konfiguracije vrednosti helm predloge za Qdrant gručo

```
replicaCount: 3

config:
  cluster:
    enabled: true
  storage:
    storage_driver:
      on_disk_payload: true

```

```
env:
  - name: QDRANT__SERVICE__API_KEY
    valueFrom:
      secretKeyRef:
        name: qdrant-auth
        key: root-api-key

resources:
  requests:
    cpu: "2"
    memory: "4Gi"
  limits:
    cpu: "2"
    memory: "4Gi"

persistence:
  storageClassName: standard
  size: 16Gi
```

7.2 Spremenljivke

Pri različnih testih smo spreminjali parameter `m` grafa HNSW. `m` predstavlja največje število povezav na vozlišče v grafu na posameznem sloju. Večji `m` pomeni bolje povezan in s tem natančnejši graf, vendar zahteva več pomnilnika. `m` lahko nastavimo na 0, `payload_m` pa na želeno vrednost – v tem primeru se ne ustvari graf nad vsemi točkami, temveč graf povezuje točke z ujemajočimi se metapodatki (npr. točke, ki pripadajo istemu uporabniku). Če hkrati nastavimo `m` in `payload_m`, naenkrat ustvarimo več grafov, kar poveča porabo pomnilnika. Qdrant strukturam, hranjenim v bazi, omogoča nastavitev parametra `memory`, ki določa, kako so hranjene znotraj vozlišč. Qdrant ponuja več nastavitev hrambe za posamezne vrste podatkov, ki jih nastavljamo prek parametra `memory`. Pri testih ohranimo privzete nastavitve Qdrant: originalni vektorji in vektorska kazala so hranjeni kot strukture `cached`, kar pomeni, da so podatki na disku predhodno naloženi v pomnilnik,

zato poizvedbe na začetku testov ne trpijo za povečano latenco; kazala nad metapodatki in kvantizirani vektorji pa so hranjeni kot strukture `pinned`, kar pomeni, da so podatki vedno v pomnilniku. Nastavitve `cold` ne uporabimo, saj bi upočasnila začetne poizvedbe v bazo [16].

Naslednje nastavitve so namenjene porazdeljenim podatkovnim bazam. `shard_number` določa število drobcev, na katere je zbirka razdeljena; privzeto je to enako številu vozlišč (v našem primeru 3), drobci pa so enakomerno porazdeljeni po vozliščih. `sharding_method` določa način deljenja točk med drobci. Privzeta nastavitev je `auto`, ki porazdeli točke na podlagi identifikatorja; če jo nastavimo na `custom`, lahko ob vnosu nove točke sami določimo, v kateri drobec bo shranjena. `replication_factor` določa, na kolikih vozliščih se hrani kopija istega drobca (privzeto 1). Pri višjih vrednostih se poveča raba diska in pomnilnika (ker so vektorska kazala predpomnjena v pomnilniku, se raba pomnilnika ne poveča multiplikativno, temveč se kazalo med poizvedbami večkrat odstrani iz pomnilnika in ga moramo znova prebrati z diska, kar upočasni poizvedbe).

Uporabili smo tudi nekatere nastavitve za kvantizacijo. Pri kvantizaciji smo predvsem izbrali uporabljen postopek in faktor pokrčenja vektorja. Velikost posameznega vektorja lahko zmanjšamo do 64-krat.

Pri poizvedbah sta glavna parametra število vzporednih klicev `parallel` (kratica p) in širina poizvedbe `hnsw_ef`, to je velikost dinamičnega seznama kandidatov, ki se uporablja med K-ANNS po grafu HNSW.

7.3 Nastavitve kazala nad metapodatki

Namen prvega kroga meritev je ugotoviti, kako nastavitve kazala nad metapodatki vektorja vplivajo na zmogljivost podatkovne baze. Ločimo več scenarijev:

- **Brez kazala nad metapodatki:** za implementacijo tega scenarija zakomentiramo del obstoječe kode merila, ki privzeto ustvarja kazala nad metapodatki. Pričakujemo, da so poizvedbe brez rabe kazala nad

tovorom počasne, saj morajo prehoditi globalen graf HNSW in sproti zavračati kandidate.

- **Kazalo nad metapodatki brez podgrafa:** zbirko vzpostavimo s kazalom nad metapodatki, vendar brez ločenega podgrafa HNSW. Namen je ovrednotiti pohitritev, ki jo dobimo že z navadnim kazalom nad skalarjem.

- **Kazalo s podgrafom HNSW:** kazalu dodamo še podgraf HNSW, ki povezuje točke z enako vrednostjo metapodatka. Pričakujemo, da bo ta pristop še hitrejši.

- **Popolno preiskovanje (angl. *full scan*):** mejo iskanja prek vseh vektorjev (nastavitev `full_scan_threshold`), ki se ujemajo s filtrom, nastavimo tako visoko, da ob vsaki poizvedbi preskočimo uporabo grafa HNSW in vektor poizvedbe primerjamo z vsemi ostalimi vektorji.

- **Optimizacija za polje najemnika:** polje nastavimo kot najemnika, kar sproži dodatno raven optimizacije grafa, pri čemer onemogočimo ustvarjanje podgrafov na podlagi metapodatkov.

- **Podgraf najemnika brez globalnega grafa:** na podlagi vrednosti polja najemnika ustvarimo podgraf in izklopimo globalni graf.

7.3.1 Rezultati

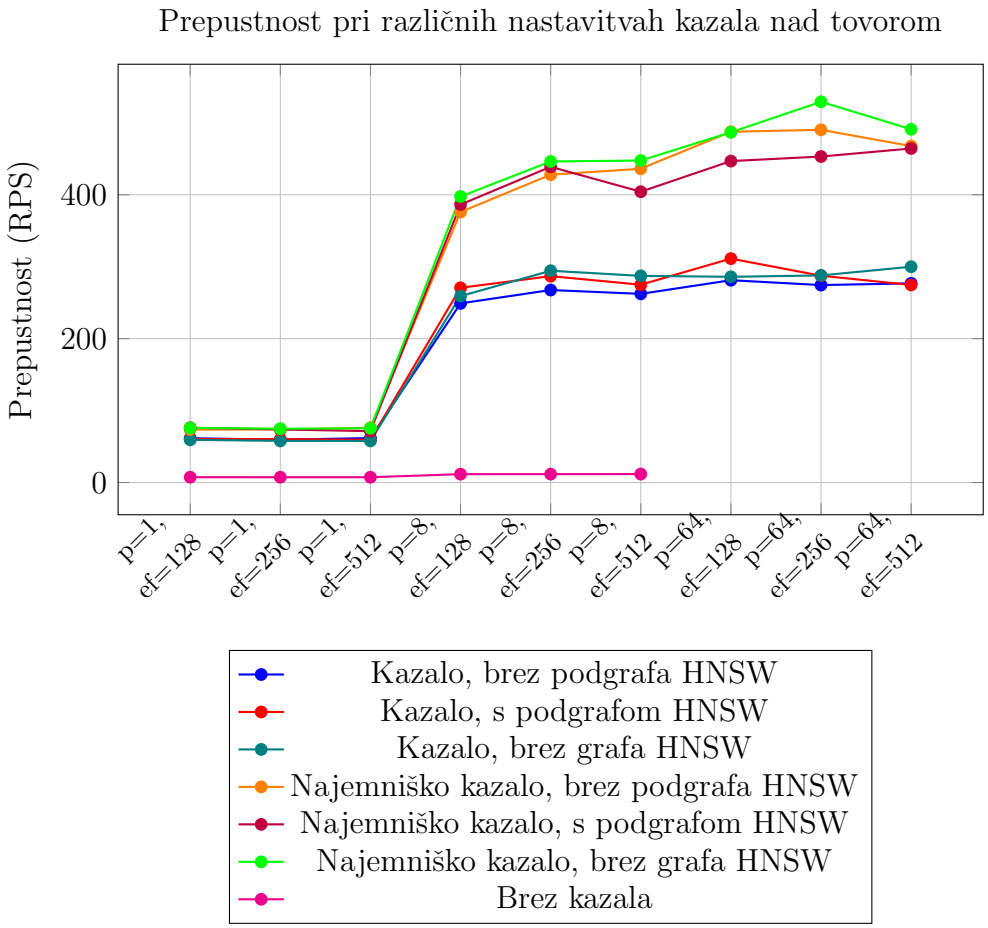
Ob odsotnosti kazala nad metapodatki se je sprožilo iskanje po celotnem grafu, kar je bilo časovno izjemno potratno, toliko, da je bila povezava do VDB ob visoki ravni vzporednih klicev po 30 sekundah prekinjena.

V primerih, ko nismo ustvarili ločenega podgrafa, se hitrost poizvedb ni zmanjšala ali se je celo povečala, vendar se je bistveno zmanjšal priklic, poleg tega se je čas gradnje grafa HNSW občutno povečal. Opazimo, da pri poizvedbi prek globalnega grafa povečanje `hnsw_ef` ne izboljša priklica. Razlog za to je zelo verjetno raba zelo selektivnega filtra nad najemnikom,

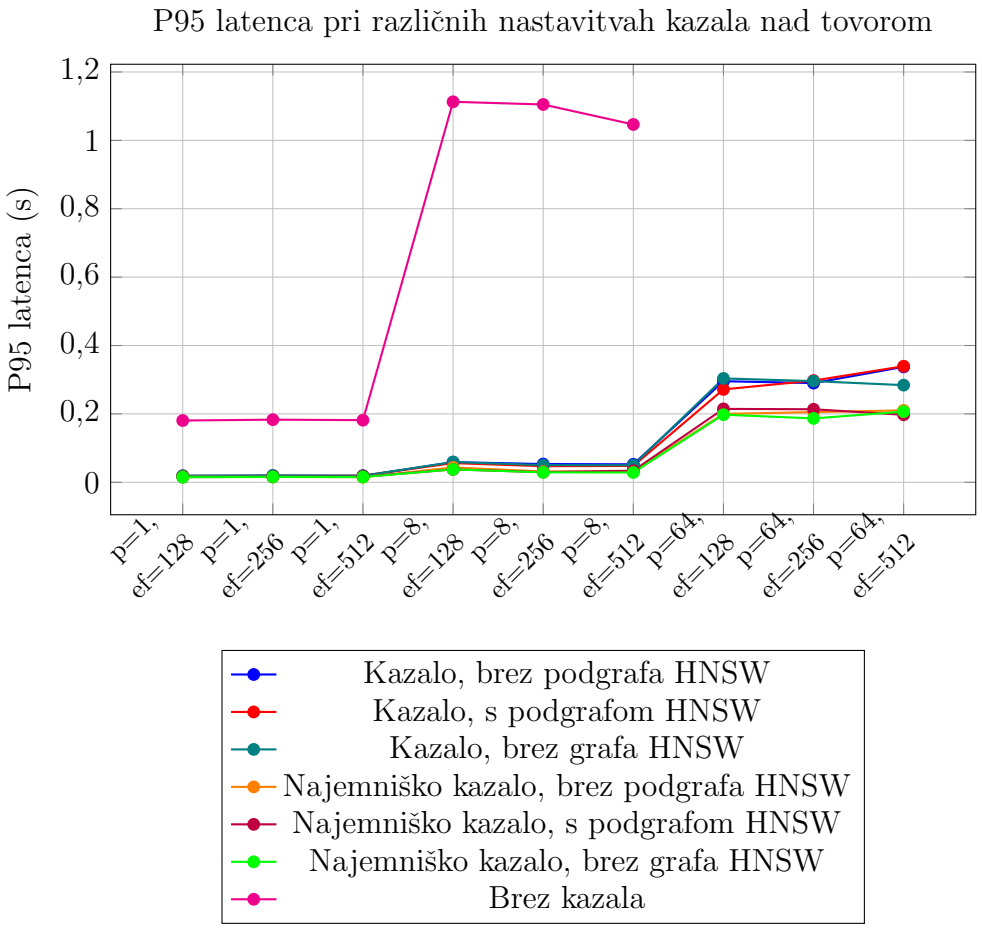
ki omeji učinkovitost globalnega grafa: obhod ne obišče vozlišč, ki so iz vstopne točke nedosegljiva prek povezav med vozlišči, ki ustrezajo filtru. Ker je filtriran podgraf globalnega grafa pri zelo selektivnem filtru redek in pogosto nepovezan, priklica ne omejuje velikost seznama kandidatov, temveč dosegljivost; zato povečevanje `ef` priklica ne izboljša. Pri rabi podgrafov na podlagi vrednosti polja ostane graf povezan in spreminanje vrednosti `ef` se obnaša kot pričakovano. Če smo poizvedbo pognali prek primerjave vseh vektorjev, je bilo število teh dovolj majhno, da ni imelo večjega vpliva na hitrost poizvedbe, pri tem pa je priklic dosegel 100 %.

Opazimo pa bistveno razliko med poizvedbami prek filtra nad navadnim poljem in filtra nad poljem najemnika. Poizvedbe po slednjem so bistveno hitrejše, saj so podatki z isto vrednostjo polja hranjeni v istih odsekih znotraj drobcev, poleg tega se zapisi nahajajo v sosednjih blokih pomnilnika oziroma diska, kar zmanjša čakanje na nalaganje podatkov iz pomnilnika v predpomnilnik oziroma iz diska v pomnilnik.

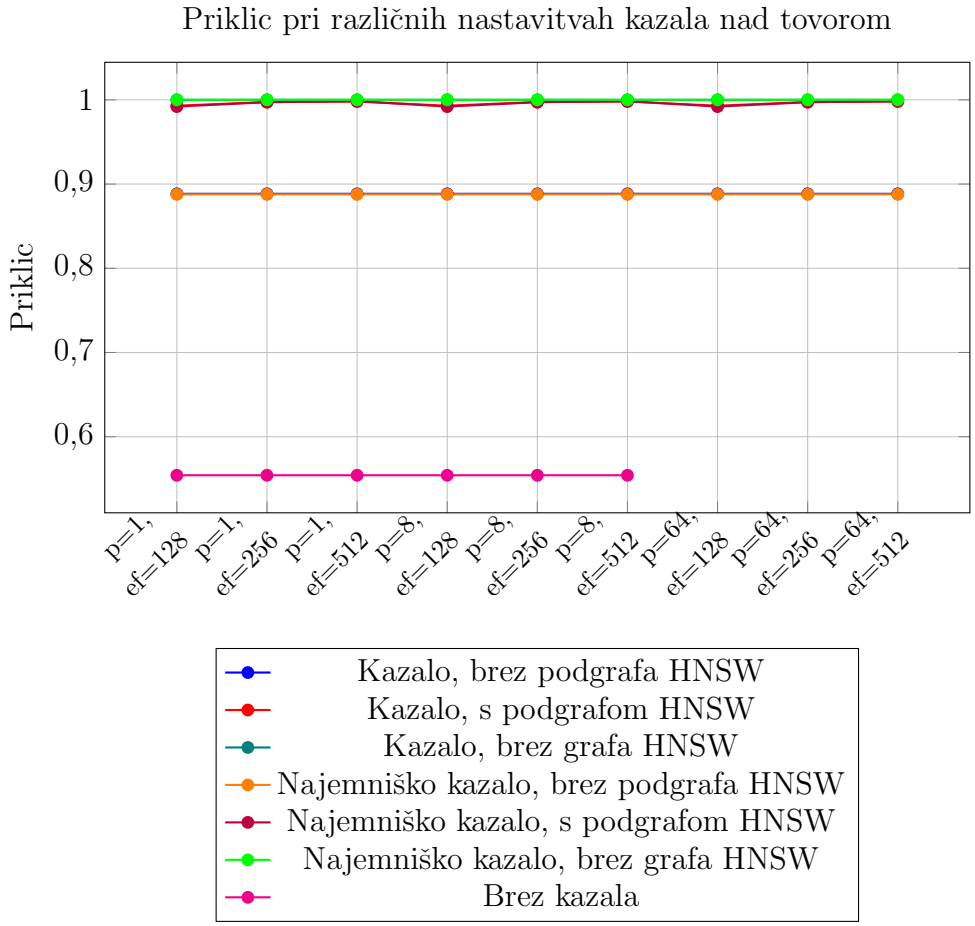
Verjeten razlog, zakaj so poizvedbe ob uporabi globalnega grafa hitrejše, je ta, da se sprehod po grafu prej zaključi, ker so povezave med vozlišči, ki ustrezajo filtru, redke.



Slika 7.1: Primerjava prepustnosti pri različnih nastavitvah kazala nad tovorom.



Slika 7.2: Primerjava P95 latence pri različnih nastavitvah kazala nad tovorom.



Slika 7.3: Primerjava priklica pri različnih nastavitvah kazala nad tovorom.

7.4 Nastavitve gruče Qdrant

Cilj te serije testov je določiti, kako nastavitve gruče Qdrant, predvsem število drobcev in replik segmentov, vplivajo na njeno zmogljivost. Testiramo ločitev na tri drobce z eno repliko, kar je privzeto obnašanje Qdranta, ločitev na tri drobce s tremi replikami, pri čemer vsako vozlišče hrani celotno zbirko, razdeljeno na drobce, ločitev na en drobec s tremi replikami, kjer, podobno kot prej, vsako vozlišče VDB hrani celotno zbirko, vendar so podatki razdeljeni le na odseke znotraj edinega drobca, ter ločitev na en drobec z eno repliko, s katero prikažemo, kaj se zgodi, ko zbirko hrani le del vozlišč gruče. Nazadnje postavimo eno samo vozlišče, kateremu dodelimo skupne vire, ki jih je imela na voljo gruča (torej 6 procesnih enot in 12 GB pomnilnika), da primerjamo horizontalno skalabilnost gruče z vertikalno. Pri vseh testih postavimo kazalo nad poljem najemnika in na podlagi tega polja ustvarimo podgraf HNSW. Vozlišča gruče bodo imela na voljo 2 CPU in 4 GB RAM, razen v primeru, ko testiramo eno samo vozlišče.

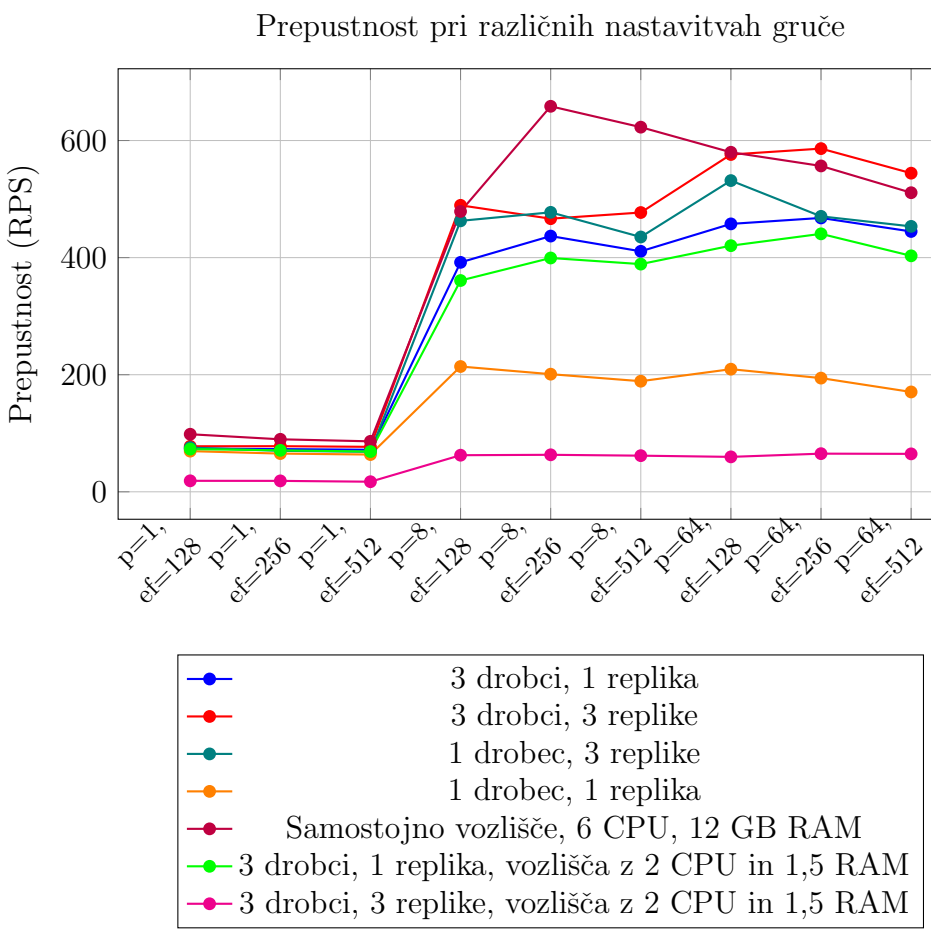
7.4.1 Rezultati

Najprej se osredotočimo na analizo prepustnosti vozlišč. Na tem področju se izkaže, da je eno samo vozlišče najboljša izbira, saj doseže najvišjo prepustnost. Uporaba treh replik omogoča boljšo prepustnost, saj vsako vozlišče hrani celotno zbirko in zato ni potrebe, da bi poizvedbe koordiniralo z ostalimi vozlišči. Poizvedbe so nekoliko hitrejše, če je zbirka še vedno razdeljena na tri drobce. Če se zbirka razdeli na več drobcev, ki niso replicirani, se prepustnost rahlo zmanjša, saj mora poizvedba potekati na treh ločenih vozliščih. Če je zmnožek števila drobcev in replik manjši od števila vozlišč, se vsa vozlišča pri izvajanju poizvedb ne izkoristijo in prepustnost se zmanjša. Če zbirko razdelimo na več drobcev pod tem pogojem in po odsekih razpršimo točke najemnika, bo poizvedba natančnejša, saj prehodimo tri manjše grafe in združimo rezultate.

Dodatno smo analizirali še dva scenarija ob pogoju, da so viri, ki so na

razpolago za vsako vozlišče, omejeni na 2 procesorja in 1,5 GB pomnilnika: tri drobce brez replikacije ter tri drobce, replicirane po vozliščih. Pri manjši količini pomnilnika se poizvedba upočasni le v primeru, ko je faktor replikacije 3, ker posamezno vozlišče ne more naložiti celotnega grafa v pomnilnik, vendar skuša izvesti celotno poizvedbo na enem vozlišču, ker ima to možnost na razpolago. Ko je faktor replikacije 1, se poizvedbe ne upočasnijo, ker se graf HNSW porazdeli med vsa tri vozlišča, ki sodelujejo med seboj.

Če pa se ozremo na priklic, opazimo, da se ob ločevanju zbirke na več drobcev priklic poveča: za vsak drobec se namreč posebej poišče k najbližjih sosedov, nato pa se rezultati združijo.



Slika 7.4: Prepustnost različnih nastavitev Qdrant gruče.

7.5 Vrste kvantizacije

Za vse teste, povezane s kvantizacijo, smo uporabili eno samo vozlišče Qdrant. Najprej smo testirali več načinov kvantizacije – skalarno, kvantizacijo produkta, binarno in rotacijsko.

Testirali smo večino načinov kvantizacije, ki so na voljo v Qdrantu:

- **skalarno kvantizacijo** pri 8-kratni kompresiji;
- **kvantizacijo produkta** pri 4-, 16- in 64-kratni kompresiji;
- **binarno kvantizacijo** na 2- in 1-bitno vrednost;
- **rotacijsko kvantizacijo** na 4-, 2- in 1-bitno vrednost.

Izmerili smo metrike pri več vrednostih `hnsw_ef` in pri različnih številih sočasnih poizvedb. Poleg tega smo preverili vpliv ponovne uvrstitve rezultatov pri različnih faktorjih vzorčenja.

7.5.1 Rezultati

Izkaže se, da kvantizacije, ki informacijo vsake razsežnosti skrčijo na manj kot 4 bite, bistveno poslabšajo priklic. 16-kratna kvantizacija produkta in rotacijska kvantizacija na 2 bita lahko izboljšata priklic, če vzorčimo večje število točk, vendar to zmanjša prepustnost VDB.

Kvantizacija produkta je z vidika gradnje kazala bolj potratna, saj zahteva daljšo učno fazo, da se določijo centroidi posameznih podprostorov. Rotacijska kvantizacija porabi nekaj dodatnega časa pri gradnji kazala, saj mora rotirati vsak vektor, medtem ko skalarna in binarna kvantizacija porabita najmanj časa, ker so preslikave vrednosti vektorja trivialne.

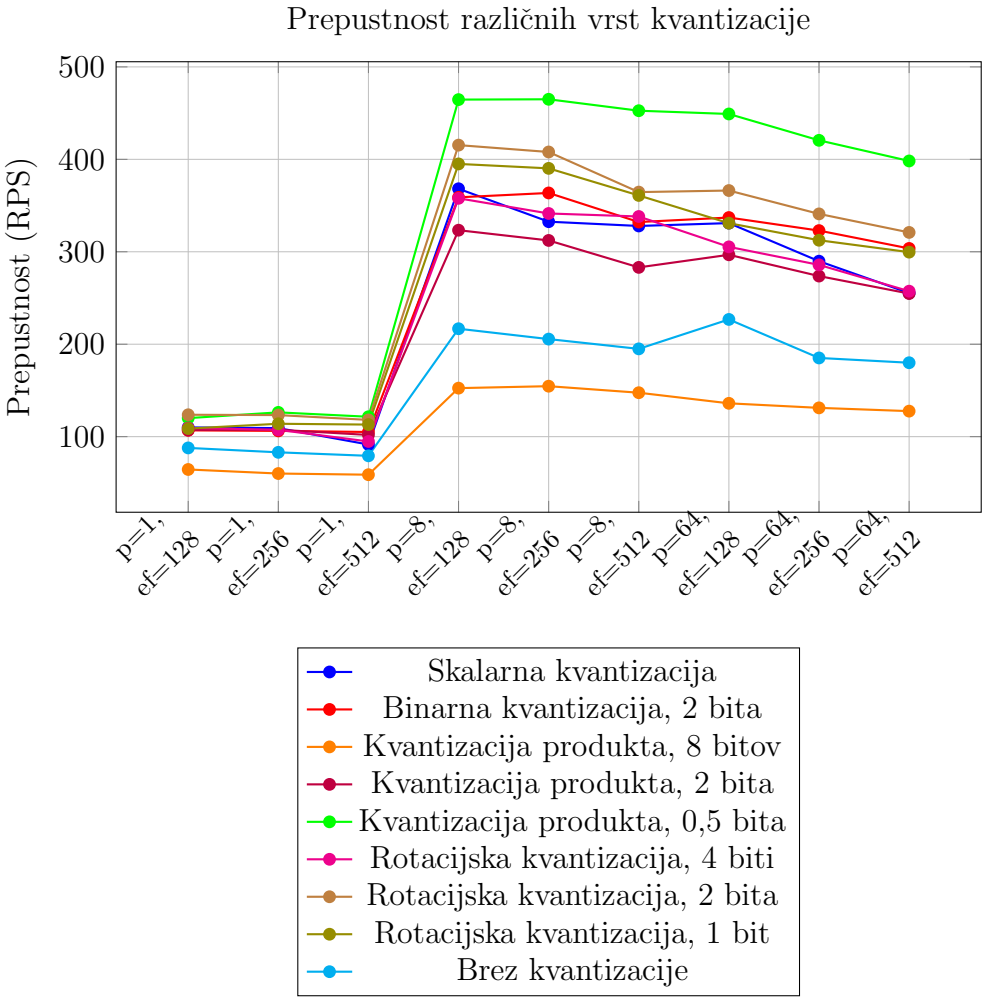
Skalarna kvantizacija in rotacijska kvantizacija na 4 bite sta primerljivi v vseh merilih, pri čemer velja omeniti, da je kompresija pri rotacijski kvantizaciji dvakrat večja. S povečevanjem kompresije se prepustnost rahlo poveča, vendar se priklic bistveno zmanjša. Če kot najmanjši dopusten priklic vzamemo 98 %, lahko pri rotacijski kvantizaciji na 2 bita uporabimo štirikrat

večji vzorec in dosežemo podobno prepustnost kot pri skalarni in rotacijski kvantizaciji na 4 bite, vendar moramo vzorčiti večje število točk. Če 99 % vzamemo za najmanjši dopusten priklic, se velikost vzorca pri rotacijski kvantizaciji na 2 bita bistveno poveča, kar zmanjša prepustnost, medtem ko se pri ostalih dveh prepustnost bistveno ne spremeni.

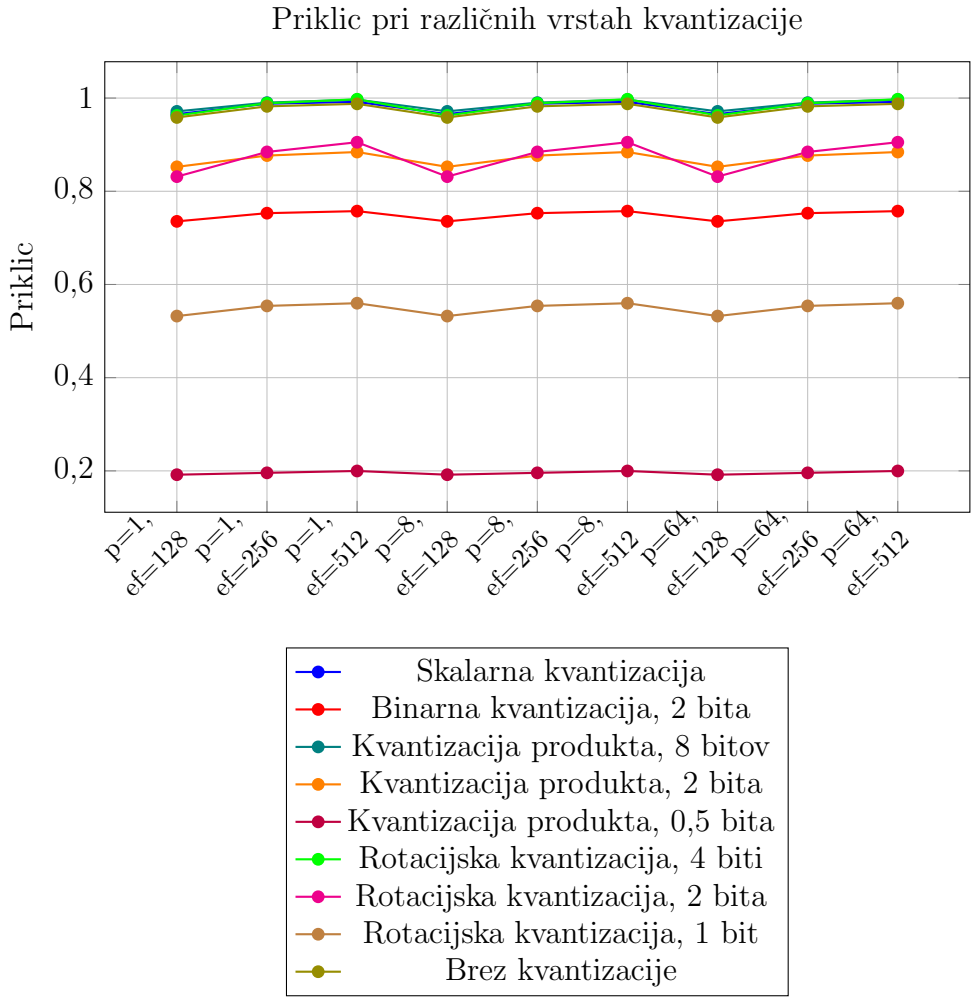
Prepustnost pri binarni kvantizaciji ni verodostojna, saj se vsi vektorji preslikajo v isti prikaz. Ker BQ hrani predznak komponente, so vse komponente vektorjev iz tega nabora pozitivne; če pogledamo priklic, vidimo, da je enak priklicu pri naključni izbiri. Rezultate testiranja BQ lahko zato obravnavamo kot maksimalno dosegljivo prepustnost pri danih parametrih, ne pa kot natančno merilo prepustnosti binarne kvantizacije.

Pri kvantizacijah z nizko kompresijo uporaba ponovne uvrstitve rezultatov bistveno izboljša priklic, medtem ko povečanje vzorca izboljša priklic le pri velikih faktorjih, kar pa poslabša prepustnost. Povečanje parametra `hnsw_ef` zagotavlja izboljšanje rezultatov poizvedbe. Ob uporabi ponovne uvrstitve rezultatov je priklic primerljiv s sprehodom po grafu HNSW brez kvantizacije.

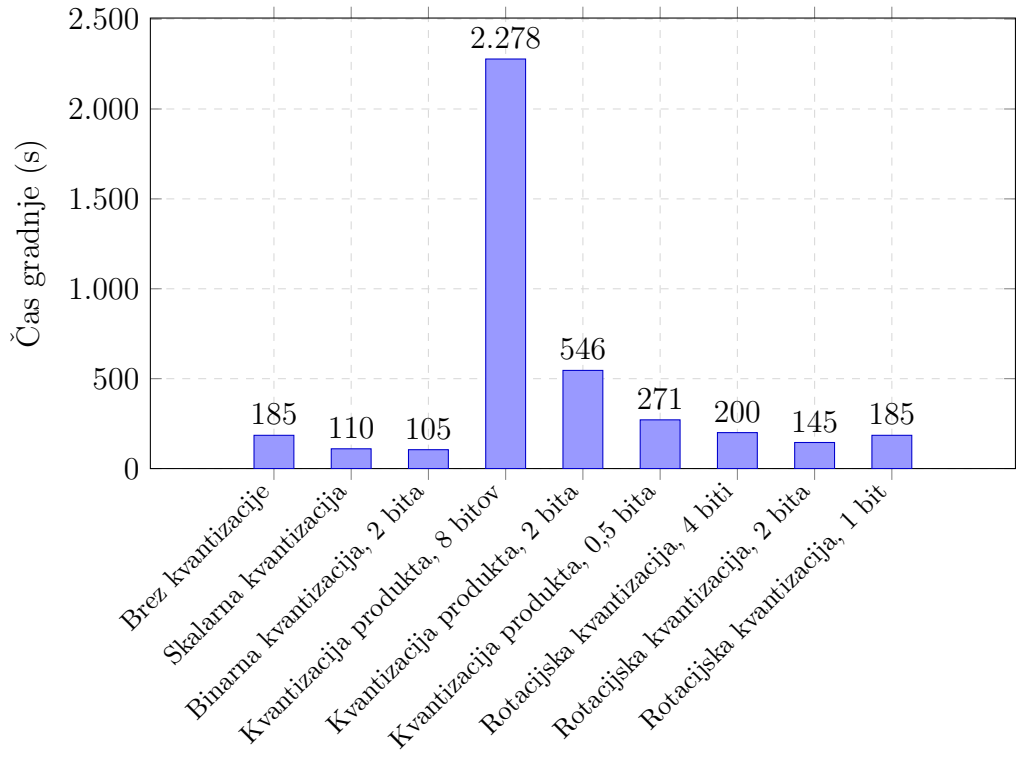
Pri rotacijski kvantizaciji na 2 bita povečanje parametra `hnsw_ef` izboljša priklic, vendar je začetni priklic bistveno slabši. Povečanje vzorca, uporabljenega pri ponovni uvrstitvi rezultatov, v tem primeru v vseh izbranih primerih še dodatno poveča priklic.



Slika 7.5: Primerjava prepustnosti pri različnih vrstah kvantizacije, samostojno vozlišče 2 CPU in 4 GB RAM. Faktor vzorčenja $v = 2,0$. Vse meritve so na voljo na `https://github.com/peter-savron/vector-db-benchmark`.



Slika 7.6: Priklic pri različnih vrstah kvantizacije, samostojno vozlišče 2 CPU in 4 GB RAM. Faktor vzorčenja $v = 2,0$. Vse meritve so na voljo na `https://github.com/peter-savron/vector-db-benchmark`.



Slika 7.7: Primerjava časa gradnje kazala pri različnih vrstah kvantizacije

7.6 Vertikalno skaliranje vozlišč

Nastavitve iz prejšnjega dela, ki kažejo najboljši potencial, testiramo še pod dodatnimi pogoji. Najprej spremenimo število procesorjev; pričakujemo, da se prepustnost sorazmerno spreminja, nato pa zmanjšamo količino razpoložljivega pomnilnika. Pričakujemo, da izgubi v tem primeru skalarna kvantizacija več prepustnosti kot preostali dve, ker kazalo porabi več prostora v pomnilniku.

Pri poizvedbah uporabimo več faktorjev vzorčenja, da bomo lahko analizirali vpliv različnih kombinacij širine poizvedbe in faktorja vzorčenja na prepustnost in priklic.

Ob upoštevanju kompresije, ki jo omogoča posamezna kvantizacija, ter stopnje grafa HNSW lahko ocenimo porabo pomnilnika za hrambo strukture grafa HNSW na približno $2m \times 32 \times$ št_vektorjev bitov. Oceno bomo pomnožili z 1,5, da upoštevamo še vozlišča višje v hierarhiji in prostor, ki ga zavzame kazalo nad poljem najemnika. Tako dobimo oceno 192 MB za hrambo kazala. Posamezna kvantizacija pokrči originalni vektor (768×32 bitov) za 4- do 16-krat, zato bo skupna poraba prostora kvantiziranih vektorjev 750 MB za skalarno kvantizacijo, 375 MB za rotacijsko kvantizacijo na 4 bite in 187 MB za rotacijsko kvantizacijo na 2 bita, medtem ko originalni vektorji porabijo 3 GB prostora. Za teste dodelimo tolikšno količino pomnilnika, da bodo posamezne kvantizacije na meji. Opazimo, da vozlišče potrebuje vsaj 1,2 GB pomnilnika, da se zažene. Omejitve pomnilnika bomo določili na 2,2 GB, 1,7 GB, 1,4 GB in 1,2 GB [13].

Nastavitve hrambe struktur bomo ohranili take, kot so bile do sedaj – kazala nad tovorom in kvantizirani vektorji bodo `pinned`, torej vedno hranjeni v pomnilniku, medtem ko bodo druge strukture, če je mogoče, predpomnjene.

7.6.1 Rezultati

Opazimo, da je prepustnost ob nespremenjeni količini pomnilnika sorazmerna s procesorsko močjo, ki je na voljo. Ob omejitvi na 0,5 procesorja je upad

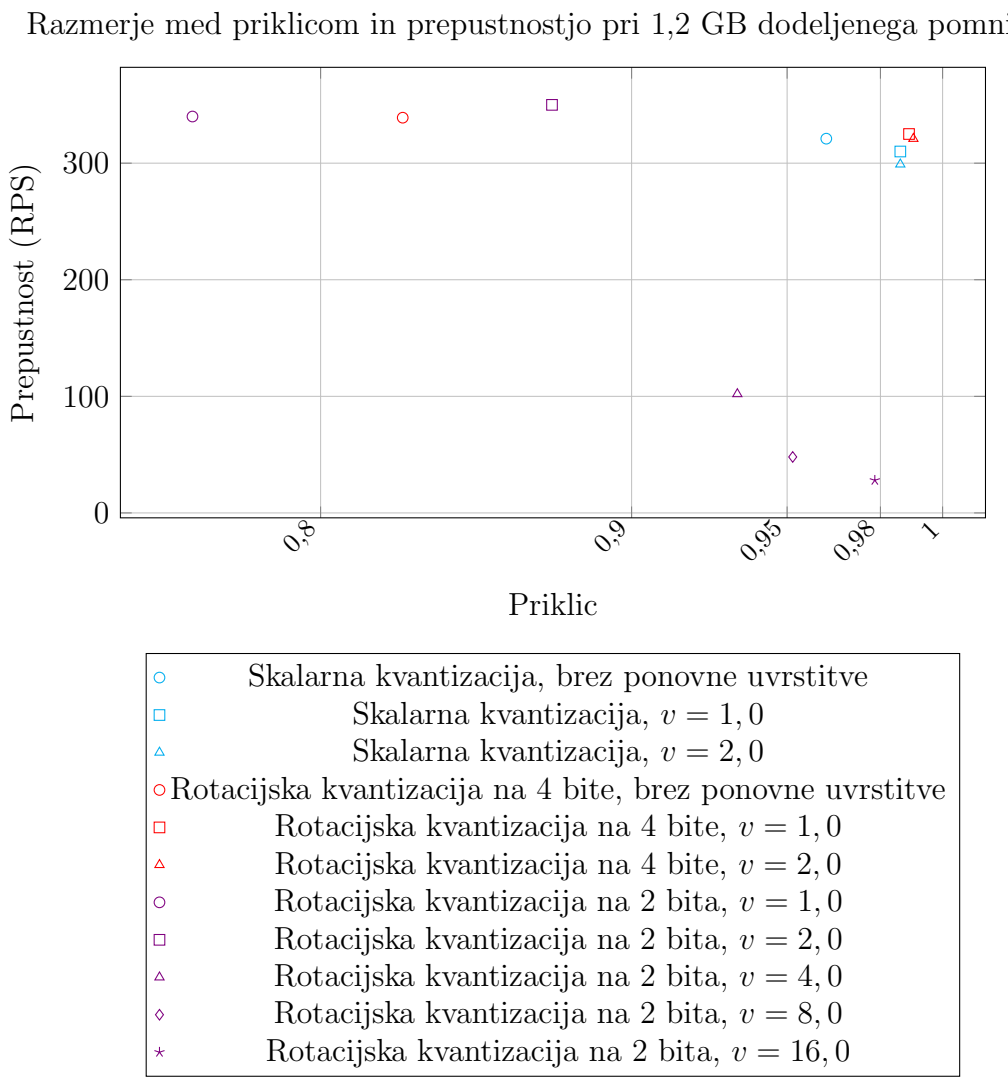
prepustnosti rahlo večji od zmanjšanja procesorske moči. Ob povečanju procesorske moči se prepustnost sprva skoraj sorazmerno poveča, vendar se ob dodajanju dodatnih procesorjev ne izboljša več bistveno, medtem ko izkoristek procesorske moči občutno pade.

Ob zmanjševanju razpoložljivega pomnilnika se prepustnost ne spreminja sorazmerno z zmanjševanjem pomnilnika. Opazimo, da se prepustnost spremeni, ko se pri poizvedbi uporablja višji faktor vzorčenja, medtem ko širina poizvedbe in tip kvantizacije ne vplivata na rezultat. Pri nobeni izmed treh vrst kvantizacije ni prišlo do zmanjšanja prepustnosti pri faktorju vzorčenja $\leq 2{,}0$.

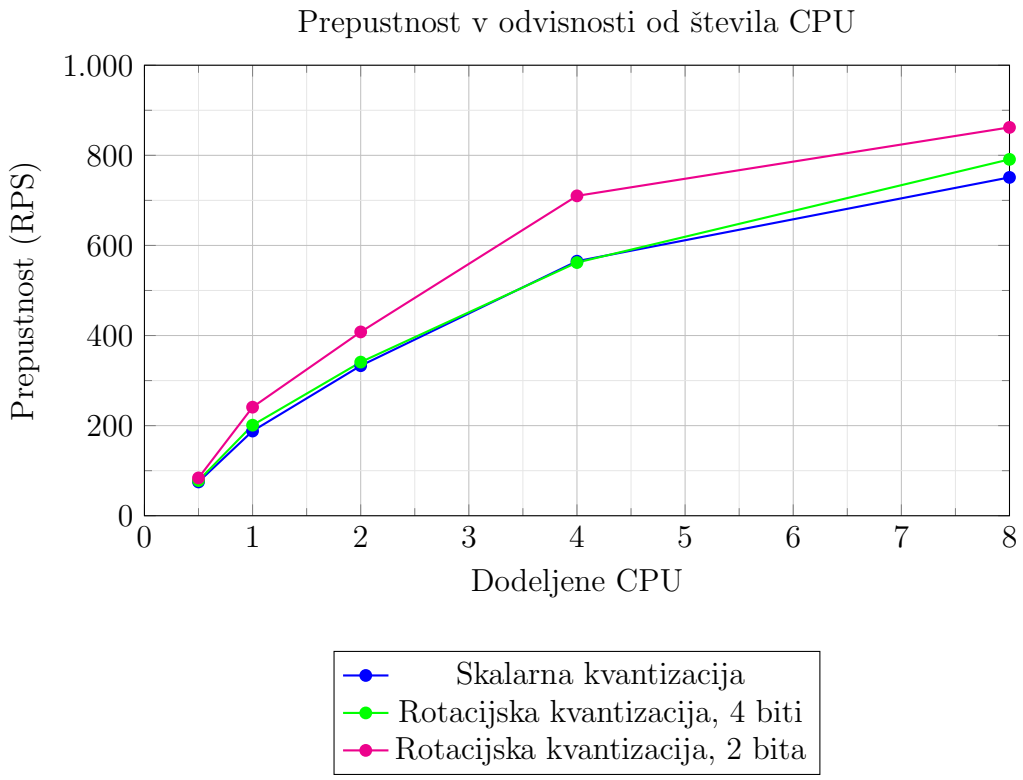
Ob primerjavi latence poizvedb z grafom ali brez grafa pri 1,2 GB pomnilnika vidimo, da struktura grafa predstavlja večji del podatkov, hranjenih v pomnilniku. Pojavi se vprašanje, ali se sploh splača uporabljati kvantizacijo na 2 ali manj bitov.

Če še dodatno zmanjšamo razpoložljivi pomnilnik na 1,0 GB, se strok ne bo pognal, ker nima na voljo dovolj pomnilnika za zagon. Tudi ko smo prišli do točke, ko se strok komaj poganja, nobena izmed uporabljenih kvantizacij ni izgubila prepustnosti pri navadnem vzorčenju.

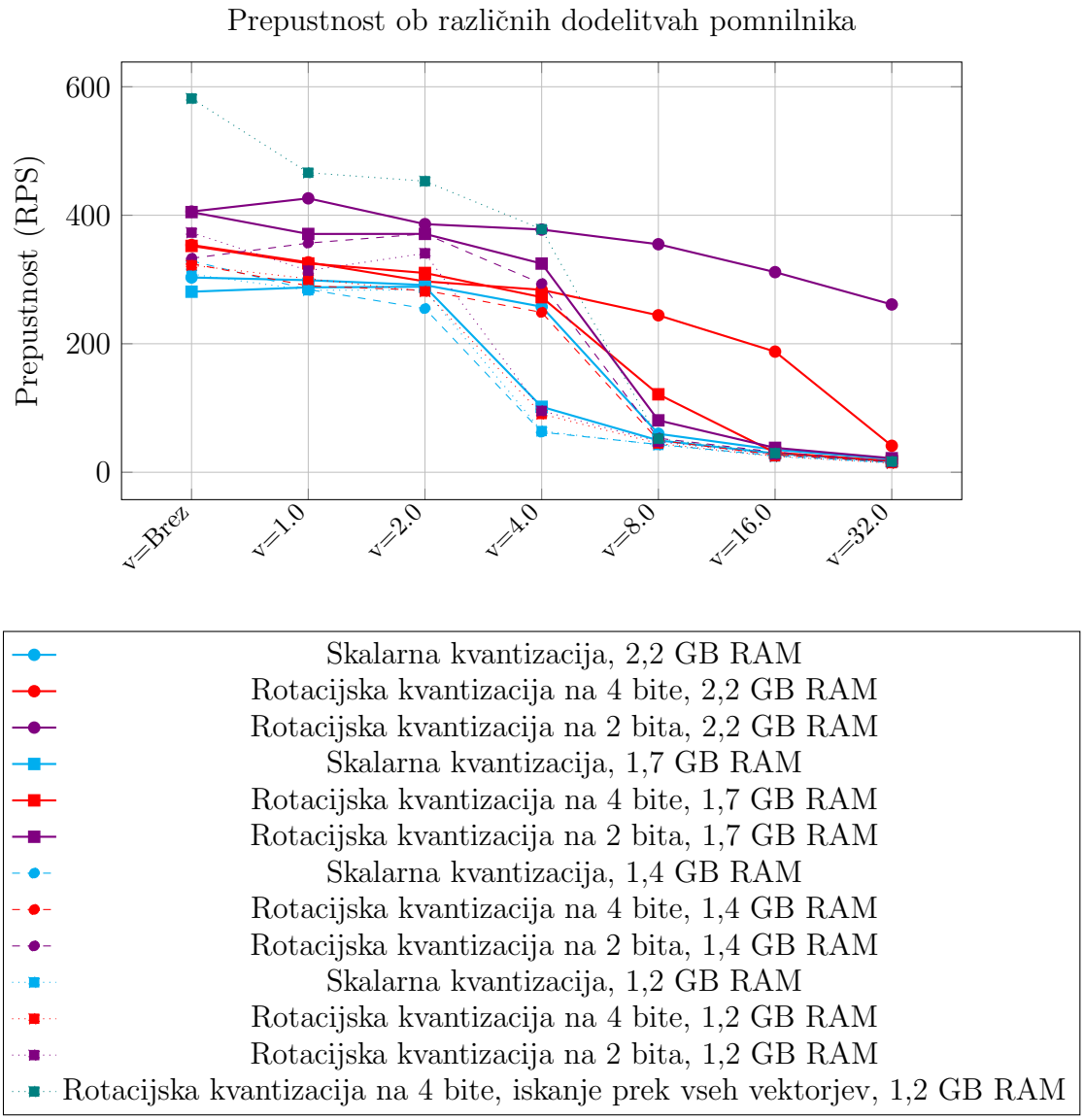
Na postavitvi z najmanjšo količino pomnilnika smo dodatno testirali RQ na 4 bite ob poizvedbi prek vseh najemnikovih vektorjev in pri uporabi globalnega grafa namesto ločenih podgrafov. V obeh primerih se je prepustnost izboljšala, ker so zbirke majhne in je nabor vektorjev majhen. Opazili smo, da se je pri povečanju faktorja vzorčenja prepustnost začela zmanjševati kasneje kot pri rotacijski kvantizaciji na 2 in 4 bite, iz česar lahko sklepamo, da graf HNSW porabi znaten del pomnilnika pri kompaktnih kvantizacijah.



Slika 7.8: Primerjava razmerja med prepustnostjo in priklicem več načinov kvantizacije in faktorjev vzorčenja pri 1,2 GB dodeljenega pomnilnika.



Slika 7.9: Prepustnost v odvisnosti od števila CPU. Faktor vzorčenja $v = 2,0$.



Slika 7.10: Primerjava prepustnosti ob različni porabi pomnilnika in različnih faktorjih vzorčenja.

Poglavje 8

Analiza rezultatov in priporočila

Na podlagi rezultatov testiranja lahko precej natančno opišemo predvideno uporabo infrastrukturnih virov vektorske podatkovne baze. Z zmanjševanjem razpoložljive procesorske moči in pomnilnika smo ugotovili, kako posamezen vir vpliva na delovanje VDB. S testiranjem različnih nastavitev gruče smo ugotovili, kako število drobcev in replikacijski faktor vplivata na poizvedbe. Pridobili smo številne podatke o zmogljivosti in priklicu restriktivnih poizvedb pri različnih nastavitvah kvantizacije in faktorja vzorčenja. Na podlagi teh rezultatov lahko postavimo nekaj vodil glede postavitve in nastavitve Qdrant VDB.

8.1 Postavitev gruče

Ob velikem številu najemnikov bo glavni vir, ki ga bomo uporabljali, procesorska moč. Ob rabi primernega postopka kvantizacije lahko pri zelo majhni porabi pomnilnika izvedemo veliko število poizvedb, ne da bi morali segati po podatkih na disku. Ob rabi dveh procesnih enot, 2 GB pomnilnika in 16 GB prostora na disku (kar je dovolj za hranjenje vseh vektorjev in struktur, uporabljenih med testiranjem), bi bili skupni mesečni stroški VDB 15,8

EUR ob upoštevanju navedenega cenika. Procesorska moč bi predstavljala 84,9 %, pomnilnik 12,7 % in prostor na disku 2,4 % stroškov glede na cene iz tabele 3.1. Pri testiranju so bili vsi uporabniki aktivni, v realnih scenarijih pa bi pričakovali, da bo le majhen odstotek uporabnikov istočasno uporabljal storitev, zato lahko pričakujemo, da bodo stroški trajne hrambe (torej najema diskov) večji, kot bi bili v scenariju testiranja. Ker je nabor testnih podatkov majhen, nismo imeli priložnosti testirati, kako se obnašajo podgrafi HNSW, ko imamo na razpolago majhno količino pomnilnika. Ob pogoju, da se posamezni podgrafi ločeno nalagajo v pomnilnik, bi pri isti prepustnosti poraba pomnilnika ostala nespremenjena. Ob majhnem razmerju aktivnih uporabnikov bo hramba uporabniških podatkov na disku predstavljala največji strošek.

Če poznamo predvideno razmerje aktivnih uporabnikov d_a, skupno število uporabnikov U, ceno procesne enote C_{CPU}, ceno pomnilnika C_{RAM} in ceno diska C_{DISK}, je predviden strošek $c = \alpha d_a U C_{CPU} + \beta d_a U C_{RAM} + \gamma U C_{DISK}$. Razmerje $k = \frac{\gamma}{\beta}$ je stopnja kvantizacije; pri skalarni kvantizaciji je malo manjša od 4. Ker se stopnja kvantizacije ne bo spreminjala, lahko trdimo, da $\beta \propto \gamma$. Na razmerje vplivajo vrsta kvantizacije, poraba pomnilnika grafa HNSW in število originalnih vektorjev, ki se hkrati naložijo v pomnilnik za potrebe ponovne uvrstitve. i_u naj bo povprečna količina podatkov, ki pripada vsakemu uporabniku; vrednost ustreza vrednosti γ. Predvidene stroške lahko ocenimo približno kot:

$$c = \alpha d_a U C_{CPU} + k i_u d_a U C_{RAM} + i_u U C_{DISK} \tag{8.1}$$

Na stroške vodenja gruče v glavnem vplivajo tri spremenljivke: α (faktor rabe CPU, na katerega vpliva število in pogostost poizvedb vsakega uporabnika), d_a (delež aktivnih uporabnikov) in i_u.

Na α vplivajo nastavitve zbirke in frekvenca poizvedb v bazo. Ob majhni vrednosti α ne potrebujemo velike prepustnosti, še vedno pa zahtevamo nizko latenco in hrambo vloženih podatkov, kar pomeni, da bo večji del stroškov namenjen hrambi podatkov v pomnilniku in na disku. Primer takega sis-

tema bi bila baza, namenjena brskanju fizičnega uporabnika po vloženih datotekah. Ob velikem deležu aktivnih uporabnikov d_a predstavlja večji del stroškov izvajanje poizvedb in hramba podatkovnih struktur v pomnilniku, stroški hrambe na disku pa so zanemarljivi. Primer takega sistema je baza za trajno hrambo zgodovine pogovorov agenta. Ob majhnem deležu aktivnih uporabnikov d_a in veliki količini hranjenih podatkov i_u predstavlja največji strošek trajna hramba podatkov na disku. Primer takega sistema je arhiv podatkov.

Ob velikem številu uporabnikov je smiselno vsakemu uporabniku ali ločenim podskupinam dodeliti svoj drobec zbirke, saj bo tako hitrost poizvedb ostala nespremenjena tudi ob horizontalnem skaliranju Qdrant gruče. Mapiranje najemnika v drobec bi lahko hranili v shrambi parov ključ-vrednost, ki omogoča hitro in porazdeljeno iskanje, kot je npr. etcd. Vlogo dodeljevanja drobca uporabnikom lahko prevzame vmesnik VDB. Ker so vsa vozlišča enaka in imajo na razpolago enake vire, je najboljši pristop za zagotavljanje višje prepustnosti določeni skupini uporabnikov (naj bodo to administratorji ali uporabniki, ki plačujejo višjo naročnino) ta, da se ustvari ločena Qdrant gruča. Usmerjanje omrežnega prometa do prave gruče upravlja posrednik, izbiro pravilne gruče pa izvede vmesnik VDB na podlagi uporabnika.

Ob majhnih količinah podatkov je vertikalno skaliranje boljši pristop, saj se ne uporabljajo računski viri za koordinacijo med vozlišči znotraj gruče. Ko prekoračimo maksimalno prepustnost, se začne latenca poizvedb postopoma večati; za povečanje prepustnosti je dovolj, da vozliščem dodelimo več procesorske moči. Količino pomnilnika povečamo le, ko opazimo, da se je hitrost poizvedb bistveno zmanjšala, ne da bi beležili sorazmeren porast števila zahtevkov. Za horizontalno skaliranje se odločimo, ko se prepustnost ne povečuje več sorazmerno s procesorsko močjo. Ob prvi postavitvi se odločimo za več vozlišč le, če pričakujemo veliko količino prometa že od začetka ali če zahtevamo, da je faktor replikacije podatkov večji od 1. Predhodno smo se odločili za gručo treh vozlišč. Taka izbira bi bila utemeljena, če želimo že od začetka uporabljati Qdrant gručo, da pridobimo izkušnje. Če postavimo več okolij,

na primer razvojno in produkcijsko, je utemeljeno tudi na razvojnem okolju uporabljati gručo, da je struktura sistema čim bolj podobna produkcijski. Na razvojnem okolju bi skalirali vozlišča horizontalno v primeru, da jih je na produkcijskem okolju res veliko; vozlišča lahko vertikalno skaliramo do zelo majhnih količine virov. Izbira replikacije podatkov s faktorjem r v zbirki bo zahtevala r-kratno povečanje količine pomnilnika na vsakem vozlišču, saj bo vsako vozlišče med poizvedbami hranilo r-kratno količino podatkov, da se pomanjša rabo pomnilnika se lahko zmanjša faktor replikacije in bolj pogosto ustvarja varnostne kopije podatkov.

8.2 Nastavitve zbirke

Če odmislimo arhitekturo gruče Qdrant in se osredotočimo na nastavitve zbirke, ki deluje na enem samem vozlišču, opazimo, da so nekatere nastavitve ključne. Ne glede na to, ali bomo poizvedbe filtrirali ali ne, se vedno splača uporabiti kvantizacijo. Vse vrste kvantizacije nimajo enakega učinka: PQ ne pohitri poizvedb, temveč ohrani podobno latenco in prepustnost kot poizvedbe brez kvantizacije. Skalarna in binarna kvantizacija se lahko na določenih naborih podatkov izkažeta za neprimerni [58], kar potrjujejo tudi primeri BQ, ki smo jih izvedli med testiranjem. RQ se izkaže za najbolj zanesljiv postopek kvantizacije, hkrati pa pohitri poizvedovanje, saj je mogoče rezultate množenja kvantiziranih vektorjev hraniti v majhni tabeli v pomnilniku (pri 4-bitni kvantizaciji je možnih kombinacij faktorjev le $16^2 = 256$), zato se med poizvedbami bistveno zmanjša uporaba enote za računanje s plavajočo vejico. Tudi na podatkih, ki smo jih uporabljali, ki vsebujejo vektorje iz množice $[0, 1]^{768}$, priklic ni drastično upadel. Preden uporabimo agresivne oblike kvantizacije, ki vsako komponento vektorja posplošijo na dva bita ali manj, se splača preveriti, ali bomo sploh dosegli pomenljivo izboljšavo pri rabi pomnilnika oziroma hitrosti poizvedb. Ker pri takih kompresijah graf HNSW porabi več prostora kot vektorji sami, se zdi raba takih kvantizacij smiselna le, ko imajo vektorji res veliko število dimenzij (recimo 2048

ali 3072); v takih primerih vektorji še vedno porabijo znatno količino prostora, pri manj razsežnih vektorjih pa postane prostor, potreben za hrambo vektorja, minimalen.

Porabo pomnilnika pri posameznem postopku kvantizacije najbolj dosledno preverimo tako, da sproti zmanjšujemo količino razpoložljivega pomnilnika in ponavljamo poizvedbe. V trenutku, ko ne bo več dovolj pomnilnika na razpolago, bomo opazili znaten upad prepustnosti. Kot smo opazili, lahko pri večjih, kompaktnejših kvantizacijah porabimo več pomnilnika, če uporabimo večji faktor vzorčenja. Porabo pomnilnika posameznega postopka kvantizacije je smiselno analizirati na večjih naborih podatkov, saj smo pri milijonu vektorjev uporabili le toliko pomnilnika, da smo lahko sploh pognali vozlišče; pri večjem naboru vektorjev bi lahko natančneje določili, kolikšna je kritična količina pomnilnika za vsak postopek.

Pri iskanju prek vseh vektorjev smo opazili, da je RQ ohranila dovolj informacij o vektorjih, da so bile poizvedbe natančne tudi brez ponovne uvrstitve rezultatov; pri ponovni uvrstitvi rezultatov brez dodatnega vzorčenja pa smo dosegli skoraj popoln priklic. Iz tega lahko sklepamo, da 4-bitna RQ ohrani dobršen del informacij izvirnega vektorja. Isti postopek bi lahko ponovili na večjih naborih vektorjev in preverili, ali se priklic ohrani.

Če nameravamo izvajati filtrirane poizvedbe, je najpomembnejša nastavitev definicija kazala nad metapodatki. Če poznamo pričakovan vzorec dostopa do baze, lahko polje najemnika definiramo tako, da hranimo vektorje, ki bodo hkrati dostopani, na sosednjih blokih pomnilnika. Pohitritev, ki jo dosežemo, ko poizvedujemo po polju najemnika, je tolikšna, da se splača v polje vstavljati sestavljene ključe: to pomeni, da če določeni podatki pripadajo uporabniku, drugi pa nekemu deljenemu delovnemu okolju, ne bomo hranili identifikatorjev v dveh ločenih poljih, recimo `id_uporabnika` in `id_delovnega_okolja`, temveč bomo uporabljali eno samo polje `najemnik`, kamor bomo zapisali vrednosti ključa v obliki `uporabnik:{id}` oziroma `delovno_okolje:{id}`. Identifikatorju dodamo naziv predmeta, ki ga identificira, da ne pride do uporabe podvojenih ključev (v primeru, ko se za

identifikatorje uporablja sekvenca števil).

Če ne bomo nikoli dostopali do vseh podatkov hkrati, se splača izklopiti ustvarjanje globalnega grafa in ustvariti grafe, ki združujejo polja istega najemnika. Ustvarjanje podgrafov HNSW za vsako indeksirano polje ni smiselno, saj lahko v primeru, ko vrednost polja ne bo bistveno skrčila rezultatov iskanja, uporabimo že definiran podgraf; če pa je polje zelo omejujoče, se bo splačalo primerjati vse vektorje namesto poizvedbe prek grafa. Raba podgrafov za več različnih polj je primerna le, kadar izvajamo poizvedbe po globalnih grafih prek več različnih filtrov, ki niso obvezni. Iskanje po arhivu znanstvenih člankov je primer takšne poizvedbe, filtriramo lahko po avtorju, letu izdaje, vrsti članka ipd.

Če vemo, da bo poizvedb po majhnih naborih točk veliko, se splača testirati, kolikšna je dopustna meja za primerjavo vseh vektorjev. V našem primeru se je izkazalo, da bi privzeto mejo lahko zvišali.

Ker je bil nabor vektorjev majhen, nismo utegnili ovrednotiti vpliva širine poizvedbe ef na prepustnost. Vpliv tega parametra na iskanje bi morali testirati pri bistveno večjih grafih HNSW.

Nastavitve zbirke je smiselno testirati na zelo velikih naborih podatkov (kopijah produkcijskih podatkov ali sintetičnih naborih), saj se tudi pri zmerno velikih količinah vektorjev (milijon v našem primeru) ob uporabi filtriranja ne da zlahka analizirati zmogljivosti in uporabe virov baze pri različnih nastavitvah.

8.3 Raba vmesnika za dostop do VDB

Vmesnik VDB nam omogoča, da učinkoviteje razvijamo uporabo in dostop do VDB ter učinkoviteje upravljamo navigacijo po omrežju. Drugim storitvam izpostavimo le poslovno logiko, vezano na hrambo vektorjev, medtem ko predelavo podatkov v vektorje, omejevanje dostopa, izbiro gruče Qdrant in delca zbirke prepustimo eni storitvi, ki se razvija prek ene same kode.

Nadalje bi lahko testirali prepustnost VDB ob pogoju, da bi se prek

vmesnika in posrednika Envoy povezali do treh samostojnih vozlišč namesto do gruče treh vozlišč, vmesnik pa bi izbral pravilno vozlišče za poizvedbo.

8.4 Omejitve in grožnje veljavnosti

Ker ne obstaja veliko naborov podatkov, ki analizirajo zmogljivost filtriranega iskanja, smo uporabljali že obstoječe merilo `random-768-100-tenants` z minimalnimi prilagoditvami. Število točk v grafu je premajhno, da bi lahko pravilno ocenili zmogljivost vsake nastavitve. Da bi pravilno ocenili zmogljivost različnih pristopov kvantizacije pri HNSW-podgrafih, bi morali imeti na razpolago bistveno večji nabor vektorjev za 100 najemnikov. Celoten kvantiziran graf je porabil približno 400 MB prostora pri RQ na 4 bite, graf posameznega uporabnika pa okoli 4 MB pomnilnika. Upoštevati je treba, da se je sočasno izvajalo le minimalno število poizvedb (tudi če smo vzporedno pognali 64 poizvedb, vozlišče ni obravnavalo vseh istočasno). Da bi dosegli večjo porabo pomnilnika, bi morali ohraniti isto število najemnikov, vendar vsaj desetkrat povečati število pripadajočih vektorjev.

Vrednosti vektorjev so bile porazdeljene na intervalu $[0, 1]$. Za komponente vektorja bi bilo bolj primerno uporabljati porazdelitev $\mathcal{N}(0, 1)$ ali vsaj $\mathcal{U}(-1, 1)$, da so podatki bolj enakomerno razpršeni po kvadrantih. Ker so si vektorji podobni, je mogoče, da je pri večji kompresiji prišlo do večjih napak, kot če bi uporabljali vektorje s komponentami na intervalu $[-1, 1]$; predvsem pri kvantizacijah na 2 bita je ta lastnost vektorjev zmanjšala natančnost postopka kvantizacije.

Testnih primerov je malo, najemniki pa se stalno izmenjujejo. Primerno bi bilo sestaviti dva scenarija poizvedb: prvega, kjer se poizvedbe izvajajo s strani naključnih najemnikov kot doslej, in drugega, kjer le nekaj najemnikov hkrati izvaja poizvedbe, da je scenarij bolj podoben uporabi v realnih primerih. Skupno število med seboj različnih poizvedb bi moralo biti vsaj 5000 tudi pri visoki prepustnosti, da bi lahko testirali sočasno rabo najemnikov in zmanjšali napako meritev. Uporabljali smo 200 originalnih testnih pri-

merov in jih 10-krat ponovili znotraj enega zagona, kar ne izboljša meritve priklica, temveč le uravnoteži meritve latence in prepustnosti. Poleg tega smo opravili le en testni zagon za vsako kombinacijo nastavitev poizvedbe in zbirke, zato imamo izmerjen le standardni odklon med poizvedbami znotraj istega zagona, ne pa med posameznimi zagoni, zato ne poznamo variance med ponovitvami istega testa.

Dodatno bi lahko pripravili še nabor podatkov, kjer ima vsak najemnik različno število točk, da analiziramo primer, ko ne moremo predpostaviti, kolikšno količino vektorjev ima posamezen najemnik.

Pri večjem naboru podatkov bi lahko podoben pristop testiranja aplicirali tudi na podatkovni bazi Weaviate in Milvus, da primerjamo njuno zmogljivost pri isti količini virov.

Testi so bili pognani na lokalnem računalniku, na 'kind' Kubernetes gruči; poraba virov za izvrševanje drugih procesov bi lahko vplivala na rezultate meritev. Ob upoštevanju dejstva, da razpolaga računalnik z 64 GB RAM in 18 CPU procesorske enote Intel®Core™Ultra 5 125H [43], je vpliv porabe virov drugih procesov na računalniku na teste, ki so uporabljali 2 CPU, zanemarljiv. Vse procesne enote na uporabljenem procesorju niso enakovredne, zato obstaja tveganje, da so se ob dodajanju več procesorske moči uporabljale manj zmogljive procesorske enote. Prek uporabe samostojne Kubernetes gruče v oblaku bi dosegli bolj dosledne rezultate.

Poglavje 9

Zaključek

Predstavljena arhitektura drugim storitvam znotraj aplikacije omogoča enostaven vnos podatkov v VDB, ki abstrahira logiko upravljanja z VDB. Enostaven REST API omogoča, da lahko razvijalci brez znanja o VDB implementirajo shranjevanje podatkov v VDB. Tak pristop dolgoročno nudi možnost, da migriramo med ponudniki VDB z zgolj spremembo ene same storitve. Hkrati vmesnik poenostavi razvoj agentov in njihov dostop do VDB, saj prek MCP strežnika agentu izpostavlja dolgoročni spomin, ki ga agent sam ne more hraniti v svojem delovnem spominu, ter mu nudi možnost iskanja podatkov po semantični podobnosti.

Prek empiričnih poskusov smo izmerili zmogljivost različnih konfiguracij VDB in našli nabor konfiguracij, ki najbolje ustreza našemu problemu. Iskanje prek kazala nad poljem najemnika s podgrafom HNSW izboljša prepustnost poizvedb za do 50 % v primerjavi z navadnim kazalom s podgrafom HNSW, zato je pomembno uporabljati najemniško polje za najpogosteje uporabljen filter. Ustvarjanje r replik približno r-krat poveča porabo pomnilnika, saj vsa vozlišča skušajo za poizvedbe uporabljati kopijo, ki jo imajo lokalno na voljo. Če lahko izbiramo med horizontalnim in vertikalnim skaliranjem gruče, ima pri manjših količinah virov prednost vertikalno skaliranje, saj samostojno vozlišče pri isti dodeljeni količini virov doseže do 40 % večjo prepustnost v primerjavi z gručo s tremi vozlišči. Izkoristek virov pri različnem

številu vozlišč bi bilo treba dodatno raziskati pri bistveno večjem naboru virov. Testi so potrdili, da kvantizacije, ki uporabljajo manjše slovarje, kot sta skalarna in rotacijska kvantizacija, pri enaki kompresiji dosegajo večjo prepustnost kot kvantizacija produkta, ki uporablja velike zbirke seznamov centroidov. Pokazali smo, da lahko privzeto mejo za iskanje prek vseh vektorjev v Qdrant bistveno povečamo. Med testiranjem smo uporabljali iskanje prek vseh vektorjev na 4-krat večjem številu vektorjev, kot je privzeto; dodatno testiranje lahko pomaga natančneje določiti mejo, do katere lahko uporabimo tak način iskanja. Pri najemniških podgrafih lahko taka sprememba izboljša priklic, saj lahko pričakujemo, da bo vsakemu najemniku pripadalo malo podatkov. Ob premajhni dodeljeni količini pomnilnika lahko prepustnost pade na manj kot 75 % preustnosti dosežene pri manjšem faktorju vzorčenja; med testiranjem smo prag rabe pomnilnika presegli le pri ponovni uvrstitvi rezultatov s štirikratnim faktorjem vzorčenja ali več. Rezultat kaže, da čeprav lahko bolj kompaktni načini kvantizacije izboljšajo rabo pomnilnika pri istih nastavitvah poizvedbe, izgubimo prednosti kompaktnejše kvantizacije, ko poskušamo izboljšati priklic prek nadvzorčenja. Znanje, pridobljeno iz testiranja konfiguracij, smo uporabili za formulacijo še dodatnih dobrih praks, ki jih lahko apliciramo tudi na druge primere uporabe Qdrant VDB.

9.1 Nadaljnje delo

Na področju filtriranega testiranja VDB je malo naborov podatkov, zato bi bilo smiselno pripraviti večje in bolj natančne nabore podatkov, namenjene testiranju hrambe več najemniških podatkov in vzorcev dostopa več najemnikov hkrati. V večnajemniških aplikacijah bo hkrati aktiven le del najemnikov, zato je smiselno poizvedbe izvajati le za del najemnikov. Testiranje zmogljivosti pri majhni količini dodeljenega pomnilnika je smiselno izvesti tudi pri večjem naboru podatkov, saj bodo omejitve posameznega načina kvantizacije hitreje prišle do izraza. Agentov, ki hranijo spomin pogovora oziroma delovanja v VDB namesto znotraj delovnega spomina, nismo razvili, kar je

dobro izhodišče za nadaljnji razvoj sposobnosti agentov.

Literatura

[1] Envoy proxy. https://www.envoyproxy.io/. Dostopano: 3. 9. 2026.

[2] StatefulSets - Kubernetes documentation. Dosegljivo: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/. [Dostopano: 19. 8. 2026].

[3] kind: Kubernetes in docker. https://kind.sigs.k8s.io/. Dostopano: 23. 8. 2026.

[4] Architecture overview - Milvus documentation. Dosegljivo: https://milvus.io/docs/architecture_overview.md, . Verzija v3.0.x [Dostopano: 19. 8. 2026].

[5] Bitmap index - Milvus documentation. Dosegljivo: https://milvus.io/docs/bitmap.md, . Verzija v3.0.x [Dostopano: 19. 8. 2026].

[6] Clustering compaction - Milvus documentation. Dosegljivo: https://milvus.io/docs/clustering-compaction.md, . Verzija v3.0.x [Dostopano: 19. 8. 2026].

[7] Data processing - Milvus documentation. Dosegljivo: https://milvus.io/docs/data_processing.md, . Verzija v3.0.x [Dostopano: 22. 8. 2026].

[8] Index explained - Milvus documentation. Dosegljivo: https://milvus.io/docs/index-explained.md, . Verzija v3.0.x [Dostopano: 19. 8. 2026].

[9] Use partition key - Milvus documentation. Dosegljivo: https://milvus.io/docs/use-partition-key.md, . Verzija v3.0.x [Dostopano: 19. 8. 2026].

[10] Time synchronization - Milvus documentation. Dosegljivo: https://milvus.io/docs/time_sync.md, . Verzija v3.0.x [Dostopano: 19. 8. 2026].

[11] Timestamp - Milvus documentation. Dosegljivo: https://milvus.io/docs/timestamp.md, . Verzija v3.0.x [Dostopano: 19. 8. 2026].

[12] Oracle cloud infrastructure pricing. Dosegljivo: https://www.oracle.com/cloud/price-list. [Dostopano: 18. 8. 2026].

[13] Capacity planning - Qdrant documentation. Dosegljivo: https://qdrant.tech/documentation/capacity-planning, . [Dostopano: 20. 8. 2026].

[14] Distributed deployment - Qdrant documentation. Dosegljivo: https://qdrant.tech/documentation/distributed_deployment/, . [Dostopano: 19. 8. 2026].

[15] Indexing - Qdrant documentation. Dosegljivo: https://qdrant.tech/documentation/manage-data/indexing/, . [Dostopano: 3. 9. 2026].

[16] Qdrant - memory tiers. Dosegljivo: https://qdrant.tech/documentation/ops-configuration/memory-tiers/, . [Dostopano: 2. 9. 2026].

[17] Multitenancy - Qdrant documentation. Dosegljivo: https://qdrant.tech/articles/multitenancy/, . [Dostopano: 19. 8. 2026].

[18] Quantization - Qdrant documentation. Dosegljivo: https://qdrant.tech/documentation/manage-data/quantization/, . [Dostopano: 19. 8. 2026].

[19] Scroll points. Dosegljivo: https://api.qdrant.tech/api-reference/points/scroll-points, . [Dostopano: 23. 8. 2026].

[20] Qdrant - vectors. Dosegljivo: https://qdrant.tech/documentation/manage-data/vectors/, . [Dostopano: 2. 9. 2026].

[21] Cluster architecture - Weaviate documentation. Dosegljivo: https://docs.weaviate.io/weaviate/concepts/replication-architecture/cluster-architecture, . [Dostopano: 19. 8. 2026].

[22] Reranking - Weaviate documentation. Dosegljivo: https://docs.weaviate.io/weaviate/concepts/reranking, . [Dostopano: 19. 8. 2026].

[23] JSON-RPC 2.0 specification. Dosegljivo: https://www.jsonrpc.org/specification, 2010. [Dostopano: 22. 8. 2026].

[24] Hybrid search - Qdrant documentation. Dosegljivo: https://qdrant.tech/articles/hybrid-search/, 2024. [Dostopano: 19. 8. 2026].

[25] Abylay Amanbayev, Brian Tsan, Tri Dang, and Florin Rusu. Filtered approximate nearest neighbor search in vector databases: System design and performance analysis, 2026. URL https://arxiv.org/abs/2602.11443.

[26] Anthropic. Model context protocol specification (2025-11-25). Dosegljivo: https://modelcontextprotocol.io/specification/2025-11-25, 2025. [Dostopano: 19. 8. 2026].

[27] Eric Brewer. CAP twelve years later: How the "rules" have changed. *Computer*, 45(2):23–29, 2012. doi: 10.1109/MC.2012.37.

[28] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen

Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS '20, Red Hook, NY, USA, 2020. Curran Associates Inc.

[29] Hongliu Cao. Recent advances in text embedding: A comprehensive review of top-performing methods on the mteb benchmark, 2024. URL `https://arxiv.org/abs/2406.01607`.

[30] Tobias Christiani. 8-bit rotational quantization: How to improve the speed-quality tradeoff of vector search. Dosegljivo: `https://weaviate.io/blog/8-bit-rotational-quantization`, 2025. [Dostopano: 18. 8. 2026].

[31] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025. URL `https://arxiv.org/abs/2503.18813`.

[32] Yihao Ding, Siwen Luo, Yue Dai, Yanbei Jiang, Zechuan Li, Qiang Sun, Geoffrey Martin, Wei Liu, and Yifan Peng. A survey on MLLM-based visually rich document understanding: Methods, challenges, and emerging trends. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, *Findings of the Association for Computational Linguistics: ACL 2026*, pages 13319–13340, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.652. URL `https://aclanthology.org/2026.findings-acl.652/`.

[33] Kenneth Enevoldsen, Isaac Chung, Imene Kerboua, Márton Kardos, Ashwin Mathur, David Stap, Jay Gala, Wissam Siblini, Dominik

Krzemiński, Genta Winata, Saba Sturua, Saiteja Utpala, Mathieu Ciancone, Marion Schaeffer, Diganta Misra, Shreeya Dhakal, Jonathan Rystrø m, Roman Solomatin, Ömer Çağatan, Akash Kundu, Martin Bernstorff, Shitao Xiao, Akshita Sukhlecha, Bhavish Pahwa, RafałPoświata, Kranthi Kiran GV, Shawon Ashraf, Daniel Auras, Björn Plüster, Jan Harries, Loïc Magne, Isabelle Mohr, Dawei Zhu, Hippolyte Gisserot-Boukhlef, Tom Aarsen, Jan Kostkan, Konrad Wojtasik, Taemin Lee, Marek Suppa, Crystina Zhang, Roberta Rocca, Mohammed Hamdy, Andrianos Michail, John Yang, Manuel Faysse, Aleksei Vatolin, Nandan Thakur, Manan Dey, Dipam Vasani, Pranjal Chitale, Simone Tedeschi, Nguyen Tai, Artem Snegirev, Mariya Hendriksen, Michael Günther, Mengzhou Xia, Weijia Shi, Xing Han Lu, Jordan Clive, Gayatri K, Maksimova Anna, Silvan Wehrli, Maria Tikhonova, Henil Panchal, Aleksandr Abramov, Malte Ostendorff, Zheng Liu, Simon Clematide, Lester James V. Miranda, Alena Fenogenova, Guangyu Song, Ruqiya Bin Safi, Wen-Ding Li, Alessia Borghini, Federico Cassano, Lasse Hansen, Sara Hooker, Chenghao Xiao, Vaibhav Adlakha, Orion Weller, Siva Reddy, and Niklas Muennighoff. Mmteb: Massive multilingual text embedding benchmark. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, volume 2025, pages 101715–101771, 2025. URL `https://proceedings.iclr.cc/paper_files/paper/2025/file/fc0e3f908a2116ba529ad0a1530a3675-Paper-Conference.pdf`.

[34] Evropski parlament in Svet Evropske Unije. Direktiva 2002/58/ES evropskega parlamenta in Sveta z dne 12. julija 2002 o obdelavi osebnih podatkov in varstvu zasebnosti na področju elektronskih komunikacij (Direktiva o zasebnosti in elektronskih komunikacijah), 2002.

[35] Evropski parlament in Svet Evropske Unije. Uredba (EU) 2016/679 Evropskega parlamenta in Sveta z dne 27. aprila 2016 o varstvu posameznikov pri obdelavi osebnih podatkov in o prostem pretoku takih

podatkov ter o razveljavitvi Direktive 95/46/ES (Splošna uredba o varstvu podatkov) (Besedilo velja za EGP), 2016.

[36] Stan Franklin and Arthur Graesser. Is it an agent, or just a program?: A taxonomy for autonomous agents. In *Intelligent Agents III Agent Theories, Architectures, and Languages (ATAL '96)*, volume 1193 of *Lecture Notes in Computer Science*, pages 21–35. Springer, 1996.

[37] Jerome Friedman. On bias, variance, 0/1—loss, and the curse-of-dimensionality. *Data Min. Knowl. Discov.*, 1:55–77, 03 1997. doi: 10.1023/A:1009778005914.

[38] Jianyang Gao and Cheng Long. Rabitq: Quantizing high-dimensional vectors with a theoretical error bound for approximate nearest neighbor search. *Proceedings of the ACM on Management of Data*, 2(3), May 2024. doi: 10.1145/3654970. URL `https://doi.org/10.1145/3654970`.

[39] Jianyang Gao, Yutong Gou, Yuexuan Xu, Jifan Shi, Yongyi Yang, Shuolin Li, Raymond Chi-Wing Wong, and Cheng Long. Revisiting rabitq and turboquant: A symmetric comparison of methods, theory, and experiments, 2026. URL `https://arxiv.org/abs/2604.19528`.

[40] Seth Gilbert and Nancy Lynch. Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services. *SIGACT News*, 33(2):51–59, June 2002. doi: 10.1145/564585.564601. URL `https://doi.org/10.1145/564585.564601`.

[41] Mohammed Mehedi Hasan, Hao Li, Emad Fallahzadeh, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers, 2026. URL `https://arxiv.org/abs/2506.13538`.

[42] Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei,

Shanglin Wu, Ruiyao Xu, Liangwei Yang, Rui Yang, Wooseong Yang, Chin-Yuan Yeh, Hanrong Zhang, Haozhen Zhang, Siqi Zhu, Henry Peng Zou, Wanjia Zhao, Song Wang, Wujiang Xu, Zixuan Ke, Zheng Hui, Dawei Li, Yaozu Wu, Langzhou He, Chen Wang, Xiongxiao Xu, Baixiang Huang, Juntao Tan, Shelby Heinecke, Huan Wang, Caiming Xiong, Ahmed A. Metwally, Jun Yan, Chen-Yu Lee, Hanqing Zeng, Yinglong Xia, Xiaokai Wei, Ali Payani, Yu Wang, Haitong Ma, Wenya Wang, Chenguang Wang, Yu Zhang, Xin Eric Wang, Yongfeng Zhang, Jiaxuan You, Hanghang Tong, Xiao Luo, Xue Liu, Yizhou Sun, Wei Wang, Julian McAuley, James Zou, Jiawei Han, Philip S. Yu, and Kai Shu. A survey of agent memory in the second half: Towards self-evolving and long-horizon agents, 2026. URL `https://arxiv.org/abs/2602.06052`.

[43] Intel Corporation. Intel Core Ultra 5 processor 125h specifications. Dosegljivo: `https://www.intel.com/content/www/us/en/products/sku/236848/intel-core-ultra-5-processor-125h-18m-cache-up-to-4-50-ghz/specifications.html`. [Dostopano: 23. 8. 2026].

[44] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. Diskann: Fast accurate billion-point nearest neighbor search on a single node. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL `https://proceedings.neurips.cc/paper_files/paper/2019/file/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Paper.pdf`.

[45] Herve Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011. doi: 10.1109/TPAMI.2010.57.

[46] Qiang Ke, Yanjie Zhao, Hongjin Leng, Shengming Zhao, and Haoyu Wang. Not all rags are created equal: A component-wise empirical

study for software engineering tasks. *Proc. ACM Softw. Eng.*, 3(FSE), June 2026. doi: 10.1145/3808190. URL https://doi.org/10.1145/3808190.

[47] Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, and Ali Farhadi. Matryoshka representation learning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22, Red Hook, NY, USA, 2022. Curran Associates Inc.

[48] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 02 2024. doi: 10.1162/tacl_a_00638. URL https://doi.org/10.1162/tacl_a_00638.

[49] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982. Originally as a Bell Telephone Laboratories internal memorandum (1957).

[50] Yu A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42 (4):824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.

[51] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB leaderboard. Dosegljivo: https://huggingface.co/spaces/mteb/leaderboard. [Dostopano: 18. 8. 2026].

[52] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference*, USENIX ATC'14, page 305–320, USA, 2014. USENIX Association.

[53] OpenFGA. What is fga? fine-grained authorization explained. https://openfga.dev/docs/fga. [Dostopano: 22. 8. 2026].

[54] Bojan Orel. *Linearna algebra*. Založba FE in FRI, 2012.

[55] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024. URL https://arxiv.org/abs/2310.08560.

[56] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proc. ACM Manag. Data*, 2(3), May 2024. doi: 10.1145/3654923. URL https://doi.org/10.1145/3654923.

[57] Beth Plale, Sai Navya Jyesta, and Sachith Withana. Vector embedding of multi-modal texts: a tool for discovery?, 2025. URL https://arxiv.org/abs/2509.08216.

[58] Ivan Pleshkov and Jonas Schulz. TurboQuant in Qdrant. Dosegljivo: https://qdrant.tech/articles/turboquant-quantization/, 2026. [Dostopano: 18. 8. 2026].

[59] Qdrant Team. Vector database benchmark. Dosegljivo: https://github.com/qdrant/vector-db-benchmark. [Dostopano: 19. 8. 2026].

[60] Ermu Qiu, Tianyi Chen, Jun Gao, Xing Wei, Yaofeng Tu, Yinjun Han, and Yang Lin. Boomhq: Learning to boost multiple hybrid queries on vector dbmss, 2026. URL https://arxiv.org/abs/2604.24552.

[61] Vyzantinos Repantis, Harshvardhan Singh, Tony Joseph, Cien Zhang, Akash Vishwakarma, Svetlana Karslioglu, Michael Wyatt Thot, and Ameya Gawde. The 99% Success Paradox: When Near-Perfect Retrieval Equals Random Selection, 2026. URL https://arxiv.org/abs/2605.18857.

[62] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 4:1–174, 09 2009. doi: 10.1561/1500000019.

[63] Supreeth Shastri, Melissa Wasserman, and Vijay Chidambaram. Gdpr anti-patterns. *Commun. ACM*, 64(2):59–65, January 2021. doi: 10.1145/3378061. URL https://doi.org/10.1145/3378061.

[64] Harsha Vardhan simhadri, Martin Aumüller, Matthijs Douze, Dmitry Baranchuk, Amir Ingber, Edo Liberty, George Williams, Ben Landrum, Magdalen Dobson Manohar, Mazin Karjikar, Laxman Dhulipala, Meng Chen, Yue Chen, Rui Ma, Kai Zhang, Yuzheng Cai, Jiayang Shi, Weiguo Zheng, Yizhuo Chen, Jie Yin, and Ben Huang. Results of the big ANN: NeurIPS'23 competition. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2026. URL https://openreview.net/forum?id=dB6W56wQL9.

[65] Aditi Singh, Abul Ehtesham, Saket Kumar, Tala Talaei Khoei, and Athanasios V. Vasilakos. Agentic retrieval-augmented generation: A survey on agentic rag, 2026. URL https://arxiv.org/abs/2501.09136.

[66] Harald Steck, Chaitanya Ekanadham, and Nathan Kallus. Is cosine-similarity of embeddings really about similarity? In *Companion Proceedings of the ACM Web Conference 2024*, WWW '24, page 887–890. ACM, May 2024. doi: 10.1145/3589335.3651526. URL http://dx.doi.org/10.1145/3589335.3651526.

[67] Nikhil Verma. Active context compression: Autonomous memory management in llm agents, 2026. URL https://arxiv.org/abs/2601.07190.

[68] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model

based autonomous agents. *Frontiers of Computer Science*, 18(6), March 2024. doi: 10.1007/s11704-024-40231-1. URL http://dx.doi.org/10.1007/s11704-024-40231-1.

[69] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *Eleventh International Conference on Learning Representations (ICLR)*, 2023. doi: 10.48550/arXiv.2210.03629. URL https://doi.org/10.48550/arXiv.2210.03629.

[70] Amir Zandieh, Majid Daliri, Majid Hadian, and Vahab Mirrokni. Turboquant: Online vector quantization with near-optimal distortion rate. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust, editors, *International Conference on Learning Representations*, volume 2026, pages 56418–56439, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/5c802ef38ab6e366c2ea06eee554c088-Paper-Conference.pdf.

[71] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL https://arxiv.org/abs/2506.05176.

[72] Shanshan Zhao, Xinjie Zhang, Jintao Guo, Jiakui Hu, Lunhao Duan, Minghao Fu, Yong Xien Chng, Guo-Hua Wang, Qing-Guo Chen, Zhao Xu, Weihua Luo, and Kaifu Zhang. Unified multimodal understanding and generation models: Advances, challenges, and opportunities, 2026. URL https://arxiv.org/abs/2505.02567.

Dodatek A

Konfiguracije in odseki kode

A.1 Končne točke vmesnika VDB

Odsek kode A.1: Del OpenAPI specifikacije vmesnika VDB

```
---
openapi: 3.1.0
components:
  schemas:
    BatchVnosDto:
      type: object
      required:
      - najemnik
      - vnosi
      properties:
        najemnik:
          $ref: "#/components/schemas/NajemnikDto"
        vrstaVira:
          type: string
        vnosi:
          type: array
          items:
            $ref: "#/components/schemas/VnosDto"
    IdentifikatorVektorjaObject:
      type: object
      properties:
        identifikator: {}
    NajemnikDto:
      type: object
      required:
      - vrstaNajemnika
```

```
27        - identifikator
28        properties:
29          vrstaNajemnika:
30            type: string
31          identifikator:
32            type: string
33      PoizvedbaDto:
34        type: object
35        required:
36        - poizvedbaBesedilo
37        - stRezultatov
38        - najemnik
39        properties:
40          poizvedbaBesedilo:
41            type: string
42          stRezultatov:
43            type: integer
44            format: int32
45          najemnik:
46            $ref: "#/components/schemas/NajemnikDto"
47          vrstaVira:
48            type: string
49          identifikatorVira:
50            type: string
51      RezultatPoizvedbeDto:
52        type: object
53        properties:
54          mesto:
55            type: integer
56            format: int32
57          razdalja:
58            type: number
59            format: float
60          tocka:
61            $ref: "#/components/schemas/TockaDto"
62      TockaDto:
63        type: object
64        properties:
65          identifikator:
66            $ref: "#/components/schemas/IdentifikatorVektorjaObject"
67          vektor:
68            type: array
69            items:
70              type: number
71              format: float
72          najemnik:
73            $ref: "#/components/schemas/NajemnikDto"
```

```
        vrstaVira:
          type: string
        identifikatorVira:
          type: string
        tovor:
          type: object
          description: Poljubni tovor v Json obliki.
    VnosDto:
      type: object
      required:
      - vsebinaZaVlozitev
      properties:
        vsebinaZaVlozitev:
          type: string
          minLength: 1
        identifikatorVira:
          type: string
        tovor:
          type: object
          description: Poljubni tovor v Json obliki.
  securitySchemes:
    SecurityScheme:
      type: openIdConnect
      description: Authentication
paths:
  /v1/javno/nalozi:
    post:
      tags:
      - Vmesnik-VDB
      requestBody:
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/BatchVnosDto"
        required: true
      responses:
        "201":
          description: Created
        "401":
          description: Not Authorized
        "403":
          description: Not Allowed
        "400":
          description: Bad Request
      summary: Nalozi
      security:
      - SecurityScheme: []
```

```
  /v1/javno/poizvedi:
    post:
      tags:
      - Vmesnik-VDB
      requestBody:
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/PoizvedbaDto"
        required: true
      responses:
        "200":
          description: OK
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: "#/components/schemas/RezultatPoizvedbeDto"
        "401":
          description: Not Authorized
        "403":
          description: Not Allowed
        "400":
          description: Bad Request
      summary: Poizvedi
      security:
      - SecurityScheme: []
info:
  title: vmesnik-vdb API
  version: 1.0.0-SNAPSHOT
```

Odsek kode A.2: Orodja MCP strežnika vmesnika VDB na `tools/list` klic

```
{
    "jsonrpc": "2.0",
    "id": 0,
    "result": {
        "tools": [
            {
                "name": "Dolgotrajen spomin",
                "description": "",
                "annotations": {
                    "destructiveHint": false,
                    "idempotentHint": false,
                    "openWorldHint": false,
                    "readOnlyHint": true
                },
```

```
                "inputSchema": {
                    "type": "object",
                    "properties": {
                        "opisIskaneVsebine": {
                            "type": "string",
                            "description": "Natančen opis iskane vsebine"
                        },
                        "steviloOdsekov": {
                            "type": "integer",
                            "description": "Število odsekov spomina, ki jih
                                želimo priklicati iz spomina."
                        }
                    },
                    "required": [
                        "opisIskaneVsebine",
                        "steviloOdsekov"
                    ]
                }
            },
            {
                "name": "Zapomni si informacijo",
                "description": "",
                "annotations": {
                    "destructiveHint": false,
                    "idempotentHint": false,
                    "openWorldHint": false,
                    "readOnlyHint": false
                },
                "inputSchema": {
                    "type": "object",
                    "properties": {
                        "informacija": {
                            "type": "string",
                            "description": "Informacija, ki bo shranjena v
                                spomin"
                        }
                    },
                    "required": [
                        "informacija"
                    ]
                }
            }
        ]
    }
}
```

A.2 Qdrant

Odsek kode A.3: Primer sheme telesa zahteve GRPC za ustvarjanje nove zbirke Qdrant za vmesnik VDB

```
{
    "collection_name": "zdruzeni-podatki",
    "hnsw_config": {
        "ef_construct": 128,
        "full_scan_threshold": 10000,
        "m": 0,
        "max_indexing_threads": 8,
        "memory": "Cached",
        "payload_m": 16,
        "quantization_config": {
            "turbo": {
                "memory": "Cached",
                "bits": "Bits4"
            }
        }
    },
    "vectors_config": {
        "params": {
            "datatype": "Float16",
            "distance": "Cosine",
            "size": 768
        }
    },
    "optimizers_config": {
        "deleted_threshold": 0.1,
        "indexing_threshold": 20000,
        "max_segment_size": 1000000,
        "vacuum_min_vector_number": 10000
    },
    "payload": {
        "memory": "Cached"
    },
    "replication_factor": 3,
    "shard_number": 3,
    "sharding_method": "Auto",
    "write_consistency_factor": 1
}
```

A.3 Envoy

Odsek kode A.4: Primer konfiguracije Envoy posrednika za dve Qdrant gruči

```
admin:
  address:
    socket_address:
      address: 0.0.0.0
      port_value: 19000

static_resources:
  listeners:
  - name: listener_qdrant_grpc
    address:
      socket_address:
        address: 0.0.0.0
        port_value: 6334
    filter_chains:
    - filters:
      - name: envoy.filters.network.http_connection_manager
        typed_config:
          "@type": type.googleapis.com/envoy.extensions.filters.network.
              http_connection_manager.v3.HttpConnectionManager
          stat_prefix: ingress_grpc
          codec_type: HTTP2
          route_config:
            name: qdrant_grpc
            virtual_hosts:
            - name: backend
              domains: ["*"]
              routes:
              - match: { prefix: "/" }
                route:
                  cluster: qdrant_gruce
          http_filters:
          - name: envoy.filters.http.header_to_metadata
            typed_config:
              "@type": type.googleapis.com/envoy.extensions.filters.http.
                  header_to_metadata.v3.Config
              request_rules:
                # Tukaj se nastavi pravilo prek katerega bomo promet
                    usmerili na pravilno gruco
                - header: "x-cluster-name"
                  on_header_present:
                    metadata_namespace: "envoy.lb"
                    key: "name"
                    type: STRING
```

```
41                  on_header_missing:
42                    metadata_namespace: "envoy.lb"
43                    key: "name"
44                    value: "qdrant-osnovni"
45                    type: STRING
46          - name: envoy.filters.http.router
47            typed_config:
48              "@type": type.googleapis.com/envoy.extensions.filters.http.
                    router.v3.Router
49
50  clusters:
51  - name: qdrant_gruce
52    connect_timeout: 0.25s
53    type: STRICT_DNS
54    lb_policy: ROUND_ROBIN
55
56    http2_protocol_options: {}
57    lb_subset_config:
58      fallback_policy: ANY_ENDPOINT
59      subset_selectors:
60      - keys:
61        - name
62    load_assignment:
63      cluster_name: qdrant_gruce
64      endpoints:
65      - lb_endpoints:
66        # Tukaj se nastavi naslove obeh gruc in metapodatke za usmerjanje
67        - endpoint:
68            address:
69              socket_address: { address: qdrant-cluster-osnovni-headless.
                    qdrant.svc.cluster.local, port_value: 6334 }
70          metadata:
71            filter_metadata:
72              envoy.lb:
73                name: "qdrant-osnovni"
74        - endpoint:
75            address:
76              socket_address: { address: qdrant-cluster-zlati-headless.
                    qdrant.svc.cluster.local, port_value: 6334 }
77          metadata:
78            filter_metadata:
79              envoy.lb:
80                name: "qdrant-zlati"
```

Dodatek B

Tabele meritev

Dokument s tabelami z vsemi rezultati je dostopen na `https://github.com/peter-savron/vector-db-benchmark/blob/master/Rezultati%20testiranja%20-%20tabele.pdf`.

B.1 Testi nastavitev kazal nad tovorom

Vse meritve so bile izvedene na gruči treh vozlišč, vsako z 2 CPU in 4 GB RAM. Pri vseh meritvah smo ob gradnji uporabljali `ef_construct = 128`.

Tabela B.1: Gruča brez kazala nad tovorom

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
`p`: 1 `ef`: 128	0,5542	0,134	0,032	7,40	0,180
`p`: 1 `ef`: 256	0,5542	0,134	0,033	7,38	0,183
`p`: 1 `ef`: 512	0,5542	0,134	0,032	7,38	0,181
`p`: 8 `ef`: 128	0,5542	0,687	0,334	11,61	1,112
`p`: 8 `ef`: 256	0,5542	0,682	0,301	11,65	1,104
`p`: 8 `ef`: 512	0,5542	0,678	0,258	11,76	1,046
`p`: 64 `ef`: 128	/	/	/	/	/
`p`: 64 `ef`: 256	/	/	/	/	/
`p`: 64 `ef`: 512	/	/	/	/	/

Gruča brez kazala nad tovorom. Dodatne nastavitve HNSW kazala: $m = 16$, `payload_m = 0`. Čas nalaganja: 104,10 s. Okviren čas gradnje kazala: 907,75 s. Pri 64 sočasnih poizvedbah so bile povezave prekinjene, ker so presegle 30 s.

Tabela B.2: Gruča s kazalom nad tovorom, brez podgrafa HNSW

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
`p`: 1 `ef`: 128	0,8884	0,015	0,003	61,55	0,018
`p`: 1 `ef`: 256	0,8884	0,016	0,004	59,52	0,019
`p`: 1 `ef`: 512	0,8884	0,015	0,004	61,80	0,018
`p`: 8 `ef`: 128	0,8884	0,031	0,015	249,03	0,058
`p`: 8 `ef`: 256	0,8884	0,029	0,015	267,57	0,053
`p`: 8 `ef`: 512	0,8884	0,029	0,014	262,26	0,052
`p`: 64 `ef`: 128	0,8884	0,217	0,045	281,22	0,295
`p`: 64 `ef`: 256	0,8884	0,220	0,046	274,47	0,290
`p`: 64 `ef`: 512	0,8884	0,220	0,071	276,89	0,337

Gruča s kazalom nad tovorom, brez podgrafa HNSW. Dodatne nastavitve HNSW kazala: $m = 16$, `payload_m = 0`. Čas nalaganja: 99,63 s. Okviren čas gradnje kazala: 947,68 s.

Tabela B.3: Gruča s kazalom nad tovorom, s podgrafom HNSW

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
`p`: 1 `ef`: 128	0,9930	0,015	0,003	60,43	0,018
`p`: 1 `ef`: 256	0,9974	0,015	0,002	60,37	0,018
`p`: 1 `ef`: 512	0,9980	0,016	0,002	59,39	0,019
`p`: 8 `ef`: 128	0,9930	0,028	0,013	270,69	0,056
`p`: 8 `ef`: 256	0,9974	0,027	0,010	286,90	0,047
`p`: 8 `ef`: 512	0,9980	0,028	0,010	274,96	0,047
`p`: 64 `ef`: 128	0,9930	0,193	0,053	311,25	0,271
`p`: 64 `ef`: 256	0,9974	0,214	0,047	287,48	0,297
`p`: 64 `ef`: 512	0,9980	0,223	0,062	274,62	0,339

Gruča s kazalom nad tovorom, s podgrafom HNSW. Dodatne nastavitve HNSW kazala: $m = 0$, `payload_m = 16`. Čas nalaganja: 103,17 s. Okviren čas gradnje kazala: 55,26 s.

Tabela B.4: Gruča z najemniškim kazalom nad tovorom, s podgrafom HNSW

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
`p`: 1 `ef`: 128	0,9920	0,012	0,002	76,12	0,015
`p`: 1 `ef`: 256	0,9972	0,012	0,003	73,78	0,015
`p`: 1 `ef`: 512	0,9984	0,013	0,001	71,43	0,016
`p`: 8 `ef`: 128	0,9920	0,020	0,011	386,40	0,037
`p`: 8 `ef`: 256	0,9972	0,017	0,006	438,86	0,029
`p`: 8 `ef`: 512	0,9984	0,019	0,010	404,26	0,033
`p`: 64 `ef`: 128	0,9920	0,130	0,054	446,83	0,214
`p`: 64 `ef`: 256	0,9972	0,128	0,057	453,14	0,213
`p`: 64 `ef`: 512	0,9984	0,128	0,038	464,34	0,197

Gruča s kazalom nad tovorom, s pografom HNSW. Dodatne nastavitve HNSW kazala: $m = 0$, `payload_m = 16`. Čas nalaganja: 105,47 s, Okviren čas gradnje kazala: 40,18 s.

B.2 Testi nastavitev Qdrant gruče

Kjer ni navedeno drugače, so bile meritve izvedene na gruči treh vozlišč, vsako z 2 CPU in 4 GB RAM. Pri vseh meritvah smo ob gradnji uporabljali $m = 0$, `ef_construct = 128`, `payload_m = 16` in najemniško kazalo s podgrafom.

Tabela B.5: Gruča, 3 drobci in 1 replika

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
`p`: 1 `ef`: 128	0,9932	0,012	0,002	73,92	0,016
`p`: 1 `ef`: 256	0,9972	0,013	0,004	72,96	0,016
`p`: 1 `ef`: 512	0,9984	0,013	0,003	71,81	0,016
`p`: 8 `ef`: 128	0,9932	0,019	0,008	391,92	0,036
`p`: 8 `ef`: 256	0,9972	0,017	0,008	436,80	0,027
`p`: 8 `ef`: 512	0,9984	0,018	0,009	410,79	0,032
`p`: 64 `ef`: 128	0,9932	0,122	0,038	457,62	0,195
`p`: 64 `ef`: 256	0,9972	0,125	0,044	467,78	0,203
`p`: 64 `ef`: 512	0,9984	0,133	0,044	444,68	0,207

Gruča, 3 drobci in 1 replika. Čas nalaganja: 106,13 s, Okviren čas gradnje kazala: 40,22 s.

Tabela B.6: Gruča, 1 drobec in 3 replike

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
`p`: 1 `ef`: 128	0,9671	0,012	0,006	75,57	0,015
`p`: 1 `ef`: 256	0,9867	0,013	0,003	69,99	0,016
`p`: 1 `ef`: 512	0,9906	0,013	0,002	68,29	0,017
`p`: 8 `ef`: 128	0,9683	0,016	0,006	462,82	0,029
`p`: 8 `ef`: 256	0,9861	0,016	0,009	477,34	0,024
`p`: 8 `ef`: 512	0,9901	0,017	0,006	435,24	0,030
`p`: 64 `ef`: 128	0,9672	0,101	0,053	531,58	0,195
`p`: 64 `ef`: 256	0,9885	0,121	0,100	470,49	0,298
`p`: 64 `ef`: 512	0,9912	0,130	0,089	453,37	0,280

Gruča, 1 drobec in 3 replike. Čas nalaganja: 108,66 s, Okviren čas gradnje kazala: 345,93 s.

Tabela B.7: Samostojno vozlišče, 6 CPU, 12 GB RAM

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
`p`: 1 `ef`: 128	0,9594	0,009	0,001	98,33	0,011
`p`: 1 `ef`: 256	0,9856	0,010	0,003	89,68	0,012
`p`: 1 `ef`: 512	0,9904	0,010	0,001	86,22	0,013
`p`: 8 `ef`: 128	0,9594	0,016	0,006	478,86	0,030
`p`: 8 `ef`: 256	0,9856	0,011	0,003	658,48	0,018
`p`: 8 `ef`: 512	0,9904	0,012	0,003	622,84	0,018
`p`: 64 `ef`: 128	0,9594	0,091	0,033	579,96	0,162
`p`: 64 `ef`: 256	0,9856	0,104	0,036	556,59	0,177
`p`: 64 `ef`: 512	0,9904	0,115	0,039	510,86	0,194

Samostojno vozlišče, 6 CPU, 12 GB RAM. Čas nalaganja: 100,41 s, Okviren čas gradnje kazala: 75,13 s.

Tabela B.8: Gruča, 3 drobci in 3 replike. Vozlišča po 2 CPU in 1,5 GB RAM

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
`p`: 1 `ef`: 128	0,9939	0,052	0,057	18,78	0,186
`p`: 1 `ef`: 256	0,9981	0,052	0,061	18,67	0,192
`p`: 1 `ef`: 512	0,9994	0,057	0,069	17,24	0,217
`p`: 8 `ef`: 128	0,9934	0,126	0,166	62,52	0,526
`p`: 8 `ef`: 256	0,9977	0,125	0,189	63,21	0,500
`p`: 8 `ef`: 512	0,9994	0,112	0,161	61,73	0,493
`p`: 64 `ef`: 128	0,9938	1,028	1,516	59,67	4,800
`p`: 64 `ef`: 256	0,9979	0,939	1,280	65,17	4,163
`p`: 64 `ef`: 512	0,9989	0,946	1,314	64,81	3,458

Gruča, 3 drobci in 3 replike. Vozlišča po 2 CPU in 1,5 GB RAM. Čas nalaganja: 107,26 s. Okviren čas gradnje kazala: 181,23 s.

B.3 Testi nastavitev kvantizacije

Vse meritve so bile izvedene na samostojnem vozlišču z 2 CPU in 4 GB RAM. Pri vseh meritvah smo ob gradnji uporabljali $m = 0$, `ef_construct = 128`, `payload_m = 16` in najemniško kazalo s podgrafom.

Tabela B.9: Skalarna kvantizacija

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,9654	0,008	0,001	110,00	0,011
p: 1 ef: 256 v: 2,00	0,9878	0,008	0,001	109,24	0,010
p: 1 ef: 512 v: 2,00	0,9918	0,010	0,005	91,49	0,012
p: 8 ef: 128 v: 2,00	0,9654	0,021	0,015	368,16	0,065
p: 8 ef: 256 v: 2,00	0,9878	0,020	0,009	332,55	0,040
p: 8 ef: 512 v: 2,00	0,9918	0,023	0,010	327,92	0,043
p: 64 ef: 128 v: 2,00	0,9654	0,184	0,039	331,02	0,276
p: 64 ef: 256 v: 2,00	0,9878	0,211	0,077	289,82	0,302
p: 64 ef: 512 v: 2,00	0,9918	0,240	0,061	255,34	0,374

Čas nalaganja: 107,05 s, Okviren čas gradnje kazala: 110,22 s.

Tabela B.10: Kvantizacija produkta na 2 bita

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,8524	0,008	0,001	107,28	0,011
p: 1 ef: 256 v: 2,00	0,8766	0,008	0,001	107,46	0,010
p: 1 ef: 512 v: 2,00	0,8842	0,009	0,002	101,84	0,011
p: 8 ef: 128 v: 2,00	0,8524	0,024	0,017	323,35	0,075
p: 8 ef: 256 v: 2,00	0,8766	0,021	0,009	312,27	0,039
p: 8 ef: 512 v: 2,00	0,8842	0,027	0,011	283,12	0,046
p: 64 ef: 128 v: 2,00	0,8524	0,207	0,058	296,75	0,305
p: 64 ef: 256 v: 2,00	0,8766	0,226	0,075	273,72	0,393
p: 64 ef: 512 v: 2,00	0,8842	0,241	0,071	254,81	0,391

Čas nalaganja: 101,84 s, Okviren čas gradnje kazala: 545,76 s.

Tabela B.11: Rotacijska kvantizaija na 4 bite

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,9626	0,008	0,004	109,42	0,010
p: 1 ef: 256 v: 2,00	0,9888	0,008	0,002	107,38	0,011
p: 1 ef: 512 v: 2,00	0,9974	0,009	0,002	94,79	0,012
p: 8 ef: 128 v: 2,00	0,9626	0,021	0,016	357,86	0,066
p: 8 ef: 256 v: 2,00	0,9888	0,022	0,010	341,43	0,043
p: 8 ef: 512 v: 2,00	0,9974	0,023	0,010	338,17	0,042
p: 64 ef: 128 v: 2,00	0,9626	0,200	0,045	305,31	0,295
p: 64 ef: 256 v: 2,00	0,9888	0,213	0,055	285,83	0,307
p: 64 ef: 512 v: 2,00	0,9974	0,239	0,064	257,37	0,385

Čas nalaganja: 105,58 s, Okviren čas gradnje kazala: 200,29 s.

Tabela B.12: Rotacijska kvantizacija na 1 bit

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,5322	0,008	0,004	108,74	0,010
p: 1 ef: 256 v: 2,00	0,5540	0,008	0,001	113,94	0,010
p: 1 ef: 512 v: 2,00	0,5598	0,008	0,002	113,06	0,010
p: 8 ef: 128 v: 2,00	0,5322	0,019	0,016	395,05	0,065
p: 8 ef: 256 v: 2,00	0,5540	0,019	0,012	390,24	0,040
p: 8 ef: 512 v: 2,00	0,5598	0,021	0,010	360,94	0,041
p: 64 ef: 128 v: 2,00	0,5322	0,185	0,043	330,92	0,280
p: 64 ef: 256 v: 2,00	0,5540	0,193	0,050	312,48	0,296
p: 64 ef: 512 v: 2,00	0,5598	0,204	0,067	299,61	0,336

Čas nalaganja: 108,40 s, Okviren čas gradnje kazala: 185,32 s.

B.4 Testi izkoristka CPU

Vse meritve so bile izvedene na samostojnem vozlišču s 4 GB RAM. Pri vseh meritvah smo ob gradnji uporabljali $m = 0$, `ef_construct = 128`, `payload_m = 16` in najemniško kazalo s podgrafom.

Tabela B.13: Rotacijska kvantizacija na 4 bite, 4 CPU

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,9588	0,008	0,003	105,08	0,012
p: 1 ef: 256 v: 2,00	0,9852	0,008	0,003	106,83	0,011
p: 1 ef: 512 v: 2,00	0,9934	0,009	0,002	102,17	0,011
p: 8 ef: 256	0,8248	0,012	0,005	594,76	0,022
p: 8 ef: 256 v: 1,00	0,9850	0,013	0,006	585,86	0,023
p: 8 ef: 256 v: 2,00	0,9852	0,013	0,008	562,20	0,025
p: 8 ef: 256 v: 4,00	0,9852	0,014	0,006	523,46	0,024
p: 8 ef: 256 v: 8,00	0,9852	0,014	0,005	546,04	0,026
p: 8 ef: 256 v: 16,00	0,9916	0,017	0,006	443,26	0,031
p: 8 ef: 256 v: 32,00	0,9942	0,019	0,006	349,80	0,032
p: 64 ef: 256 v: 2,00	0,9852	0,119	0,039	488,96	0,197

Čas nalaganja: 105,65 s, Okviren čas gradnje kazala: 160,32 s.

Tabela B.14: Rotacijska kvantizacija na 4 bite, 1 CPU

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,9636	0,009	0,006	101,10	0,012
p: 1 ef: 256 v: 2,00	0,9880	0,009	0,004	96,54	0,012
p: 1 ef: 512 v: 2,00	0,9948	0,008	0,003	107,38	0,011
p: 8 ef: 256	0,8292	0,034	0,017	228,59	0,065
p: 8 ef: 256 v: 1,00	0,9864	0,037	0,018	208,61	0,068
p: 8 ef: 256 v: 2,00	0,9880	0,039	0,019	200,67	0,071
p: 8 ef: 256 v: 4,00	0,9880	0,039	0,018	201,55	0,070
p: 8 ef: 256 v: 8,00	0,9880	0,043	0,020	183,13	0,075
p: 8 ef: 256 v: 16,00	0,9938	0,050	0,020	157,79	0,081
p: 8 ef: 256 v: 32,00	0,9956	0,059	0,020	133,76	0,093
p: 64 ef: 256 v: 2,00	0,9880	0,309	0,080	200,85	0,492

Čas nalaganja: 120,39 s, Okviren čas gradnje kazala: 360,59 s.

Tabela B.15: Rotacijska kvantizacija na 4 bite, 0,5 CPU

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,9602	0,011	0,008	84,31	0,034
p: 1 ef: 256 v: 2,00	0,9892	0,011	0,009	80,45	0,037
p: 1 ef: 512 v: 2,00	0,9952	0,012	0,010	74,26	0,041
p: 8 ef: 256	0,8280	0,097	0,031	81,59	0,111
p: 8 ef: 256 v: 1,00	0,9878	0,102	0,031	77,34	0,191
p: 8 ef: 256 v: 2,00	0,9892	0,102	0,032	77,63	0,192
p: 8 ef: 256 v: 4,00	0,9892	0,107	0,031	74,06	0,193
p: 8 ef: 256 v: 8,00	0,9892	0,117	0,039	67,84	0,196
p: 8 ef: 256 v: 16,00	0,9946	0,140	0,046	56,75	0,199
p: 8 ef: 256 v: 32,00	0,9956	0,171	0,047	46,50	0,204
p: 64 ef: 256 v: 2,00	0,9892	0,798	0,141	78,62	1,009

Čas nalaganja: 228,00 s. Okviren čas gradnje kazala: 681,86 s.

B.5 Testi porabe pomnilnika

Vse meritve so bile izvedene na samostojnem vozlišču z 2 CPU. Pri vseh meritvah smo ob gradnji uporabljali $m = 0$, `ef_construct = 128`, `payload_m = 16` in najemniško kazalo s podgrafom, razen kjer je izrecno napisano drugače.

Tabela B.16: Rotacijska kvantizacija na 4 bite, 2,2 GB RAM

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,9634	0,009	0,004	96,60	0,017
p: 1 ef: 256 v: 2,00	0,9876	0,008	0,001	109,34	0,011
p: 1 ef: 512 v: 2,00	0,9934	0,009	0,005	101,98	0,011
p: 8 ef: 256	0,8286	0,020	0,011	384,78	0,040
p: 8 ef: 256 v: 1,00	0,9860	0,019	0,010	355,16	0,039
p: 8 ef: 256 v: 2,00	0,9876	0,022	0,012	347,55	0,044
p: 8 ef: 256 v: 4,00	0,9876	0,025	0,016	308,15	0,056
p: 8 ef: 256 v: 8,00	0,9876	0,031	0,026	248,38	0,089
p: 8 ef: 256 v: 16,00	0,9922	0,039	0,042	198,85	0,146
p: 8 ef: 256 v: 32,00	0,9946	0,182	0,098	43,71	0,373
p: 64 ef: 256 v: 2,00	0,9876	0,206	0,101	298,46	0,434

Čas nalaganja: 108,18 s, Okviren čas gradnje kazala: 230,38 s.

Tabela B.17: Rotacijska kvantizacija na 2 bita, 2,2 GB RAM

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,8248	0,007	0,002	119,42	0,013
p: 1 ef: 256 v: 2,00	0,8678	0,007	0,002	121,17	0,010
p: 1 ef: 512 v: 2,00	0,8888	0,008	0,003	109,55	0,010
p: 8 ef: 128	0,4714	0,017	0,015	432,08	0,058
p: 8 ef: 128 v: 1,00	0,7184	0,016	0,009	472,09	0,037
p: 8 ef: 128 v: 2,00	0,8248	0,016	0,009	457,51	0,038
p: 8 ef: 128 v: 4,00	0,8754	0,020	0,013	381,69	0,047
p: 8 ef: 128 v: 8,00	0,9268	0,024	0,017	317,62	0,062
p: 8 ef: 128 v: 16,00	0,9686	0,034	0,032	226,55	0,116
p: 8 ef: 128 v: 32,00	0,9842	0,056	0,057	139,84	0,188
p: 8 ef: 256	0,4856	0,017	0,009	444,26	0,038
p: 8 ef: 256 v: 1,00	0,7536	0,018	0,009	425,91	0,038
p: 8 ef: 256	0,8678	0,018	0,010	420,36	0,039

Tabela B.18: Skalarna kvantizacija, 1,2 GB RAM

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,9682	0,008	0,005	107,33	0,014
p: 1 ef: 256 v: 2,00	0,9864	0,008	0,001	108,86	0,010
p: 1 ef: 512 v: 2,00	0,9932	0,008	0,001	104,74	0,011
p: 8 ef: 256	0,9626	0,024	0,016	321,09	0,069
p: 8 ef: 256 v: 1,00	0,9864	0,025	0,015	310,03	0,056
p: 8 ef: 256 v: 2,00	0,9864	0,026	0,014	299,33	0,055
p: 8 ef: 256 v: 4,00	0,9864	0,115	0,043	68,89	0,193
p: 8 ef: 256 v: 8,00	0,9864	0,193	0,102	41,09	0,351
p: 8 ef: 256 v: 16,00	0,9920	0,324	0,155	24,54	0,611
p: 8 ef: 256 v: 32,00	0,9942	0,567	0,302	14,08	1,155
p: 64 ef: 256 v: 2,00	0,9864	0,232	0,115	262,19	0,544

Čas nalaganja: 107,29 s, Okviren čas gradnje kazala: 230,39 s.

Tabela B.19: Rotacijska kvantizacija na 4 bite, 1,2 GB RAM

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,9656	0,008	0,005	104,91	0,014
p: 1 ef: 256 v: 2,00	0,9906	0,008	0,002	103,92	0,011
p: 1 ef: 512 v: 2,00	0,9986	0,009	0,002	102,46	0,011
p: 8 ef: 256	0,8264	0,023	0,019	338,91	0,070
p: 8 ef: 256 v: 1,00	0,9892	0,024	0,014	324,72	0,053
p: 8 ef: 256 v: 2,00	0,9906	0,024	0,014	321,59	0,054
p: 8 ef: 256 v: 4,00	0,9906	0,086	0,033	91,45	0,134
p: 8 ef: 256 v: 8,00	0,9906	0,180	0,077	44,19	0,331
p: 8 ef: 256 v: 16,00	0,9972	0,287	0,136	24,29	0,538
p: 8 ef: 256 v: 32,00	1,0000	0,562	0,296	14,17	1,120
p: 64 ef: 256 v: 2,00	0,9906	0,216	0,112	280,99	0,511

Čas nalaganja: 107,28 s, Okviren čas gradnje kazala: 285,46 s.

Tabela B.20: Rotacijska kvantizacija na 4 bite, 1,2 GB RAM, brez rabe kazala

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	1,0000	0,008	0,002	115,15	0,014
p: 1 ef: 256 v: 2,00	1,0000	0,007	0,001	131,40	0,009
p: 1 ef: 512 v: 2,00	1,0000	0,006	0,001	136,25	0,009
p: 8 ef: 256	0,8358	0,014	0,009	538,21	0,036
p: 8 ef: 256 v: 1,00	0,9992	0,016	0,012	474,95	0,039
p: 8 ef: 256 v: 2,00	1,0000	0,017	0,013	434,80	0,041
p: 8 ef: 256 v: 4,00	1,0000	0,020	0,019	373,28	0,069
p: 8 ef: 256 v: 8,00	1,0000	0,148	0,064	53,68	0,262
p: 8 ef: 256 v: 16,00	1,0000	0,281	0,162	28,29	0,530
p: 8 ef: 256 v: 32,00	1,0000	0,464	0,233	17,20	0,916
p: 64 ef: 256 v: 2,00	1,0000	0,145	0,043	409,96	0,208

Čas nalaganja: 105,38 s, Okviren čas gradnje kazala: 195,31 s.

Tabela B.21: Rotacijska kvantizacija na 2 bita, 1,2 GB RAM

Nastavitve poizvedbe	Priklic	Povprečna latenca (s)	Standardni odklon (s)	Prepustnost (RPS)	p95 latenca (s)
p: 1 ef: 128 v: 2,00	0,8214	0,008	0,004	111,35	0,016
p: 1 ef: 256 v: 2,00	0,8744	0,008	0,002	108,30	0,012
p: 1 ef: 512 v: 2,00	0,8974	0,008	0,004	106,10	0,011
p: 8 ef: 256	0,4898	0,021	0,016	370,26	0,061
p: 8 ef: 256 v: 1,00	0,7588	0,022	0,014	339,74	0,049
p: 8 ef: 256 v: 2,00	0,8744	0,022	0,013	350,47	0,048
p: 8 ef: 256 v: 4,00	0,9340	0,077	0,030	102,18	0,127
p: 8 ef: 256 v: 8,00	0,9518	0,166	0,074	47,87	0,302
p: 8 ef: 256 v: 16,00	0,9782	0,288	0,139	27,67	0,539
p: 8 ef: 256 v: 32,00	0,9926	0,484	0,240	16,48	0,950
p: 64 ef: 256 v: 2,00	0,8744	0,212	0,122	289,21	0,507

Čas nalaganja: 105,80 s, Okviren čas gradnje kazala: 240,37 s.